

—◆—
WEEKS 1 - 12

Software Engineering

—◆—
A Complete Study Guide

—◆—
Made by Alok Tripathi

DISCLAIMER

This document is an **unofficial, student-made study guide**, prepared **solely for personal exam preparation purposes**. It is **not affiliated with, endorsed by, or officially connected to** IIT Madras, the IITM BS Degree Programme, or the *Software Engineering* course team in any way.

The conceptual content in this guide has been derived from the **official course slides and lecture transcripts** of the *Software Engineering* course (Dr. Sridhar Iyer, IIT Bombay, and Dr. Prajish Prasad, FLAME University). All rights to the original slides, lectures, and course material remain with their **official owners and creators**. No copyright over the original material is claimed by the maker of this guide.

Every effort has been made to keep the explanations, examples, and solved questions in this guide accurate, but the guide has **not been reviewed or approved** by the course instructors or IIT Madras, and it may contain errors, omissions, or interpretations that differ from the official course material or the official answer keys.

Please do not rely solely on this guide for your exam preparation. Use it alongside, and always defer to, the official course slides, lectures, and other materials provided by IIT Madras. This guide is offered purely as a supplementary aid, in good faith, for fellow students.

About This Volume

Scope of this Iteration

This volume covers **Week 1** (Deconstructing the Software Development Process), **Week 2** (Software Requirements), **Week 3** (Software User Interfaces), **Week 4** (Software Project Management), **Week 5** (Software Design), **Week 7** (Software Development), **Week 8** (Software Architecture), **Week 9** (Software Testing), **Week 10** (Software Deployment and Monitoring), and **Week 11** (Other Aspects in Software Engineering) of the IITM BS Degree *Software Engineering* course, exactly as defined by the official course slides. **Weeks 6 and 12 had no published slide content** and are therefore skipped, chapter numbers below run consecutively (1–10) while the corresponding week numbers jump from 5 to 7, and end at 11. This volume now covers the complete course, end to end.

Chapter	Corresponds to
1	Week 1 – Deconstructing the Software Development Process
2	Week 2 – Software Requirements
3	Week 3 – Software User Interfaces
4	Week 4 – Software Project Management
5	Week 5 – Software Design
6	Week 7 – Software Development (<i>Week 6 had no content</i>)
7	Week 8 – Software Architecture
8	Week 9 – Software Testing
9	Week 10 – Software Deployment and Monitoring
10	Week 11 – Other Aspects in Software Engineering (<i>Week 12 had no content</i>)

How the Previous Year Questions (PYQs) were used. Eleven Quiz-2 papers and eighteen End-Term papers (Sep 2022 – Jan 2026) were analysed. Every question that tested a concept covered in this volume was identified and mapped back to the relevant section, existing sections were *enriched* with new End-Term insights rather than duplicated. Topics that are tested repeatedly are flagged with a

★ FREQUENTLY ASKED IN PYQs

banner and a star-rating:

★★★	High frequency	tested in most/all of the 11 papers
★★☆	Medium frequency	tested in roughly a third to half of the papers
★☆☆	Low frequency	tested occasionally, but still worth knowing

How to read a chapter. Every important concept in this guide follows the same eight-part structure: *Motivation* → *Real-life Analogy* → *Formal Definition* → *Detailed Explanation* → *Worked Examples* → *Quiz-Oriented Notes* → *Memory Tricks* → special highlight boxes (*Intuition*, *Common Mistake*, *Exam Tip*, *Remember This*). Each chapter closes with a Quick Revision sheet, a Quiz Preparation set, Practice Questions, and a coverage checklist confirming that every slide has been expanded upon.

Assumed background. None. Every technical term, every programming idea, and every piece of jargon is explained the first time it is used, right from graphs and functions to Agile ceremonies – so this guide can be read start to finish by someone who has never studied software engineering before.

Contents

About This Volume	ii
1 Deconstructing the Software Development Process	1
1.1 Software as a Collection of Components	1
1.2 Requirement Specification and Clients	4
1.2.1 The First Step: Studying Before Building	4
1.2.2 Who is a Client? Three Types of Clients	4
1.3 Software Design and Development	6
1.3.1 The Development Phase	7
1.4 Software Testing and Maintenance	8
1.4.1 The Cost of Skipping Testing	9
1.4.2 Levels of Testing	9
1.4.3 Alpha Testing and Beta Testing	9
1.4.4 The Maintenance Phase	10
1.5 The Software Development Lifecycle (SDLC)	11
1.5.1 The Five Core Phases, Recap	12
1.6 Software Development Models: The Plan-and-Document Perspective	12
1.6.1 The Waterfall Model	13
1.6.2 The Prototype Model	14
1.6.3 The Spiral Model	15
1.7 Software Development Models: The Agile Perspective	16
1.7.1 The Agile Manifesto	17

1.7.2	Incremental Development and the Agile Cycle	17
1.7.3	Three Named Agile Approaches	18
1.7.4	Core Agile / Scrum Practices	18
1.7.5	The Agile Philosophy	20
1.7.6	Choosing Between Plan-and-Document and Agile	21
2	Software Requirements	24
2.1	Requirement Gathering and Analysis	24
2.2	Identifying Users and Requirements	25
2.2.1	Common Problems Found During Requirement Analysis	27
2.3	Requirement Gathering Techniques	28
2.3.1	The Five Techniques	28
2.3.2	Summary Table	31
2.3.3	Basic Requirement-Gathering Guidelines	31
2.3.4	Worked Example: Identified Requirements for the Seller Portal	31
2.4	Functional and Non-Functional Requirements	32
2.4.1	Reliability and Robustness	33
2.4.2	Comparison Table: Functional vs. Non-Functional	33
2.5	The Software Requirements Specification (SRS) Document	35
2.5.1	What an SRS Document Contains	35
2.5.2	Advantages of Maintaining an SRS	36
2.5.3	The Drawback of the SRS Approach	36
2.6	Behaviour Driven Design and User Stories	37
2.6.1	Behaviour Driven Design (BDD)	37
2.6.2	User Stories	37
2.6.3	Benefits of User Stories	38
2.6.4	SMART User Stories	39
2.6.5	Drawbacks of User Stories	40
3	Software User Interfaces	43
3.1	Introduction to Interaction Design	43
3.2	Usability Goals	44
3.3	User Experience (UX) Goals	46
3.4	Prototyping Techniques	46
3.4.1	The Fidelity Spectrum: Four Named Prototype Types	47
3.4.2	Benefits of Storyboards	48
3.4.3	Benefits of Paper Prototypes	48
3.5	Heuristic Evaluation	49
3.5.1	Heuristics for Understanding	49
3.5.2	Heuristics for Action	50
3.5.3	Heuristics for Feedback	51
4	Software Project Management	54
4.1	Project Management Overview	54
4.2	Project Estimation Techniques	55

4.2.1	Empirical Technique 1: Expert Judgement	55
4.2.2	Empirical Technique 2: The Delphi Technique	55
4.3	The COCOMO Estimation Model	56
4.3.1	Worked Example: The Amazon Seller Portal	57
4.3.2	Refining the Estimate: Cost Drivers and the Effort Adjustment Factor	57
4.3.3	Worked Examples: Organic and Embedded Projects	58
4.3.4	COCOMO Model Summary	59
4.4	Project Scheduling	59
4.4.1	Main Activities in Scheduling	59
4.4.2	Work Breakdown Structure (WBS)	60
4.4.3	Activity Networks and the Critical Path	60
4.4.4	Gantt Charts	61
4.5	Risk Management	62
4.5.1	Risk Assessment: Building a Risk Table	64
4.6	Project Management in Agile	65
4.6.1	Iterations and Team Structure	65
4.6.2	The Scrum Team	65
4.6.3	Sprint Planning and the Product Backlog	65
4.6.4	The Recurring Ceremonies	66
4.6.5	Scheduling in Agile: Points and Velocity	66
4.6.6	Plan-and-Document vs. Agile Scheduling, Side by Side	67
5	Software Design	69
5.1	Outcomes of the Design Process	69
5.1.1	Characterising a Good Software Design	69
5.2	Design Modularity: Coupling and Cohesion	70
5.2.1	Four Types of Coupling	71
5.2.2	Five Types of Cohesion	72
5.2.3	Connecting Cohesion/Coupling to the “Good Design” Question	73
5.3	Object-Oriented Design: Basic Concepts	73
5.4	Class Relationships	74
5.5	UML Diagrams	75
5.5.1	Purposes of UML Modelling	76
6	Software Development	79
6.1	Development Environments, IDEs, and Frameworks	79
6.2	Version Control	80
6.3	Debugging	80
6.3.1	Error, Failure, and Fault	81
6.3.2	Steps in Debugging	81
6.3.3	Debugging Techniques	82
6.3.4	Debugging Strategies	82
6.4	Software Metrics	83
6.5	Writing Clean Code	84
6.5.1	Code Smells	84

6.5.2	Refactoring	85
7	Software Architecture	87
7.1	Software Architecture and Architecture Styles	87
7.1.1	Components, Connectors, and Design Patterns (Overview)	88
7.2	SOLID Principles	89
7.2.1	S: Single Responsibility Principle (SRP)	89
7.2.2	O: Open-Closed Principle (OCP)	90
7.2.3	L: Liskov Substitution Principle (LSP)	90
7.2.4	I: Interface Segregation Principle (ISP)	91
7.2.5	D: Dependency Inversion Principle (DIP)	91
7.3	Design Patterns	91
7.3.1	Creational: Factory Design Pattern	92
7.3.2	Creational: Builder Design Pattern	92
7.3.3	Structural: Facade Design Pattern	93
7.3.4	Structural: Adapter Design Pattern	93
7.3.5	Behavioural: Iterator Design Pattern	94
7.3.6	Behavioural: Observer Design Pattern	94
7.3.7	Behavioural: Strategy Design Pattern	94
8	Software Testing	97
8.1	Testing: Motivation and Terminology	97
8.2	Designing Test Cases: Blackbox and Whitebox Testing	98
8.2.1	Blackbox Testing: Equivalence Classes	98
8.2.2	Blackbox Testing: Boundary Value Analysis	99
8.2.3	Whitebox Testing: Coverage-Based Testing	100
8.3	Levels of Testing, Revisited	101
8.3.1	Unit Testing, In Detail	101
8.3.2	Integration Testing: Four Approaches	102
8.3.3	System, Acceptance, and Other Testing Types	104
8.4	Test-Driven Development (TDD)	104
9	Software Deployment and Monitoring	108
9.1	Deployment Environments	108
9.2	Deployment Strategies	109
9.3	Deployment Hosting	111
9.4	Continuous Integration	112
9.5	Performance Optimisation and Monitoring	113
9.5.1	Three Ways to Improve Performance	114
9.5.2	Monitoring	115
10	Other Aspects in Software Engineering	118
10.1	Software Organisations	118
10.2	Communication, Collaboration, and Productivity	120
10.2.1	Knowledge-Sharing Tools	120

10.2.2 Productivity	121
10.3 What Makes a Great Software Engineer? Decision Making	122

Deconstructing the Software Development Process

In this chapter: Software as components · Requirement specification and clients · Software design and development · Testing and maintenance · The software development lifecycle (SDLC) · Plan-and-document models (Waterfall, Prototype, Spiral) · The Agile perspective (Manifesto, XP/Scrum/Kanban, User Stories) · Choosing between Plan-and-Documents and Agile.

1.1 Software as a Collection of Components

► MOTIVATION

Imagine trying to build a car by carving it out of a single block of metal, engine, wheels, seats, dashboard, all as one inseparable lump. It would be almost impossible to design, impossible to repair, and impossible for more than one mechanic to work on at the same time. Real cars are not built this way: they are built from an engine, a chassis, wheels, a braking system, and hundreds of other parts, each designed, built, tested, and replaced independently, but all working together as a whole.

Large software systems face exactly the same problem. A system like Amazon is enormous, it recommends products, manages inventory, processes payments, handles shipping, calculates taxes, and much more. If all of this were written as a single undifferentiated mass of code, no team could design it, build it, or fix it without everyone stepping on everyone else's toes. Software engineering begins with a simple but powerful idea: **break the system into components**.

REAL-LIFE ANALOGY

Think of a large shopping mall. The mall as a whole is one building, but inside it there are independent stores, a food court, a parking structure, and a security office. Each of these can be renovated, restocked, or repaired without shutting down the entire mall, as long as the corridors and doorways connecting them (their *interfaces*) keep working the way shoppers expect. A software system is similar: the overall application is like the mall, and each component (inventory management, payments,

recommendations) is like a separate store, independently maintained but connected through well-defined doorways.

DEFINITION

A **software component** (also called a **module**) is a separately addressable, self-contained unit of a larger software system that performs a specific, well-defined piece of functionality, and that can be developed, tested, and modified largely independently of the other components, as long as it continues to interact correctly with them.

Detailed Explanation

Consider Amazon as a running example throughout this chapter, exactly as used in the original course material. Two of its many components are:

- **Inventory Management System.** This component keeps track of how much stock exists for every product. It updates itself automatically based on current purchases and seasonal buying trends, and it customises what a shopper sees on their homepage based on that shopper's browsing and purchase history.
- **Payment Gateway.** This component is responsible for securely collecting payment details from a customer, communicating with banks or card networks, and confirming whether a transaction succeeded or failed.

Notice something important: a person shopping on Amazon almost never thinks about the fact that “Inventory Management” and “Payment Gateway” are separate pieces of software. From the outside, Amazon just looks like one website. The decomposition into components is an *internal engineering decision*, it exists purely to make the system possible to build, test, and maintain. This is one of the most important mental shifts a new software engineer must make: stop thinking of “the app” as one thing, and start thinking of it as a federation of cooperating components.

★ INTUITION

Why not just have one giant program? Because complexity grows far faster than size. A program with 10 tangled responsibilities does not have “10 units” of complexity, it can have closer to $10 \times 9 = 90$ units of complexity, because every part might interact with every other part. Splitting the system into independent components with narrow, well-defined interfaces is the single most effective tool software engineers have for taming this explosion of complexity.

✗ COMMON MISTAKE

Students sometimes assume “component” means the same thing as “a single class” or “a single file.” In this course, a component/module is a larger, functionally coherent unit, it can (and usually does) consist of many classes, functions, and files. The defining property of a component is that it addresses one recognisable piece of the system's functionality (e.g. *all* of payments) and can be reasoned about, replaced, or scaled independently.

QUIZ-ORIENTED NOTES

- Software can be divided into separately addressable components called **modules**, which are integrated to satisfy the requirements of the overall system.
- The Amazon example: Inventory Management System and Payment Gateway are two components of one large system.
- A component is defined by *what it does*, not by how many files or classes implement it.
- Breaking software into components lets **different teams own different components**, and lets each component **conceal its implementation details** behind an interface. It does *not* require, and in fact defeats the purpose if it did require, that every developer on the team knows how to work with *every* component, that would recreate exactly the complexity components are meant to eliminate.

■ REMEMBER THIS

A large software system is not one program, it is a team of cooperating components, each solving one well-defined piece of the puzzle.

1.2 Requirement Specification and Clients

► MOTIVATION

Suppose you are asked to build “a new component” for a large system, but nobody tells you what it should do, who will use it, or why it is needed. You could still write code, but there is a very high chance it would solve the wrong problem. Before a single line of code is written, software engineers must answer a much more basic question: *what exactly are we building, and for whom?* This is the job of the very first phase of software development.

REAL-LIFE ANALOGY

Think of an architect commissioned to design a building. Before drawing a single wall, the architect asks: who will live or work here? A hospital, a school, and a nightclub all need very different buildings, even though all three are “buildings.” The architect’s very first task is to write down what the client actually needs. Software requirement specification plays exactly this role for software.

1.2.1 The First Step: Studying Before Building

Before designing a brand-new component, engineers first (a) study the **existing components** of the system, so the new piece will interact correctly with what already exists, and (b) look at **similar systems already in the market** (e.g., when building a payments feature, studying how PayTM or PhonePe already solve the problem) to understand what features are expected. Only after this groundwork is complete does the team move to writing down formal requirements.

DEFINITION

Requirements are the goals that the implemented system should achieve. They must cater to the needs of the system’s **clients**.

1.2.2 Who is a Client? Three Types of Clients

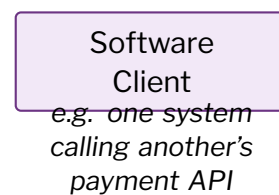
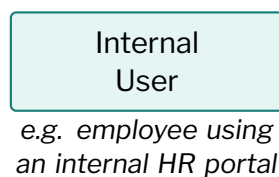
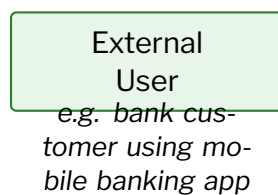
★ FREQUENTLY ASKED IN PYQs

Client-type identification questions (“is this an external user, internal user, or software client?”) appear in **almost every single** quiz paper analysed (Jan 2023, Jan 2024, Jan 2025, May 2023, May 2024, Sep 2022, Sep 2024, Sep 2025). ★★ ★

A “client” in software engineering is not always a paying customer. The course identifies three distinct kinds of clients, and distinguishing between them is one of the most frequently tested skills in this unit.

1.2. Requirement Specification and Clients

Client type	Definition	Example
External user	A person outside the developing organisation who directly uses the software.	Mobile banking software serves the bank's customers.
Internal user	A person or team <i>inside</i> the developing company who uses the software.	An internal "employee resource portal" built by the internal products team, for the company's own employees.
Software client	Another piece of software (not a human at all) that consumes the system through an interface.	A payment gateway that is called by an e-commerce system.



WORKED EXAMPLE

Scenario (adapted from Jan 2023, Q235). An online flower store lets customers order bouquets. Once an order is placed, the store manager phones a courier company to pick up and deliver the order. What type of client is the courier service provider?

Reasoning. The courier company does not use the flower-ordering software at all, it is contacted by phone. But it is directly affected by, and involved in fulfilling, the outcome of an order placed through the system. This makes it a **tertiary user** (a concept formally introduced in Chapter 2, Section 2.2, do not confuse "client" from this section with "tertiary user"; a tertiary user does not use the software but is impacted by it, whereas the three *client* categories here are about who/what directly interacts with a system through requirements or interfaces).

WORKED EXAMPLE

Scenario (adapted from Jan 2024 Q39 / Sep 2022 Q115). A company builds a web service that finds the optimal delivery route between two locations (considering cost, distance, traffic). Food-delivery and courier apps integrate this web service into their own apps. What type of client is being described?

Reasoning. The "client" here is not a human at all, it is another app/system that calls this web service through an interface (an API). This is a **software client** (a software or software component).

✗ COMMON MISTAKE

The most common trap in PYQs is confusing an *internal* user with a *software* client. Ask yourself: is a *human being* using this, and are they part of the company that built it, or outside it? If instead *another program* is calling this system automatically (no human directly involved at that moment), it is a software client, regardless of who ultimately benefits.

EXAM TIP

When a PYQ scenario says “*used by another organisation for its own employees*” (e.g. Sep 2024 Q2, a leave-management app sold to be used by other companies’ employees), be careful: those employees are *external users* from the point of view of the software company that built the app, even though they are “internal” employees of *their own* company. Always ask the question from the perspective of the organisation *developing* the software.

✗ COMMON MISTAKE

A “select all valid clients” style question (e.g. End Term, digital wallet system) may list the **developers who write the code** as an option. They are **not** a client of the system, a client is someone or something the software must *serve*; the development team is who *builds* it, not who it’s built for. Don’t confuse “people who touch the system” with “people/systems the system exists to serve.”

◆ MEMORY TRICK

E-I-S: External = outside person, Internal = in-house person, Software = no person at all, just code talking to code.

QUIZ-ORIENTED NOTES

- Clients are *who is going to use your software, for what purpose, and in what way* – this framing itself has appeared directly as an exam distractor/summary line.
- Software engineering considers software itself (another system/component) to be a valid “client” – this is a frequently tested MCQ trap (Sep 2022 Q127: “All of these” is the correct answer when the options list external user, someone from the developer company, and a software component).
- Requirement specification is described as the **first step** in the software development process.

■ REMEMBER THIS

Requirements must cater to the needs of clients, and a client can be an external human, an internal human, or another piece of software.

1.3 Software Design and Development

➤ MOTIVATION

Suppose several developers are simply told “go implement the requirements” with no shared plan. What goes wrong? In practice, quite a lot:

- Different developers may have *different ideas* about how a piece of functionality should be implemented, so when their code is combined (*integrated*), the pieces do not fit together.
- When new features must be added later, developers need a *big-picture view* of the whole system, without one, changes made in isolation can silently break other parts of the system.

Both problems have the same root cause: coding started before anyone had a shared, structural plan for the system. This is exactly the gap that the **design** phase fills.

REAL-LIFE ANALOGY

Before construction workers pour concrete for a building, an architect produces blueprints. The blueprint does not lay a single brick, but it tells every worker, plumber, electrician, mason, exactly how their part fits with everyone else's. Software design is the blueprint stage: it exists so that when many developers write code in parallel, the pieces click together instead of colliding.

DEFINITION

Design provides a **big-picture view** of the software system and gives it **structure**, it decides how the system will be broken into components/modules and how those components will interact, *before* detailed code is written.

1.3.1 The Development Phase

Once a design exists, the **Development phase** begins: developers write the actual code that realises the requirements and design. Because large systems have many components, development work is usually **distributed** across many developers or teams, each responsible for implementing one component (for instance, one developer might implement “Interface A” while others build the modules W, X, Y, Z that depend on or interact with it).

For this distributed work to succeed without chaos, two things are essential:

1. **Developer documentation** describing how each component is meant to be used.
2. **Precise interface definitions**, an exact, unambiguous specification of what inputs a component accepts, what outputs it produces, and what behaviour it guarantees, so that other developers can use a component correctly *without needing to read or understand its internal code*.

★ INTUITION

An interface is a contract, not an implementation. If component B only ever talks to component A through a well-defined interface, then the team building A is free to completely rewrite A's internals later (to make it faster, for instance) without breaking B, *as long as the interface itself does not change*. This is precisely why clear interfaces matter so much in distributed development: they let teams work in parallel and evolve their code independently.

★ FREQUENTLY ASKED IN PYQs

Interface-definition questions appear in Jan 2026 (Q3) and Sep 2025 (Q4, Q12). ★★☆☆ A very common scenario: “Two developers work on different modules; one changes a function's parameters and breaks the other's code.” The tested concept is almost always **interface definition** (the fix), not testing or refactoring.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q3). Two developers are working on different modules of an e-commerce system. One developer modifies a function's parameters, causing the other developer's code to break.

Reasoning. This is exactly the failure mode that clear **interface definitions** are designed to prevent: if the function's interface (its accepted parameters, return type, and behaviour) had been pre-

cisely defined and treated as a stable contract, the second developer's code would not silently break when the first developer changed the internal implementation. The correct concept being tested is *interface definition*, not unit testing, acceptance testing, or code refactoring.

✗ COMMON MISTAKE

Students sometimes answer such questions with “unit testing” because testing *would eventually catch* the bug. But the question is asking what would have **prevented** the issue at the design/development stage, that is a matter of having a precise, agreed-upon interface, not of catching the bug after the fact.

QUIZ-ORIENTED NOTES

- Design = big picture + structure. Development = writing code based on requirements *and* design.
- Development is usually **distributed** across developers/teams.
- Precise interface definitions and developer documentation are what make distributed development possible.

■ REMEMBER THIS

Design gives the system its skeleton before code (its muscles) is written; interfaces are the joints that let independently-built parts move together without colliding.

1.4 Software Testing and Maintenance

➤ MOTIVATION

Even with careful design, code written by humans contains mistakes. If software were released the moment coding finished, those mistakes would reach real users, sometimes with catastrophic and expensive consequences. Testing exists to catch problems *before* release; maintenance exists to keep the system healthy *after* release.

REAL-LIFE ANALOGY

Think of a new car rolling off the factory line. Before it ever reaches a customer, it is crash-tested, its brakes are checked, its engine is run on a dynamometer. This is testing. After the car is sold, the manufacturer still issues recalls, software updates for the car's computer, and scheduled services, this is maintenance. Software goes through exactly the same two post-construction phases.

1.4.1 The Cost of Skipping Testing

DEFINITION

Testing is done to ensure that software behaves according to its requirements. Even after development is “finished”, many bugs might still exist in the system, testing is how they are found before customers find them.

★ FREQUENTLY ASKED IN PYQs

The financial cost of bugs is directly quoted in the original slides and referenced in exam-style scenario questions about the “importance of testing.” ★☆☆

The scale of this problem is not small. According to the figures used in the course material, software errors were estimated to have cost the US economy **\$59.5 billion** in 2002, and by 2016 the estimated global financial cost of software bugs had grown to roughly **\$1.1 trillion**. Beyond financial loss, a failure to catch bugs can cause severe real-world catastrophes (failures in medical devices, aviation systems, or financial systems, for instance).

1.4.2 Levels of Testing

The course introduces three broad levels of testing, arranged by scope:

Level	What it checks
Unit Testing	Tests one small, isolated piece of code (e.g. a single function) in complete isolation from the rest of the system.
Integration Testing	Tests whether multiple units/components work correctly <i>together</i> once combined.
Acceptance Testing	Tests whether the complete system satisfies the client’s/user’s actual requirements, usually the final check before release.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q4). Before releasing a banking application, the development team tests individual functions such as “check balance” and “transfer money” *in complete isolation*.

Reasoning. Testing single functions in isolation, with no other components involved, is the textbook definition of **unit testing**. If instead the team tested whether “check balance” correctly triggers an update that “transfer money” then reads, that would be integration testing.

1.4.3 Alpha Testing and Beta Testing

★ FREQUENTLY ASKED IN PYQs

Alpha vs. Beta testing identification is one of the most reliably repeated question types across every quiz paper studied (Jan 2025 Q54, May 2024 Q3, May 2025 Q94, Sep 2024 Q3, Sep 2025 Q3).

★★★

	Alpha Testing	Beta Testing
Conducted by	Internal employees, in a lab/staging environment	Actual (real) users, in a real-world/live setting
Goal	Catch as many issues as possible <i>before</i> public release	Get authentic feedback and catch issues that only surface under real usage patterns
Timing	Earlier, before any external exposure	Later, just before (or as part of) general release

WORKED EXAMPLE

Scenario (adapted from Sep 2024 Q3 / May 2024 Q3). A company builds a website for a school. The website is given to the school's *faculty members* to check whether it meets the requirements. The faculty find that uploading results for more than 1000 students crashes the site.

Reasoning. The faculty are real, intended end-users (not the software company's own internal testers), and they are using it in a realistic setting before general public release. This matches the definition of **Beta Testing** (some papers alternatively frame this scenario's overall SDLC phase as the "**Testing phase**" when the question asks about SDLC phase rather than the testing sub-type, both readings are consistent: beta testing is one specific activity that happens *within* the broader testing phase).

EXAM TIP

If the scenario says "internal employees / staging environment", pick **Alpha**. If it says "real users / real-world conditions / selected customers", pick **Beta**. Watch for scenario chains like Sep 2025 Q3, where a question asks about *two* sequential activities ("internal testers simulate trips ...then release to selected users") – the correct pairing there is **Alpha Testing** → **Beta Testing**, in that order.

1.4.4 The Maintenance Phase**DEFINITION**

Maintenance is the phase that begins *after* a feature has been rolled out to users. Its purposes are to (a) monitor how users are actually using the software, (b) change the code to deliver upgrades and updates, and (c) add new features based on what is learned from real usage.

X COMMON MISTAKE

A very common PYQ pattern (Jan 2023 Q233, Jan 2025 Q55) describes an app that is *already installed and receiving periodic upgrades from time to time*. Students sometimes answer "Development phase" because upgrades involve writing new code, but the correct answer is **Maintenance phase**: the defining signal is that the software has *already been released* and is now simply being monitored and incrementally updated, that is maintenance by definition, even though it does involve writing code.

QUIZ-ORIENTED NOTES

- Three purposes of maintenance: monitor usage, change code for upgrades, add features.
- “Already released + being updated/monitored” = Maintenance phase (not Development).
- “Given to a limited/predefined set of real customers before public release, and a bug is found” = commonly a **Testing phase** / **Beta testing** scenario, *not* maintenance (maintenance happens *after* full release).

◆ MEMORY TRICK

A-B-C of testing scope: Alpha is internal. Beta is by the public (real users). Unit → Integration → Acceptance grows in scope like a zoom lens, one function → several components → the whole system against real requirements.

1.5 The Software Development Lifecycle (SDLC)

► MOTIVATION

So far this chapter has introduced five separate ideas: components, requirements, design, development, and testing/maintenance, one section at a time. But in reality, no team performs these as isolated, unrelated activities; they follow one another in some organised sequence, over and over, as software evolves. Software engineering needed a name and a formal structure for “the well-defined set of activities a piece of software goes through, from an initial customer request to a fully developed product.”

REAL-LIFE ANALOGY

Think of the human life cycle: infancy, childhood, adolescence, adulthood, old age. Each stage has recognisable characteristics, and every person passes through them in the same broad order (even though the details differ hugely from person to person). A software system similarly passes through recognisable *stages* as it matures from an idea into a released, maintained product.

DEFINITION

The **Software Lifecycle** (also called the **Software Development Life Cycle, SDLC**, or the **software development process model**) refers to the different stages/phases/activities over which a piece of software evolves, from the initial customer request through to a fully developed, released, and maintained software system.

Historically, before software engineering existed as a formal discipline, projects were often built using a “build and fix” approach, code first, patch problems as they are discovered, with no upfront planning. As the industry matured, good principles, practices, and research were combined into what we now call **Software Engineering**, and a well-defined set of lifecycle activities emerged.

1.5.1 The Five Core Phases, Recap



Phase	What happens
Requirements	Identify the goals the system should have; cater to the needs of clients.
Design	Establish a big-picture view and structure for the software system.
Development	Write code (usually distributed across a team) based on the requirements and design, supported by developer documentation and precise interface definitions.
Testing	Ensure the software behaves according to the requirements (unit, integration, acceptance, alpha, beta).
Maintenance	Monitor how users use the software; change code for updates and new features.

EXAM TIP

A huge share of PYQ “which SDLC phase is this?” questions are answered correctly just by asking: *has the software already been released to the public, and are we now watching how it behaves and patching it?* If yes → Maintenance. *Is it being shown to a limited group before public release, to check for problems?* If yes → Testing (often specifically Beta Testing). *Is code being written for the first time based on a plan?* If yes → Development.

REMEMBER THIS

The SDLC is not a single fixed procedure, it is a template of five recurring phases; *how* a project moves through those phases (all at once vs. repeatedly, in strict order vs. flexibly) is exactly what the rest of this chapter, the development process *models*, is about.

1.6 Software Development Models: The Plan-and-Document Perspective

> MOTIVATION

Knowing the five SDLC phases is not enough, real projects must decide *how* to move through them. Do you finish requirements completely before starting design? Do you build something rough first, just to see if it’s on the right track? Do you repeat the whole cycle many times in small increments? Different answers to this question give rise to different **development process models**. This section covers the **Plan-and-Document** family, models that emphasise finishing each phase thoroughly, with detailed documentation, before moving to the next.

1.6.1 The Waterfall Model

★ FREQUENTLY ASKED IN PYQs

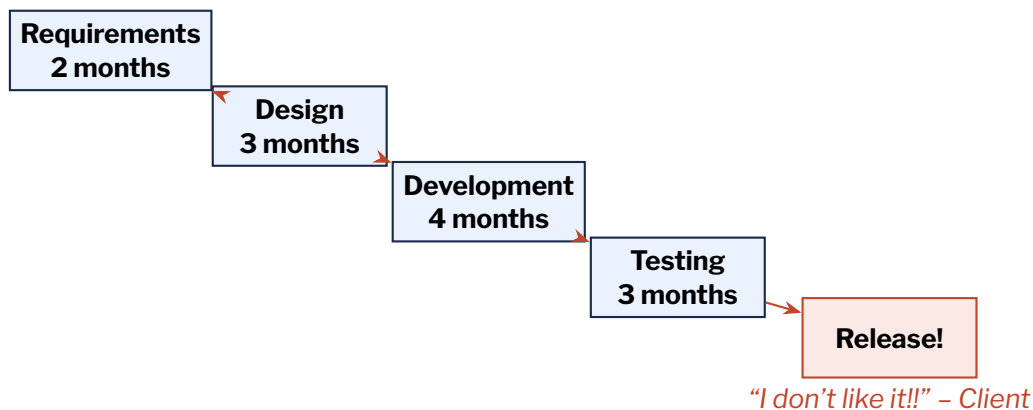
Waterfall identification and its drawbacks are tested very frequently (Jan 2023 Q234, Jan 2026 Q5, May 2025 Q108, Sep 2023 Q176). ★★★

REAL-LIFE ANALOGY

A waterfall flows only in one direction, water never climbs back up. The Waterfall model gets its name from exactly this property: each phase (requirements → design → development → testing) flows into the next, strictly in sequence, and the process is not designed to flow backwards.

DEFINITION

The **Waterfall Model** executes the SDLC phases **sequentially**, one phase must be fully completed before the next begins, and the client typically only sees the working software *during or after the testing phase*, not before.



The staircase shape in the diagram above is intentional: each phase (each “step”) must finish completely before the next one begins, this is the defining trait tested in PYQs: “*all the phases take place sequentially, and the users get their first view of the software during or after testing.*”

WORKED EXAMPLE

Scenario (adapted from May 2025 Q108). A team develops a system over 12 months using a strict sequential process. After the final release, the client is dissatisfied because their requirements were misunderstood.

Reasoning. “Strict sequential process” plus “client only discovers the mismatch at the very end” is the signature of the **Waterfall model**: because the client sees nothing until testing/release, any misunderstanding from the requirements phase is not caught until it is very expensive to fix.

Drawbacks of the Waterfall Model

- **Costly late changes.** If requirements were misunderstood, discovering this only at release means re-doing work, cost and time increase sharply.
- **Clients may not know what they need up front.** It is often unrealistic to expect a client to specify complete, correct requirements before seeing anything built.

- **Designers may guess wrong.** Without early feedback, designers may choose a design that is not actually the most feasible or usable one for the client.
- **Long cycle time.** A single Waterfall cycle typically takes **6–18 months**, which is a long time to wait before finding out something is wrong.

✗ COMMON MISTAKE

Do not confuse “sequential” with “no testing.” A wrong distractor sometimes seen in PYQs is “*faster development with no need for testing*” being offered as an advantage of Agile over Waterfall (Sep 2023 Q176), this is false on two counts: Waterfall absolutely includes a testing phase, and skipping testing is never a real advantage of any model.

1.6.2 The Prototype Model

★ FREQUENTLY ASKED IN PYQs

Advantages of the Prototype model are tested repeatedly, almost always as a “select all that apply” style question (Jan 2024 Q43, May 2024 Q11, Sep 2024 Q17, Sep 2025 Q5). ★★★

► MOTIVATION

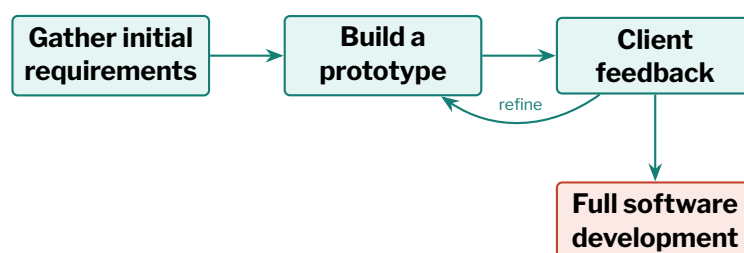
Waterfall’s biggest weakness is that the client cannot give meaningful feedback until the very end. What if, instead, the team builds a rough, throwaway version early, just good enough to show the client and gather feedback, *before* committing to full development?

REAL-LIFE ANALOGY

This is exactly what an interior designer does with a small-scale physical model of a room before any real construction begins, or what a fashion designer does with a rough muslin “toile” before cutting the expensive final fabric. The model is not the final product and is usually thrown away, but it lets everyone see and react to the idea cheaply.

DEFINITION

The **Prototype Model** builds a working, usually stripped-down, **prototype** *before* full development of the actual software begins, purely to get early feedback and clarify uncertain requirements or technical approaches. The prototype itself is usually **not used later**, it is typically discarded once its job (gathering feedback) is done.



Advantages	Disadvantages
Very useful when the exact form of the solution and technical issues are still unclear. Excellent way to get early, concrete feedback from customers.	Increased overall development cost (you are essentially building the system twice: once as a prototype, once for real). Bugs can still appear later in the actual development cycle (a prototype does not guarantee a bug-free final product).

✗ COMMON MISTAKE

A very reliable PYQ distractor claims that “prototypes are delivered to the client along with the actual product” or that the prototype model “reduces cost significantly” or “ensures no bugs appear later.” All three are **false**. The prototype is normally thrown away, the model can *increase* cost (because of the extra prototyping work), and it does not guarantee bug-free software; it only helps clarify requirements and surface technical risk earlier.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q5). A team building a mobile banking app is unsure of the exact requirements, so they build a limited prototype and share it with the client for feedback before building the full system. Which are genuine advantages of this approach?

Correct selections. “Helps clients understand what they actually need” and “Helps developers identify technical issues and improve design before full development” are both true. “Reduces cost because prototypes are always used in production” and “Ensures all bugs are caught before the final system is built” are both **false** distractors, prototypes are not always reused in production, and no process can guarantee zero bugs.

1.6.3 The Spiral Model

► MOTIVATION

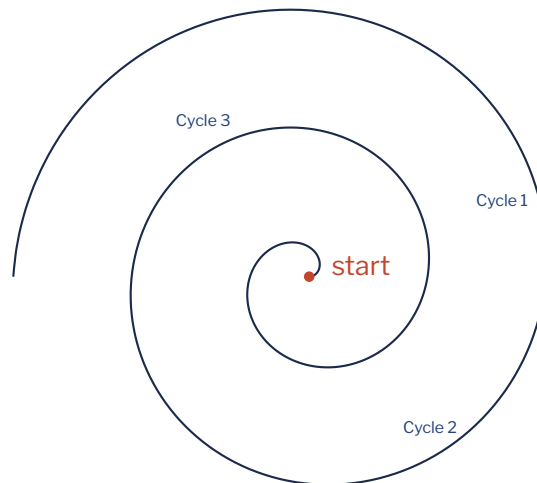
Waterfall commits to a full plan up front; Prototype tests ideas cheaply but is still, in the version taught here, a single feedback loop before full development. What if a project is large, risky, and needs *repeated* rounds of building, testing, and refining, incrementally growing the software while continually gathering feedback?

REAL-LIFE ANALOGY

Think of sculpting a statue by making many successive, ever more detailed passes over the stone, roughing out the overall shape first, then refining the limbs, then the face, then the fine details, stepping back to look at (and get feedback on) the work after every pass, rather than trying to carve the finished statue in one uninterrupted attempt.

DEFINITION

The **Spiral Model** incrementally builds the software in repeated cycles (“loops of the spiral”), getting feedback after each cycle and using it to refine the next cycle, gradually converging on the finished, well-tested product.



Each loop of the spiral incrementally builds more of the software and gathers more feedback, refining the design and requirements as understanding improves, this makes the Spiral Model especially suited to large, high-risk projects where requirements cannot realistically be nailed down completely in advance.

QUIZ-ORIENTED NOTES

- Plan-and-Document family: **Waterfall** (fully sequential), **Prototype** (one early feedback loop via a throwaway prototype), **Spiral** (repeated incremental cycles with feedback).
- Waterfall's biggest weakness: no client feedback until testing/release; typical cycle length 6–18 months.
- Prototype's biggest strength: early, cheap client feedback; biggest weakness: extra cost, no bug-free guarantee.
- Build-and-Fix is *not* one of the three named models, it is the informal, undisciplined precursor that motivated software engineering as a discipline in the first place (watch for this as a PYQ distractor, e.g. Jan 2026 Q5).

◆ MEMORY TRICK

W-P-S: **W**aterfall = one-way, no looking back. **P**rototype = build a rough draft first, throw it away. **S**piral = keep looping, a little more built and refined every lap.

1.7 Software Development Models: The Agile Perspective

► MOTIVATION

Plan-and-Document models assume that requirements can be pinned down early and will not change much. In reality, clients frequently *do not* know exactly what they want until they see something working, and requirements change as the business changes. The Agile perspective was born from accepting this reality rather than fighting it: instead of trying to eliminate change, build a process that *welcomes* it.

1.7.1 The Agile Manifesto

★ FREQUENTLY ASKED IN PYQs

The four Agile Manifesto values are tested directly as recall/matching questions and as the basis of many scenario questions. ★★☆☆

DEFINITION

The **Agile Manifesto** expresses four key value statements, each of the form “X over Y”, meaning X is valued *more*, not that Y is abandoned entirely:

1. Individuals and interactions **over** processes and tools.
2. Working software **over** comprehensive documentation.
3. Customer collaboration **over** contract negotiation.
4. Responding to change **over** following a plan.

★ INTUITION

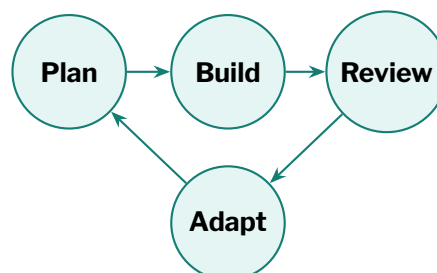
Read each line as “we still value the thing on the right, but when we must choose, we choose the thing on the left.” Agile teams still write documentation, still use tools, still make plans, they just refuse to let those things become more important than working software, real conversations, and the flexibility to change course.

✗ COMMON MISTAKE

A frequent wrong-answer choice frames Agile’s advantage over Waterfall as “strong emphasis on documentation” or “strict adherence to a fixed project plan”, both describe Waterfall’s mindset, not Agile’s (Sep 2023 Q176). The correct Agile advantage is the **ability to accommodate changing requirements**.

1.7.2 Incremental Development and the Agile Cycle

Rather than one long Waterfall-style pass, Agile methods build software **incrementally**: small, workable slices of functionality are built, tested, and delivered in short, repeated cycles, with continuous opportunities to adjust course based on feedback.



One short cycle = one Sprint (1–4 weeks), then repeat

1.7.3 Three Named Agile Approaches

Approach	Core idea
Extreme Programming (XP)	An Agile approach placing very strong emphasis on engineering discipline, e.g., continuous testing, frequent releases, and close collaboration.
Scrum	A structured Agile framework organised around fixed-length Sprints , defined roles, and a small set of recurring ceremonies (meetings).
Kanban	A visual, flow-based Agile approach that manages ongoing work using a board of columns (e.g. To-Do / In-Progress / Completed) rather than fixed-length sprints.

1.7.4 Core Agile / Scrum Practices

★ FREQUENTLY ASKED IN PYQs

Scrum ceremonies, roles, and artefacts (sprint planning, daily scrum, sprint review, sprint retrospective, product backlog, product owner, scrum master, velocity) form one of the single most heavily and repeatedly tested topic clusters across all 11 papers. ★★★

User Stories

The Agile/BDD (Behaviour Driven Design) way of capturing a requirement, covered in full detail in Chapter 2, Section 2.6.

Sprints

DEFINITION

A **Sprint** is a short, fixed-length (typically 1–2 weeks) time-boxed period during which a Scrum team works to complete a specific, agreed set of work.

The Product Backlog

DEFINITION

The **Product Backlog** is a prioritised list of tasks for the development team, derived from the user stories and requirements, everything that could possibly be built, ordered by priority/value.

✗ COMMON MISTAKE

PYQs (Jan 2023 Q240) offer close-sounding but wrong definitions as distractors: a sprint (“a short time-boxed period”), a sprint retrospective’s output (“list of what went well/didn’t go well”), and a burndown/task-board style summary (“tasks moved from To-Do to Completed”) are all *not* the product backlog. The product backlog specifically is the **prioritised list of tasks derived from user stories and requirements**.

1.7. Software Development Models: The Agile Perspective

Who is responsible for generating the product backlog? While the **Product Owner** typically owns and prioritises it, PYQ answer keys (Sep 2022 Q129) have treated this as a collaborative responsibility, with **Product Owner, Scrum Master, and Development Team all**, contributing, so read such questions carefully for whether they are asking about *ownership/prioritisation* (Product Owner) or *collaborative creation* (all of these).

The Four Sprint Ceremonies

Ceremony	Purpose
Sprint Planning	The team decides <i>what</i> work can be completed in the upcoming sprint and discusses <i>how</i> the selected work will be implemented. Time-boxed.
Daily Scrum / Daily Stand-up	A short daily meeting (development team, Scrum Master, product manager/owner present) where each member says what they did yesterday, what they will do today, and what is blocking them. Purpose: identify and address impediments quickly, not to report to upper management.
Sprint Review	The team demonstrates what was actually completed during the sprint, typically to stakeholders.
Sprint Retrospective	The team evaluates the <i>last</i> sprint itself, team dynamics, process, tools, to decide how to improve in future sprints.

EXAM TIP

These four ceremonies are frequently tested via matching or “which meeting is being described” questions. The reliable distinguishing keyword for each: Planning → “what/how, before the sprint”. Daily Scrum → “yesterday/today/blockers, every day”. Review → “demonstrate/show what’s done, at sprint end”. Retrospective → “evaluate the process itself, how can we improve”.

WORKED EXAMPLE

Scenario (adapted from Jan 2024 Q38). Which Scrum activity involves the development team, the Scrum Master, and the product manager, where each member talks about what they did yesterday, what they will do today, and what is blocking their work?

Answer. Daily Scrum Meeting (daily stand-up), the “yesterday / today / blockers” phrasing is the signature of this ceremony.

Scrum Roles

Role	Responsibility
Product Owner	Ensures the development team delivers maximum value to the client; prioritises and maintains the product backlog considering business needs and technical constraints.
Scrum Master	Facilitates the Scrum process, helps the team follow Agile practices and remove blockers, is <i>not</i> the person who prioritises the backlog.

X COMMON MISTAKE

Do not swap Product Owner and Scrum Master. “Prioritises/maintains the backlog, focuses on business value” → **Product Owner**. “Facilitates ceremonies, removes team impediments, coaches the process” → **Scrum Master**. This distinction is directly tested (Jan 2026 Q12, Sep 2023 Q186).

Velocity**DEFINITION**

Velocity is the amount of work (usually measured in **story points**) a Scrum team completes in a single sprint. It is used to *predict* how much work the team can realistically take on in future sprints.

WORKED EXAMPLE

Scenario (adapted from May 2025 Q101). A Scrum team completes 2 stories totaling 5 story points in its first sprint. What is the team’s velocity?

Answer. 5 (velocity is measured as total story points completed per sprint, not the number of stories).

Test-Driven Development (TDD)

A practice, most associated with Extreme Programming, where automated tests are written *before* the corresponding code, so that the code is written specifically to make the (already-written) test pass. This tightens the feedback loop between writing code and knowing whether it works.

1.7.5 The Agile Philosophy

The course material is explicit that Agile is best understood as a **philosophy**, not merely a checklist of practices (user stories, sprints, stand-ups, TDD). A team can perform every Agile ceremony correctly and still fail to be “Agile” in spirit if it does not also embrace the four Manifesto values (individuals over process, working software over documentation, collaboration over negotiation, responding to change over following a plan).

QUIZ-ORIENTED NOTES

- Manifesto: Individuals/interactions, Working software, Customer collaboration, Responding to change, each valued *over* (not *instead of*) its counterpart.
- XP, Scrum, Kanban are three named Agile approaches; Scrum is organised around fixed sprints and roles/ceremonies; Kanban is flow-based using a visual board.
- Backlog = prioritised list of tasks from user stories/requirements. Velocity = story points completed per sprint.
- Four ceremonies: Planning (what/how), Daily Scrum (yesterday/today/blockers), Review (demo), Retrospective (process improvement).
- Product Owner prioritises value; Scrum Master facilitates process.

◆ MEMORY TRICK

P-D-R-R for the four ceremonies in order across a sprint: **P**lan it, **D**o it (daily standups throughout), **R**eview it, **R**eflect on it (retrospective).

1.7.6 Choosing Between Plan-and-Document and Agile

★ FREQUENTLY ASKED IN PYQs

This decision framework (Boehm's questions) is tested as a reflection-spot style concept; understanding the *direction* of each question (which answer favours Agile vs. Plan-and-Document) is what matters. ★☆☆

Neither approach is universally “better”, the right choice depends on the nature of the project. The course frames this as a set of yes/no questions: a **No** answer to most of these questions suggests **Agile** is a better fit; a **Yes** suggests **Plan-and-Document** is a better fit.

#	Question	“Yes” favours
1	Is specification required (must requirements be nailed down precisely up front)?	Plan-and-Document
2	Are customers unavailable (for ongoing collaboration)?	Plan-and-Document
3	Is the system to be built large?	Plan-and-Document
4	Is the system complex (e.g. real-time)?	Plan-and-Document
5	Will it have a long product lifetime?	Plan-and-Document
6	Are poor software tools being used?	Plan-and-Document
7	Is the project team geographically distributed?	Plan-and-Document
8	Is the team part of a documentation-oriented culture?	Plan-and-Document
9	Does the team have poor programming skills?	Plan-and-Document
10	Is the system subject to regulation?	Plan-and-Document

(I. Sommerville, *Software Engineering*, 9th Edition, Addison-Wesley, 2010.)

★ INTUITION

Notice a pattern: nearly every “Yes” answer points toward *more rigidity being necessary or safer* (large scale, regulation, distributed/unavailable stakeholders, weak tooling or skills), exactly the conditions under which up-front planning and documentation pay off. Agile thrives on the opposite conditions: a small, skilled, co-located team, an available customer, and a system that is not safety-critical

or heavily regulated.

END OF CHAPTER 1: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- **Components/modules** → separately addressable units integrated to satisfy requirements.
- **Client types:** External user, Internal user, Software client.
- **SDLC phases:** Requirements → Design → Development → Testing → Maintenance.
- **Testing levels:** Unit → Integration → Acceptance. **Testing modes:** Alpha (internal, staging) → Beta (real users, real setting).
- **Plan-and-Document models:** Waterfall (strict sequence, 6–18 months, no feedback until end), Prototype (throwaway early feedback build), Spiral (repeated incremental cycles with feedback).
- **Agile Manifesto (4 values):** Individuals/interactions > processes/tools; Working software > documentation; Customer collaboration > contract negotiation; Responding to change > following a plan.
- **Agile approaches:** XP, Scrum, Kanban. **Scrum ceremonies:** Planning, Daily Scrum, Review, Retrospective. **Scrum roles:** Product Owner (value/backlog), Scrum Master (process/facilitation).
- **Backlog** = prioritised task list from user stories. **Velocity** = story points completed per sprint.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

High-probability traps to watch for:

1. “Already released + being monitored/updated” ⇒ Maintenance, *not* Development.
2. “Given to a limited set of real users before public release” ⇒ usually Testing / Beta Testing, *not* Maintenance.
3. Prototype model does *not* guarantee bug-free software and does *not* reduce cost, it is used for early feedback and risk reduction.
4. “Faster development with no need for testing” is never a true advantage of Agile (or any

model).

5. Product Owner prioritises the backlog for business value; Scrum Master facilitates the process, do not swap them.
6. Product Backlog $\neq$ Sprint Backlog $\neq$ Retrospective notes $\neq$ Kanban board, read the definition carefully.
7. “Another software system calling this system through an interface” $\Rightarrow$ Software client, not Internal or External user.

PRACTICE QUESTIONS

- Q1.** A ride-sharing company lets its own internal logistics team use a dashboard to monitor driver locations. What type of client is the logistics team?
- Q2.** A company builds a recommendation engine that is called automatically by its own mobile app whenever a user opens the home screen. What type of client is the mobile app, from the recommendation engine’s point of view?
- Q3.** Explain, using the courier example from Section 1.2.2, why the courier company is neither a primary nor a secondary *client* of the ordering software in the sense meant by this section.
- Q4.** A team follows a strict sequential process: requirements are frozen, then design, then development, then testing, and the client only sees the product at the very end. Name the model and list two of its drawbacks.
- Q5.** List two genuine advantages and two false (commonly mis-stated) advantages of the Prototype Model.
- Q6.** In your own words, explain why “responding to change over following a plan” does *not* mean Agile teams have no plan at all.
- Q7.** A Scrum team completes 3 stories worth 8 story points total in Sprint 1, and 3 stories worth 13 points in Sprint 2. What is the team’s velocity in each sprint, and why is this number useful for planning Sprint 3?
- Q8.** Distinguish between a Sprint Review and a Sprint Retrospective using one sentence each.
- Q9.** Using Boehm’s ten questions (Section 1.7.6), argue whether a small startup building a social media app for a handful of early-adopter friends should use Agile or Plan-and-Document.
- Q10.** A company builds air-traffic control software. Using the same ten questions, argue why Plan-and-Document is likely the safer choice here.

CHAPTER 2

Software Requirements

In this chapter: Requirement gathering & analysis · Primary, secondary and tertiary users · Requirement gathering techniques · Functional and non-functional requirements · The Software Requirement Specification (SRS) document · Behaviour Driven Design and User Stories · SMART user stories.

2.1 Requirement Gathering and Analysis

> MOTIVATION

Chapter 1 established that requirements are the very first phase of the SDLC, but it did not explain *how* a team actually discovers what the requirements are. Nobody hands a development team a perfect, complete list of requirements on day one, they must be actively **gathered** from the people and systems that will use the software, and then **analysed** to make sure they are complete, consistent, and unambiguous. Skipping or rushing this step is one of the most expensive mistakes a software project can make.

REAL-LIFE ANALOGY

Imagine a tailor asked to make a suit without ever measuring the customer or asking about their taste, formal or casual, tight or loose fit, which colours they like. The tailor could still produce *a* suit, but the odds of it actually fitting and pleasing the customer are low. Requirement gathering is the tailor taking measurements and asking questions *before* cutting any fabric.

DEFINITION

Requirement Gathering is the process of actively collecting information about what a software system needs to do, from its clients and stakeholders. **Requirement Analysis** is the subsequent process of examining the gathered information to check that it is complete, consistent, and free of ambiguity.

Why This Matters

- It helps **developers understand** what customers actually want, rather than guessing.
- It helps **customers themselves come to agreement** about their own requirements, stakeholders often disagree with each other, or have not even articulated their needs clearly until asked.
- Skipping this step properly leads to a sharp **increase in cost and iterations** later, requirements misunderstood at the start are usually far more expensive to fix after code has already been written (recall the Waterfall model's biggest drawback from Chapter 1).

Running Example: The Amazon Seller Portal

Throughout this chapter, we reuse the course's running example: Amazon wants to build a **Seller Portal**, a platform where sellers can list products, which then become available for people to buy.

QUIZ-ORIENTED NOTES

- Requirement gathering and analysis is typically performed by a **system analyst**, working with other members of the software team (this becomes important again in Section 2.5).
- A key skill tested repeatedly: recognising what kind of requirement *problem* a scenario illustrates, ambiguous, inconsistent, or incomplete requirements (see Section 2.2.1).

REMEMBER THIS

Requirements are not “given”, they are actively discovered, and the quality of that discovery process determines how expensive the rest of the project will be.

2.2 Identifying Users and Requirements

> MOTIVATION

Before you can gather requirements, you must first know *whose* requirements to gather. For the Amazon Seller Portal, an obvious first guess is “sellers”, but a moment's thought reveals many more people who will be affected: sales managers, an advertising department, a logistics/shipping company, buyers, and even banks. Treating only the most obvious user as “the user” is a common and costly mistake, requirements from less obvious stakeholders are just as real, and missing them causes exactly the kind of expensive late rework introduced in Section 2.1.

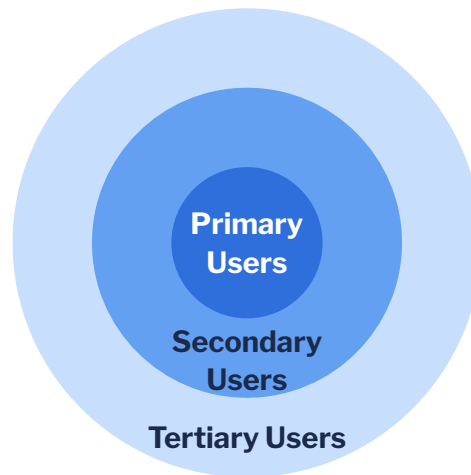
REAL-LIFE ANALOGY

Think about who is affected by a new highway being built. The people who *drive* on it every day are the most obvious group. But local shop owners near the highway (who benefit from more passing traffic), residents who now hear more noise, and the city's traffic police (who must now manage new intersections) are all affected too, even though they never personally drive on that stretch of road. A software system has exactly this same layered structure of “who is affected.”

DEFINITION

Users of a software system are classified into three types, based on **how directly** they interact with it:

- **Primary users:** Frequent, direct users of the system.
- **Secondary users:** Do not directly use the system themselves, they use it *through an intermediary*.
- **Tertiary users:** Do not use the software at all, but are **affected by** its introduction, or **influence** the decision to purchase/adopt it.



Applying This to the Seller Portal

Type	Example (Amazon Seller Portal)
Primary	Independent sellers, sales teams of consumer companies, independent authors and publishers, all use the portal frequently and directly.
Secondary	A buyer's personal assistant placing orders on their behalf; someone who interacts with what the portal produces (e.g. a listed product) but not with the portal's seller-facing tools directly.
Tertiary	Banks (they are affected because payments must be settled, but never log into the seller portal); logistics/shipping companies (affected by orders generated but not users of the seller-facing tools); the advertising department (may influence adoption/requirements without daily use).

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q235 / Sep 2024 Q19). An online flower store lets customers order bouquets. Once an order is placed, the store manager phones a courier company to arrange pickup and delivery.

Reasoning. The courier company never touches the ordering software. But its work is a direct

2.2. Identifying Users and Requirements

downstream consequence of an order being placed, it is affected by the system's use. This makes the courier company a **tertiary user**.

WORKED EXAMPLE

Scenario (adapted from May 2025 Q95). An online education platform is used by professors (who upload materials) and students (who access content). A university accreditation body occasionally reviews reports generated from the system.

Reasoning. Both professors and students interact with the system frequently and directly, so *both* are **primary users** (there is no requirement that only one role be “the” primary user). The accreditation body never logs in and never uses the system directly, it only reviews reports produced by it, this makes it a **tertiary user**.

X COMMON MISTAKE

Do not assume “primary user” means only one specific role. A system can, and usually does, have **multiple primary user types** at once (e.g. professors *and* students in the example above). Also, do not confuse “uses the system rarely” with “secondary user”, the defining trait of a secondary user is using the system *through an intermediary*, not merely using it less often.

EXAM TIP

Ask this three-question chain for any scenario: (1) Does this person/organisation log in and use the system *themselves, frequently*? → Primary. (2) If not, do they still get something from the system, but *through someone else*? → Secondary. (3) If they never touch the system at all, but are still affected by its existence, or they influence whether it gets built/bought? → Tertiary.

2.2.1 Common Problems Found During Requirement Analysis

Once requirements have been gathered from primary, secondary, and tertiary stakeholders, the analysis step (Section 2.1) looks for three characteristic problems:

Problem	What it looks like
Ambiguous	The requirement can reasonably be interpreted in more than one way; different readers would build different things from the same sentence.
Inconsistent	Two (or more) stated requirements directly contradict one another.
Incomplete	Something important was never stated at all, a case, a user type, or a condition was simply missed during gathering.

WORKED EXAMPLE

Scenario (adapted from May 2025 Q96). One stakeholder wants passwords to expire every 30 days; another insists passwords must not be changed more than once a year.

Reasoning. These two requirements cannot both hold simultaneously, this is an **inconsistent** requirement.

QUIZ-ORIENTED NOTES

- Three user types: **Primary** (direct, frequent), **Secondary** (indirect, via intermediary), **Tertiary** (not a user at all, but affected/influential).
- A system can have multiple primary user types simultaneously.
- Three requirement analysis problems: **Ambiguous**, **Inconsistent**, **Incomplete**.

◆ MEMORY TRICK

P-S-T distance from the system: **P**rimary touches the keyboard. **S**econdary is one step removed (through a middle-person). **T**ertiary never touches it at all, but still feels the ripple effect.

2.3 Requirement Gathering Techniques

★ FREQUENTLY ASKED IN PYQs

Identifying the correct requirement-gathering technique from a scenario is, together with client-type identification, the single most repeated question pattern across all 11 papers (appears in nearly every paper: Jan 2023, Jan 2025, Jan 2026, May 2023, May 2024, May 2025, Sep 2022, Sep 2023, Sep 2024, Sep 2025). ★★★

> MOTIVATION

There is no single “correct” way to talk to stakeholders and find out what they need, different situations call for different tools. Asking a thousand geographically-scattered sellers a handful of fixed questions calls for a very different technique than sitting down with three key stakeholders to resolve a disagreement. This section catalogues five techniques, each suited to a different kind of situation.

REAL-LIFE ANALOGY

A doctor diagnosing a patient does not rely on only one method, they take a written history (like a questionnaire), have a conversation (like an interview), sometimes bring in a panel of specialists to discuss a hard case (like a focus group), observe the patient directly (like naturalistic observation), and consult medical records and regulations (like documentation study). Good requirement gathering, like good diagnosis, mixes several of these methods rather than relying on just one.

2.3.1 The Five Techniques

1. Questionnaires

DEFINITION

A **questionnaire** is a series of questions designed to elicit specific information from users. It is well suited to getting answers to **specific, well-defined questions from a large group of people**, and is usually used alongside other techniques.

Example: Sending sales-team managers of several mobile companies a fixed set of questions such as “What percentage of your inventory do you sell online?”

2. Interviews

DEFINITION

An **interview** is a set of questions asked directly of a stakeholder, face-to-face, by phone, or online. Interviews are especially useful **early** in the process, to help explore issues and elicit realistic usage scenarios.

Interviews come in three flavours:

Type	Description
Structured	A fixed, predetermined set of questions is asked, in the same order, to every interviewee.
Unstructured	Open-ended questions with no fixed script; the interviewer adapts follow-up questions based on what the interviewee says, allowing free-flowing conversation.
Semi-structured	A mix: some fixed questions are asked of everyone, but the interviewer is also free to explore interesting answers with follow-up questions.

WORKED EXAMPLE

Scenario (adapted from Jan 2025 Q46). A hiring manager does not follow a predefined set of questions, instead asking open-ended questions and adapting follow-ups based on the candidate's answers, allowing free-flowing conversation.

Answer. Unstructured interview, the giveaway phrase is “no predefined questions” combined with “adapts based on responses.”

WORKED EXAMPLE

Scenario (adapted from May 2024 Q4). To understand what independent sellers sell, an interviewer prepares a *fixed* set of questions that will be asked of every seller.

Answer. Structured interview, a fixed, pre-prepared question set asked uniformly is the signature of a structured interview.

3. Focus Groups and Workshops

DEFINITION

A **focus group** brings together a **group of stakeholders** to discuss issues and requirements together, rather than one at a time.

This directly addresses a known drawback of one-on-one interviews: an interview only captures *one* person's perspective at a time. A focus group instead gains **consensus** across multiple stakeholders, and **highlights areas of conflict or disagreement** that a series of separate interviews might never surface.

Example: Sales-team managers from different verticals (mobile phones vs. furniture) have very different needs for a “sales summary” feature, mobile teams care about launch-day sales, furniture teams care about the last 30 days. A focus group surfaces this conflict directly, leading to a combined requirement: “sales summary for a specific day *or* the last x days.”

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q6). A team develops the sales-summary module. Mobile sales teams care about launch-day sales; furniture sales teams care about 30-day trends. Why conduct focus groups with sales managers from different industries?

Correct selections. “To gain consensus on common requirements across industries” and “To highlight conflicts or differences between stakeholder needs” are both true, these are precisely the two advantages of focus groups. “To observe how users perform tasks in their natural setting” describes *naturalistic observation*, not focus groups, and “To get detailed regulatory information from banks” describes *documentation study*, not focus groups, both are wrong-technique distractors.

4. Naturalistic Observations**DEFINITION**

Naturalistic observation means spending time with stakeholders as they go about their normal day-to-day tasks, observing their work in its natural setting, shadowing them, taking notes, asking clarifying questions as needed.

Example: To understand how people sell in physical shops (e.g. how a shopkeeper recommends related items, or gathers customer feedback informally), a team visits real shops and watches. This surfaces requirements like “recommend other items to sell” and “collect timely customer feedback”, requirements that people might never think to *say* explicitly in an interview, because to them it is just an unremarkable part of their daily routine.

WORKED EXAMPLE

Scenario (adapted from Sep 2022 Q118 / Sep 2023 Q179). A team visits famous grocery shops across the country to observe how customers place orders, how payments are made, and how items are delivered.

Answer. Naturalistic observation, physically visiting and watching real-world behaviour, rather than asking about it, is the defining trait.

5. Documentation**DEFINITION**

Documentation as a requirement-gathering technique means studying existing procedures and rules for a task, the steps involved in an activity, and regulations that govern it.

Example: Before building a feature to add a seller’s bank account to the portal, the team studies existing **bank regulations**, how often money can be deposited, what verification is legally required, and so on.

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q236). A team develops a pet-store app and goes through several documents about pet-shop rules and the duties/obligations of pet-shop owners.

Answer. Documentation, studying existing rules/regulations, rather than talking to people, is the signature of this technique.

2.3.2 Summary Table

Technique	Best for
Questionnaires	Answering specific, well-defined questions from a large group
Interviews	Exploring issues in depth, eliciting early scenarios
Focus groups	Collecting multiple viewpoints; surfacing consensus <i>and</i> conflict
Naturalistic observations	Understanding real-world context and unspoken/unstated behaviour
Documentation	Procedures, regulations, and standards that already exist

EXAM TIP

The fastest way to solve these PYQ scenarios: look for the **verb** in the scenario. “Sent a list of questions to many people” → Questionnaire. “Sat down and talked, one-on-one” → Interview (check: fixed script = Structured, no script = Unstructured, both = Semi-structured). “Got a group together to discuss” → Focus group. “Watched/shadowed/visited to observe” → Naturalistic observation. “Read rules/policies/regulations” → Documentation.

2.3.3 Basic Requirement-Gathering Guidelines

- Focus on identifying stakeholders’ *actual* needs (not just what they literally say).
- Involve **all** stakeholder groups, primary, secondary, and tertiary, not just the loudest or most obvious one.
- Use a **combination** of techniques, no single technique captures everything.
- Run a small **pilot session** first, where possible, to make sure the data-gathering plan actually works before committing to it at scale.
- Remember that data gathering is expensive and time-consuming: be pragmatic, and be prepared to make compromises.

2.3.4 Worked Example: Identified Requirements for the Seller Portal

Combining the outputs of the techniques above (for the running Amazon Seller Portal example), the team ends up with a concrete requirements list:

1. Add/Edit/Delete catalogue
2. Add/Edit/Delete inventory
3. Track orders
4. Track payments
5. Track inventory
6. Track sales, for a specific day or the last x days
7. Track customer feedback

This list becomes the input to the next stage of the course: classifying each requirement as **functional** or **non-functional** (Section 2.4), and eventually organising all of them into a formal **SRS document** (Section 2.5).

QUIZ-ORIENTED NOTES

- Five techniques: Questionnaires, Interviews (Structured/Unstructured/Semi-structured), Focus groups, Naturalistic observations, Documentation.
- Focus groups solve interviews' key weakness: a single interview only gives *one* perspective.
- Guidelines: involve all stakeholder types, combine techniques, pilot first, be pragmatic about cost/time.

◆ MEMORY TRICK

Q-I-F-N-D: **Q**uestionnaires for quantity (many people, fixed questions). **I**nterviews for individual depth. **F**ocus groups for friction (surfacing disagreement) and consensus. **N**aturalistic observation for the unnoticed/unspoken. **D**ocumentation for the already-decided rules.

2.4 Functional and Non-Functional Requirements

★ FREQUENTLY ASKED IN PYQs

Distinguishing functional from non-functional requirements is tested in nearly every quiz paper studied, usually as a two-requirement comparison question. ★★★

► MOTIVATION

Once requirements are gathered (Section 2.3), they turn out not to be all the same *kind* of statement. Compare these two requirements for the Seller Portal:

- “*A seller can add/edit/delete their catalogue.*”
- “*When a new product is added to the catalogue, it should appear within 5 seconds.*”

Both are legitimate requirements, but they are answering two fundamentally different questions. The first describes *what* the system must be able to do. The second describes *how well*, how fast, how reliably, how securely, it must do it. Software engineering gives these two categories separate names because they need to be specified, tested, and reasoned about differently.

REAL-LIFE ANALOGY

Think of ordering a pizza. “The restaurant must be able to make a pepperoni pizza” is a statement about *what* the restaurant can do, a functional capability. “The pizza must arrive within 30 minutes, hot, and without the box getting soggy” describes *qualities* of the delivery, not a new capability, the restaurant already “does pizza”, these are constraints on *how well* it does it. The first is functional; the second is non-functional.

DEFINITION

A **functional requirement** captures a piece of **functionality** the system must provide to its users, a specific behaviour, given some input, the system must produce some output. Formally, a functional

2.4. Functional and Non-Functional Requirements

requirement can be thought of as a function

$$f : I \rightarrow O$$

mapping some input I (e.g., a seller's request to add a product) to some output O (e.g., the product now appearing in the catalogue).

A **non-functional requirement** specifies **how the system should behave**, quality attributes and constraints on the functional behaviour, rather than a new capability itself. Common categories introduced in the course include **Reliability**, **Robustness**, **Performance**, **Portability**, and **Security**.

2.4.1 Reliability and Robustness

Term	Definition
Reliability	The extent to which a program behaves the <i>same way over time</i> , in the same operating environment (consistent, predictable behaviour).
Robustness	The extent to which a program can <i>recover from errors or unexpected input</i> , without crashing or producing garbage results.

★ INTUITION

Reliability is about *consistency*: does the system behave the same way every time, under normal conditions? Robustness is about *resilience*: what happens when something goes *wrong*, bad input, an unexpected error, a network failure? A system can be reliable but not robust (it behaves consistently under normal use, but crashes hard the moment something unusual happens), or robust but not reliable (it never crashes, but its normal behaviour is erratic). Good systems need both.

2.4.2 Comparison Table: Functional vs. Non-Functional

Functional Requirement	Non-Functional Requirement
Describes <i>what</i> the system does	Describes <i>how well/how</i> the system behaves
Expressed as a capability or feature	Expressed as a quality attribute or constraint
Usually verifiable with a pass/fail functional test	Usually verified with a measurement (time, uptime %, load capacity)
E.g. "A seller can add/edit/delete their catalogue"	E.g. "New products must appear within 5 seconds"

EXAM TIP

A very reliable trick used across many papers: look for a **number**, a **time limit**, a **percentage**, or a **scale** (“within 3 seconds”, “support 500 simultaneous users”, “99.9% uptime”), that is almost always the **non-functional** requirement. A requirement describing an *action a user can take* (“can search”, “can add”, “can view”) is almost always **functional**.

WORKED EXAMPLE

Scenario (adapted from Jan 2024 Q24 / Jan 2025 Q60). Four requirements are given for a mobile app: (1) support 100+ simultaneous requests, (2) let customers search for grocery items, (3) display different brand options, (4) generate an invoice within 1 second.

Reasoning. Requirements 2 and 3 describe *actions/capabilities* (search, display options), they are **functional**. Requirements 1 and 4 describe *numeric performance constraints* (simultaneous load, response time), they are **non-functional**. The correct statement is: “Requirements 2 and 3 are functional, whereas requirements 1 and 4 are non-functional.”

X COMMON MISTAKE

Watch for questions that put a *time constraint* inside what otherwise sounds like a functional description, e.g. “The app must notify users when an appointment is booked” (functional, this is an action/capability) versus “The app must process payments within 3 seconds” (non-functional, this constrains *how fast* an already-functional capability must work). Read each requirement carefully for whether it is introducing a new capability, or constraining an existing one.

QUIZ-ORIENTED NOTES

- Functional = $f : I \rightarrow O$, what the system does. Non-functional = how well/how it does it.
- Named non-functional categories: Reliability, Robustness, Performance, Portability, Security.
- Reliability = same behaviour over time. Robustness = recovers from errors/bad input.
- Numbers, time limits, and percentages in a requirement are a strong signal of **non-functional**.

◆ MEMORY TRICK

“WHAT vs HOW WELL”: if you can rephrase the requirement as “the user can ____”, it’s functional. If you can only rephrase it as “the system must be ____ (fast/available/secure/consistent)”, it’s non-functional.

2.5 The Software Requirements Specification (SRS) Document

> MOTIVATION

Requirements have now been gathered (Section 2.3), analysed for problems (Section 2.2.1), and classified as functional or non-functional (Section 2.4). But a pile of individually-classified requirements is still not very useful to a whole engineering team unless it is organised into one authoritative, shared document that everyone, developers, testers, managers, and the client, agrees to work from. In the **Plan-and-Document** perspective (recall Chapter 1, Section 1.6), this organisation happens through a single formal artefact: the SRS.

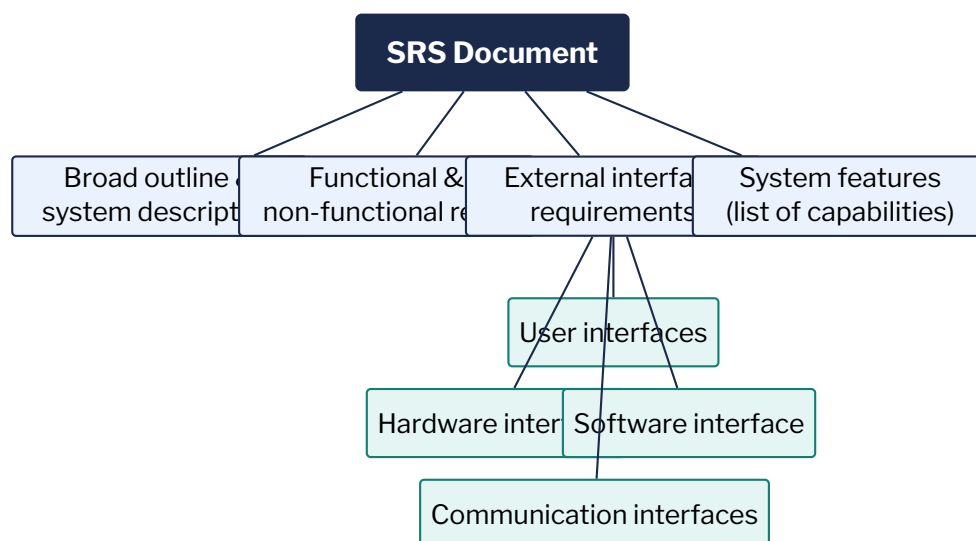
REAL-LIFE ANALOGY

An SRS document plays the same role for a software project that a signed architectural blueprint and contract play for a building project, it is the one document that both the client and the construction company agree describes exactly what will be built, so that later, if there is a dispute about whether the building matches what was promised, everyone can point to the same page.

DEFINITION

The **Software Requirements Specification (SRS)** is a formal document that organises all of a system's requirements. It is typically produced by a **system analyst**, working together with other members of the software team, and captures the requirements identified during gathering and analysis.

2.5.1 What an SRS Document Contains



- **Broad outline and description** of the software system, a high-level overview so a new reader understands the system's purpose before diving into detail.
- **Functional and non-functional requirements**, organised and classified as covered in Section 2.4.

- **External interface requirements**, everything the system must connect to outside itself:
 - *User interfaces (UI)*: how a human interacts with the system.
 - *Hardware interface*: how the system connects to physical devices.
 - *Software interface*: the connection points between this system and other software components (recall Chapter 1’s discussion of software clients and interfaces).
 - *Communication interfaces*: networking/protocol-level connections (e.g. how the system talks over the internet).
- **System features**, a concrete list of the system’s capabilities. For the Seller Portal, this list is exactly the requirement list built at the end of Section 2.3: manage catalogue, manage inventory, track orders, track payments, track inventory, track sales, track customer feedback.

2.5.2 Advantages of Maintaining an SRS

- Forms a formal **agreement between customers and developers** about exactly what will be built.
- **Reduces future rework**, because ambiguity and disagreement are ideally resolved before coding starts, not after.
- Provides a **basis for estimating costs and schedules**, since the full scope is written down.
- **Facilitates future extensions**, a clear baseline document makes it easier to reason about what changes when new features are added later.

EXAM TIP

When a PYQ asks about the “primary focus regarding clarity of requirements” in an SRS (Sep 2023 Q180), the correct answer is always about requirements being **clear, unambiguous, and understandable by both technical and non-technical stakeholders**, not “write in technical jargon for precision” and not “keep it vague for flexibility”; both of those directly undermine the entire purpose of having an SRS in the first place.

2.5.3 The Drawback of the SRS Approach

COMMON MISTAKE

The SRS is powerful, but it has one major weakness: it involves a **large amount of upfront documentation**. This works well *if* requirements are relatively fixed and unlikely to change (a regulated, well-understood domain, recall Boehm’s questions from Chapter 1, Section 1.7.6). It works poorly when requirements are expected to evolve, exactly the situation the Agile perspective was designed for.

This drawback is precisely why the course now pivots, just as Chapter 1 pivoted from Plan-and-Document models to the Agile perspective, to the **Agile version of requirement specification: Behaviour Driven Design and User Stories**, covered next.

QUIZ-ORIENTED NOTES

- SRS = the Plan-and-Document way of organising requirements; produced by a system analyst with the team.

- Four main contents: outline/description, functional & non-functional requirements, external interface requirements (UI, hardware, software, communication), system features.
- Four advantages: agreement, reduced rework, cost/schedule basis, easier future extension.
- One major drawback: heavy documentation overhead, best suited when requirements are relatively fixed.

■ REMEMBER THIS

An SRS is only as valuable as its clarity, an ambiguous SRS defeats its own purpose, because it was supposed to be the one document everyone could agree on.

2.6 Behaviour Driven Design and User Stories

► MOTIVATION

Section 2.5 ended on a problem: the SRS document works best when requirements are fixed, but real customers are often unsure of exactly what they need until they see something working (this is the same insight that motivated Agile in Chapter 1). The Agile perspective needed its own lightweight way of writing down requirements, one that stays useful even as understanding evolves throughout the project. That lightweight format is the **User Story**, and the philosophy behind it is **Behaviour Driven Design (BDD)**.

REAL-LIFE ANALOGY

An SRS is like a fully notarised legal contract, thorough, formal, hard to change. A user story is more like a sticky note pinned to a corkboard: quick to write, quick to read, easy to move around, re-prioritise, or rewrite as understanding improves. Both describe requirements, but they trade formality for flexibility in opposite directions.

2.6.1 Behaviour Driven Design (BDD)

DEFINITION

Behaviour Driven Design (BDD) asks questions about the *behaviour* of an application, both *before* and *during* development, and continuously refines requirements to keep meeting user expectations as they become clearer. The BDD way of expressing a requirement is the **User Story**.

2.6.2 User Stories

★ FREQUENTLY ASKED IN PYQs

The Role-Feature-Benefit template, and the SMART characteristics that follow it, are tested extremely frequently, both as direct recall and as “which SMART property does this change satisfy?” scenario questions. ★★★

DEFINITION

A **User Story** is a short, informal, plain-language description of something a user wants to do with the software, that is of **value** to them. It is meant to be the **smallest unit of work that can be completed within a single sprint**. User stories follow a fixed template:

As a [type of user],
I want [an action],
So that [a benefit / value].

**WORKED EXAMPLE****View Inventory (Seller Portal):**

Feature: View inventory
 As an independent seller,
 I want to view my inventory,
 So that I can take stock of products which are low in number.

Track Customer Feedback (Seller Portal):

Feature: Track customer feedback
 As an independent seller,
 I want to view my customers' feedback for each product,
 So that I can get a sense of pertinent issues in my products.

✗ COMMON MISTAKE

Jan 2026 Q7 offers close-sounding but wrong template names as distractors: “Feature–Role–Time”, “Input–Process–Output”, “Task–Module–Deadline.” The correct BDD user-story template is **Role–Feature–Benefit** (“As a ...I want ...So that ...”).

2.6.3 Benefits of User Stories

- **Lightweight**, quick to write and read compared to a full SRS.
- **Help plan and prioritise development**, developers can tell customers “feature X will be ready in 2 weeks” based on story-level estimates.
- **Focus on behaviour, not implementation**, a story says *what* value the user gets, not *how* the code will achieve it.
- Act as a **starting point for conversation** between users and the development team, a user story is deliberately incomplete on its own; it is meant to trigger a discussion (e.g. a seller reading the “track customer feedback” story might add, “I’d also like to be notified by email about 1- or 2-star ratings”).

★ INTUITION

A user story is often summarised with the phrase: *“a user story is a promise for a conversation, not a complete specification.”* This is the single biggest mental shift from the SRS mindset: instead of trying to write down every detail up front, the story captures just enough to know what to talk about next, and the details emerge through ongoing collaboration, exactly matching the Agile Manifesto’s “customer collaboration over contract negotiation.”

2.6.4 SMART User Stories

DEFINITION

A well-written user story should be **SMART**: **S**pecific, **M**easurable, **A**chievable, **R**elevant, **T**imeboxed.

Property	Meaning	Example transformation
Specific	The team knows exactly what to implement, no room for ambiguity.	<i>Vague</i> : “Search for a product.” → <i>Specific</i> : “Search for a product by title .”
Measurable	The expected results for given inputs are known/verifiable.	<i>Not measurable</i> : “good response time.” → <i>Measurable</i> : “appear within 3 seconds.”
Achievable	The story can realistically be implemented in one agile iteration (1–2 weeks); if not, subdivide it.	“View inventory” → split into “View all products on one page” <i>and</i> “Add pagination and filters.”
Relevant	The story delivers real business value to one or more stakeholders (ask “why” / “so what”).	A story with no clear stakeholder benefit should be reconsidered.
Timeboxed	Implementation stops once the time budget is exceeded; the team then gives up, subdivides, or reschedules the remainder.	A story that keeps expanding indefinitely needs to be split or rescheduled.

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q237 / Jan 2025 Q56 / Sep 2024 Q21). A user story cannot be implemented within a single agile iteration, so the team subdivides it into smaller stories (e.g. “Search catalog and place order” becomes “Search catalog” *and* “Place order”).

Reasoning. Subdividing a story specifically because it does not fit in one iteration is the textbook fix for the **Achievable** property (not Specific, Measurable, Relevant, or Timeboxed, those describe different problems entirely). Answer: **Achievable**.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q8). A user story's benefit clause is changed from "so that I can see driver details ...within a reasonable time" to "...within 4 seconds."

Reasoning. "A reasonable time" cannot be objectively verified; "within 4 seconds" can be checked precisely against real data. This is a change from an unverifiable statement to a verifiable, numeric one, the defining trait of the **Measurable** property. Answer: **Measurable**.

✗ COMMON MISTAKE

Students frequently confuse **Achievable** and **Timeboxed**, both relate to "time" in some sense. The distinguishing test: **Achievable** is about *planning ahead*, deciding *before* starting whether a story realistically fits in one iteration, and splitting it if not. **Timeboxed** is about *what happens during implementation* if the time budget set for a task is exceeded partway through (give up, split, or reschedule). If the scenario describes subdividing a story *up front* because it's clearly too big, that's Achievable. If it describes *stopping mid-task* because time ran out, that's Timeboxed.

2.6.5 Drawbacks of User Stories

- **Continuous contact with users is not always possible** in practice, the whole model assumes an available, engaged customer, which is not always realistic (recall Boehm's "are customers unavailable?" question from Chapter 1).
- **Does not scale well** to very large projects or safety-critical applications, where the lightweight, evolving nature of a user story is a liability rather than a strength (the exact opposite situation from where an SRS shines).

📌 EXAM TIP

Notice the elegant symmetry with Chapter 1's Boehm's-questions table: SRS/Plan-and-Document is favoured by large, regulated, safety-critical, geographically-distributed projects; User Stories/Agile are favoured by the opposite conditions. If a PYQ scenario mentions "safety-critical" or "large-scale, heavily regulated", that is a strong signal pointing toward the *drawbacks* of user stories, not their benefits.

◆ MEMORY TRICK

SMART: Specific (no ambiguity) → Measurable (numbers, not vague words) → Achievable (fits in one sprint, or split it) → Relevant (real value, ask "why") → Timeboxed (stop/split/reschedule when time runs out).

END OF CHAPTER 2: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- **Requirement gathering** = collecting info from stakeholders. **Requirement analysis** = checking for ambiguity, inconsistency, incompleteness.
- **User types**: Primary (direct/frequent), Secondary (via intermediary), Tertiary (not a user, but affected/influential).
- **Gathering techniques**: Questionnaires (many people, fixed Qs), Interviews (Structured/Unstructured/Semi-structured, individual depth), Focus groups (consensus & conflict), Naturalistic observation (real-world context), Documentation (existing rules/regulations).
- **Functional** = what the system does ($f : I \rightarrow O$). **Non-functional** = how well it does it (Reliability, Robustness, Performance, Portability, Security).
- **SRS** = formal Plan-and-Document artefact: outline, functional/non-functional reqs, external interfaces (UI/hardware/software/communication), system features. Advantages: agreement, less rework, cost/schedule basis, extensibility. Drawback: heavy documentation.
- **BDD / User Story** template: As a [user], I want [action], So that [benefit]. **SMART**: Specific, Measurable, Achievable, Relevant, Timeboxed.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

High-probability traps to watch for:

1. “Contradicting requirements from two stakeholders” $\Rightarrow$ Inconsistent (not ambiguous, not incomplete).
2. A courier/logistics partner who is contacted about an order but never uses the software $\Rightarrow$ Tertiary user (or, from Chapter 1’s client-type lens, simply not a direct client at all).
3. A requirement with a number/time/percentage attached $\Rightarrow$ almost always Non-functional.
4. Splitting a story *before* starting, because it clearly won’t fit one sprint $\Rightarrow$ Achievable. Stopping/rescheduling *during* a task because time ran out $\Rightarrow$ Timeboxed.
5. “Visited/shadowed/observed people working” $\Rightarrow$ Naturalistic observation, not Focus group and not Interview.
6. “Fixed set of questions asked identically to everyone” $\Rightarrow$ Structured interview (or Questionnaire, if sent to a *large* group rather than conducted as a live conversation).
7. SRS’s primary goal for clarity: understandable by *both* technical and non-technical stakeholders, never “vague for flexibility” and never “jargon-heavy for precision.”

PRACTICE QUESTIONS

- Q1.** A hospital is introducing a new patient-record system. List one plausible primary, one secondary, and one tertiary user, and justify each classification in one sentence.
- Q2.** A requirement states “the system must be easy to use.” Identify the requirement-analysis problem this illustrates, and rewrite it to fix the problem.
- Q3.** You need to understand how warehouse staff currently pack orders, but they are not able to clearly articulate their own process when asked directly. Which requirement-gathering technique would you choose, and why?
- Q4.** Classify each of the following as functional or non-functional, with a one-line justification: (a) “Users can reset their password.” (b) “The system must remain available 99.9% of the time.” (c) “Managers can approve leave requests.” (d) “The system must support 10,000 concurrent users.”
- Q5.** List the four main contents of an SRS document, and explain in one sentence why “external interface requirements” matters even for a system with no other software talking to it (hint: think about the user interface and hardware interfaces too).
- Q6.** Rewrite the following vague user story to be SMART: “As a seller, I want the app to be fast, so that I don’t get frustrated.”
- Q7.** A user story for “Generate monthly sales report” cannot be completed in one sprint. What SMART property is violated, and what is the standard fix?
- Q8.** Explain, using the Boehm’s-questions framework from Chapter 1, why safety-critical software is usually a poor fit for lightweight user stories and a better fit for a full SRS.
- Q9.** Compare an SRS document and a User Story along three dimensions: formality, flexibility to change, and intended audience.
- Q10.** A stakeholder says: “I want the checkout process to feel modern.” Identify which SMART property is missing, and propose a corrected, testable version of this requirement.

CHAPTER 3

Software User Interfaces

In this chapter: Interaction design · Usability goals · User experience (UX) goals · Prototyping techniques · Heuristic evaluation (Understanding, Action, Feedback).

3.1 Introduction to Interaction Design

► MOTIVATION

A working piece of software can still fail if people cannot figure out how to use it. Every one of us has felt this frustration, a confusing app, a button that does something unexpected, a form that gives no clue why it rejected our input. **Interaction Design** is the discipline that exists to prevent exactly this: designing software so that people can use it easily, effectively, and pleasantly.

REAL-LIFE ANALOGY

Compare a well-designed door handle to a badly-designed one. A good handle makes it obvious, just by looking at it, whether to push or pull. A bad one (a flat plate that actually needs pulling) causes even careful people to push it, resulting in the now-famous term “Norman doors.” Software interfaces have exactly the same problem: badly designed screens make people “push when they should pull.”

DEFINITION

Interaction Design is the process of designing interactive products (software, apps, devices) so that they support people effectively in their everyday and working lives. It proceeds through four core activities:

1. **Identifying needs and requirements** of the people who will use the product.
2. **Developing alternative designs** that meet those requirements.
3. **Building interactive versions** of the designs, so they can be communicated and assessed.
4. **Evaluating** what is built throughout the process.

★ INTUITION

Notice this loop is a smaller, UI-focused echo of the SDLC from Chapter 1 (Requirements → Design → Development → Testing): identify needs $\approx$ requirements, alternative designs $\approx$ design, build interactive versions $\approx$ development, evaluate $\approx$ testing. Interaction design applies the same discipline specifically to the parts of a system that humans directly touch.

QUIZ-ORIENTED NOTES

Four activities of interaction design: **Identify needs** → **Design alternatives** → **Build interactive versions** → **Evaluate**. This cycle repeats, evaluation feeds back into new design ideas.

3.2 Usability Goals

★ FREQUENTLY ASKED IN PYQs

Usability-goal identification from a scenario is one of the most frequently repeated question types in this chapter's material (Jan 2023 Q238, Jan 2024 Q28, May 2023 Q6, May 2024 Q6, Sep 2022 Q119, Jan 2026 Q8, Sep 2025 Q9, May 2025 Q111). ★★★

> MOTIVATION

“Make the interface usable” is not specific enough to actually design or test against. The course, following the ISO standard, breaks “usability” down into five concrete, checkable goals, so that a team can ask precisely *which* quality is missing when something feels wrong.

DEFINITION

Per the **ISO 9241-11** standard, **usability** is “*the extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use.*” This is broken down into five **usability goals**: **Effectiveness**, **Efficiency**, **Safety**, **Learnability**, **Memorability**.

Goal	Meaning
Effectiveness	How good the system is at doing what it's supposed to, letting people learn, work efficiently, access information, or buy what they want.
Efficiency	How well the system supports users in completing common tasks through a <i>minimal number of steps</i> .
Safety	Protecting the user from dangerous/undesirable situations (e.g. X-ray machines, chemical plants) <i>and</i> helping users avoid accidentally performing unwanted actions (reducing wrong key-presses, and providing a means of recovery from mistakes).
Learnability	How easy the system is to learn to use, so a new user can get started and be productive with little effort.
Memorability	How easy the system is to <i>remember how to use</i> , once it has already been learned.

✗ COMMON MISTAKE

Learnability vs. Memorability are the pair students confuse most. Learnability is about a *first-time* user figuring things out. Memorability is about a user who *already knew* the system, but is returning to it after a gap (e.g. a feature used only once a year), can they still remember how without relearning from scratch?

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q238). An image-editing app prompts new users to take a guided, step-by-step tour after installation.

Answer. Learnability, a guided tour is a classic learnability aid for first-time use.

WORKED EXAMPLE

Scenario (adapted from May 2024 Q6). An email app discards a drafted, unsaved email automatically if the app is closed while composing.

Answer. Safety is the goal *not* fulfilled here, the app offers no means of recovery from an accidental close, exactly the “provide a means of recovery if you made errors” failure case.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q9). Users take a long time to figure out how to add or delete a patient record in a new interface, and no tutorial is provided.

Answer. Learnability is missing, “takes a long time to figure out” with no guidance is precisely the learnability failure signature (not Efficiency, which is about *steps* for someone who already knows how, not about first-time discovery).

WORKED EXAMPLE

Scenario (adapted from May 2025 Q54). A retail website reduces the number of steps from product selection to payment; a returning customer with saved payment details can complete a purchase in a single click.

Answer. Efficiency, fewer steps to complete a routine task for someone who already knows the system is exactly what efficiency measures.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q16). A word processor deliberately places the “Save” and “Close” buttons far apart on screen, so a hurried user is less likely to accidentally close the document instead of saving it.

Answer. Safety, this is the positive/preventive mirror of the “unsaved email discarded” failure case above: here the interface is *proactively* designed to reduce the chance of an accidental, undesirable action, exactly the “protect the user from dangerous/undesirable situations” half of the Safety definition.

★ INTUITION

Usability goals and heuristics (Section 3.5) are two different lenses on the *same* underlying design problem, one names the *goal*, the other names the *technique* used to reach it. A very common End Term pattern makes this link explicit: a web application replaces its icons with unfamiliar custom pictures (e.g. random animal pictures instead of standard icons), users then **cannot remember** what

each icon does and must repeatedly consult a manual. Read purely as a heuristic question, this violates **Familiar Language and Metaphors** (Section 3.5). Read purely as a usability-goal question, the same scenario is a **Memorability** failure (not Learnability, the problem persists *every time* the user returns, it isn't just a first-use hurdle). Expect this exact scenario to be asked both ways.

🔑 EXAM TIP

Fast decision rule: mentions of a *guided tour, tutorial, onboarding, first-time use* → Learnability. Mentions of *steps, clicks, shortcuts, minimal effort for a routine task* → Efficiency. Mentions of *recovery from a mistake, avoiding accidental/dangerous actions* → Safety. Mentions of *remembering after a long gap, or unfamiliar/custom icons causing repeated confusion* → Memorability. Mentions of *whether the system can accomplish the goal at all* → Effectiveness.

3.3 User Experience (UX) Goals

DEFINITION

User Experience (UX) goals concern how a product makes users *feel*: satisfying, enjoyable, fun, helpful, entertaining, motivating, emotionally fulfilling. Unlike usability goals (which are largely objective and measurable), UX goals are **more subjective**, they describe how users experience a product *from their own perspective*.

★ INTUITION

Usability asks “can the user accomplish the task, efficiently and safely?” UX asks the deeper question “how did it *feel* to accomplish it?” A system can be perfectly usable (fast, safe, learnable) and still feel cold or joyless; conversely, a slightly less efficient system can still leave users delighted. Good interaction design pursues *both* simultaneously, usability is close to necessary, UX is what turns “usable” into “loved.”

QUIZ-ORIENTED NOTES

Usability goals are largely **objective/measurable** (Effectiveness, Efficiency, Safety, Learnability, Memorability). UX goals are **subjective/emotional** (satisfying, enjoyable, fun, motivating). A PYQ distractor list (Sep 2023 Q181) sometimes slips in “Novelty” as a fake usability goal, it is *not* one of the five standard usability goals.

3.4 Prototyping Techniques

★ FREQUENTLY ASKED IN PYQs

Identifying the prototype *type* from a scenario is tested very frequently (Jan 2026 Q9, May 2023 Q7, May 2025 Q100, Sep 2024 Q7, Sep 2025 Q10, Sep 2023 Q182). ★★★

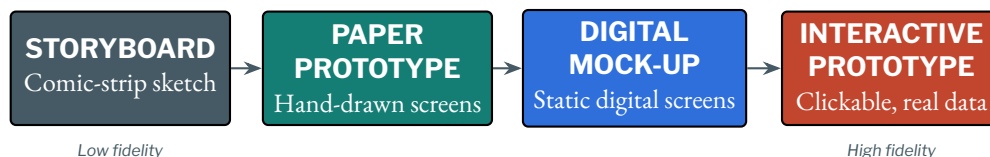
> MOTIVATION

Should a team start writing production code the moment a design idea appears? No, for the same reason Chapter 1's Prototype Model exists: initial ideas are often not the best ones, and building a full interface is expensive in time, money, and effort. **UI Prototypes** let designers quickly test ideas on real users, gather feedback, iterate, and pivot, all before committing to expensive, hard-to-change code.

DEFINITION

A **prototype** answers open design questions and helps designers choose between alternatives. Its purposes are to: test the **technical feasibility** of an idea, **clarify vague requirements**, and support **user testing and evaluation**.

3.4.1 The Fidelity Spectrum: Four Named Prototype Types



Type	Description
Storyboard	A hand-drawn <i>comic</i> capturing Setting (people/environment/task), Sequence (steps, what leads someone to use the app), and Satisfaction (what motivates use, what need it fills). Emphasises <i>how</i> the interface accomplishes a task, while deliberately avoiding commitment to any one particular UI design.
Paper Prototype	A hand-drawn or printed interface, presented as multiple physical pieces of paper , one per screen, that can be swapped by hand to simulate navigation.
Digital Mock-up	A static digital representation of screens (e.g. built in a slide/design tool), often shown with fabricated (fake) data , not yet truly interactive.
Interactive Prototype	A clickable prototype that closely simulates the real, final look-and-feel, giving a close-to-real experience (this is what “high-fidelity interactive prototyping” refers to).

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q9). Multiple screens of a mobile app are drawn on paper, and switching between screens is simulated manually during discussions with users.

Answer. Paper Prototyping, physical paper screens swapped by hand is the defining trait (not Storyboarding, which is a single comic-strip narrative, not separate interchangeable screens).

WORKED EXAMPLE

Scenario (adapted from May 2025 Q100). A startup creates *clickable images* of app screens using PowerPoint and asks users to try placing a test order.

Answer. Interactive Prototype, “clickable” plus letting users actually attempt a task is the signature of an interactive prototype, even though it was built quickly in a slide tool rather than real code.

WORKED EXAMPLE

Scenario (adapted from Sep 2024 Q7). A prototype is presented as a web application along with some fabricated data.

Answer. Digital Mock-up, a web-based (digital) presentation with *fake/fabricated* data, but the question doesn’t indicate it responds to user clicks the way an interactive prototype would.

X COMMON MISTAKE

Storyboard vs. Paper Prototype is a frequent trap: both are hand-drawn. The distinguishing test: a **storyboard** tells *one narrative story* (a comic strip showing setting/sequence/satisfaction) and is used to build shared understanding *before* any UI exists. A **paper prototype** is a set of *separate, interchangeable screens* meant to be manipulated to simulate actually using the interface.

3.4.2 Benefits of Storyboards

- Emphasises *how* the interface accomplishes a task.
- Avoids commitment to a particular user interface (keeps early discussion open-ended).
- Builds a shared understanding among stakeholders.

3.4.3 Benefits of Paper Prototypes

- Far easier than writing code just to test a UI idea.
- Starts a real conversation about user interactions early.
- Elements can be changed *immediately*, right in front of the user, based on their feedback.

EXAM TIP

Sep 2023 Q182 asks which technique gives “a close-to-real experience, simulating the look and feel of the final product”, the answer is **high-fidelity interactive prototyping**, not low-fidelity paper prototyping, which deliberately looks rough and unfinished.

QUIZ-ORIENTED NOTES

- Purposes of prototyping: technical feasibility, clarifying vague requirements, user testing/evaluation.
- Fidelity spectrum: Storyboard → Paper Prototype → Digital Mock-up → Interactive Prototype (low → high fidelity).
- “Wizard of Oz prototyping” (a human secretly operates the “intelligent” parts behind the scenes) is another named low-effort technique for simulating functionality that doesn’t exist

yet.

3.5 Heuristic Evaluation

★ FREQUENTLY ASKED IN PYQs

Matching a design scenario to the correct Nielsen-style heuristic is, by a wide margin, the single most frequently and diversely tested topic across every quiz paper analysed. ★★★

➤ MOTIVATION

Once a prototype exists, how do you judge whether its UI is actually good? You could ask real users (expensive and slow to arrange every time) or bring in expert designers to critique it. Expert critique risks being purely subjective, unless the experts use a shared, agreed checklist. That checklist is **Heuristic Evaluation**.

DEFINITION

Heuristics are strategies (rules of thumb) derived from previous experience with similar design problems. **Heuristic Evaluation** has expert evaluators inspect a prototype against this checklist, usually in **multiple independent passes**, producing a list of issues that violate one or more heuristics. The set used in this course groups Nielsen-style heuristics into three themes: **Understanding**, **Action**, and **Feedback**.

3.5.1 Heuristics for Understanding

Heuristic	What it means
Consistency	Maintain uniformity in how similar actions or elements are presented (consistent layout, consistent naming), so learning one part of the interface transfers to the rest.
Familiar language & metaphors	Reuse concepts users already know from the real world (e.g. a “shopping cart”, a “mobile wallet”), instead of inventing unfamiliar new terms.
Clean, functional design	Keep pages simple and easy to understand, avoid unnecessary clutter that makes the interface harder to parse.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q14). A banking app places the “Transfer Money” button in the *same position*, with the *same label and icon*, across the dashboard, accounts, and payments screens.

Answer. Consistency, identical placement, label, and icon across every screen is exactly what lets a user’s learning on one screen transfer automatically to the rest.

WORKED EXAMPLE

Scenario (adapted from May 2025 Q41). A mobile payment app labels the “add money to wallet” action as “*Charging Wallet*” instead of the far more familiar “*Add Money*.” Users find the term confusing.

Answer. Familiar Language and Metaphors is violated, “Charging Wallet” is not a term users already know from everyday life, whereas “Add Money” reuses a concept (adding money to a purse/account) they already understand.

WORKED EXAMPLE

Scenario (adapted from Sep 2022 Q121). Designing a medicine-selling website so pages are simple and easy to understand for end users.

Answer. Clean and functional design.

✗ COMMON MISTAKE

Consistency and Familiar Language/Metaphors are easy to conflate, both are about “things users already know.” **Consistency** is *internal*: does the interface behave the same way as *itself*, screen to screen? **Familiar metaphors** is *external*: does the interface borrow language/concepts users already know from *the real world or other apps*, rather than inventing new jargon? A button in the same place every time is Consistency; a button labelled with an everyday word instead of invented jargon is Familiar Metaphors.

3.5.2 Heuristics for Action

Heuristic	What it means
Freedom	<i>Freedom to undo</i> (recover a deleted file from trash, Ctrl+Z, unsend a message) and <i>freedom to explore</i> (browse without being forced into a commitment, e.g. viewing products or prices without logging in).
Flexibility	Both experts and new users can work efficiently. Includes personalisation (the system tailors content for an individual automatically, e.g. social-media feeds) and customisation (the user explicitly chooses how the product should work, e.g. selecting news topics).
Recognition over recall	Users find it easier to <i>recognise</i> something they’ve seen before than to <i>recall</i> it from memory unaided (e.g. right-clicking a file to move it, instead of needing to remember and type a command like <code>mv temp.txt new_folder/</code>).

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q239 / May 2023 Q8 / Jan 2026 Q10). A food-delivery or ticket-booking app lets users browse and explore without signing in, but requires sign-in only when they finally place an order.

3.5. Heuristic Evaluation

Answer. Freedom (specifically, freedom to explore), users are not forced into a commitment just to look around.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q11). A new user navigates menus to copy text, while an experienced user simply presses Ctrl+C.

Answer. Flexibility, the interface supports both a novice's menu-driven path and an expert's shortcut-driven path to the same outcome.

WORKED EXAMPLE

Scenario (adapted from Sep 2022 Q125 / Jan 2024 Q36). An IDE that flags a syntax error *and also suggests* possible corrections; a password-reset form that flags a mismatched confirmation field and asks the user to re-enter it.

Reasoning. Careful reading matters here: *merely flagging* the mismatch is Prevent Errors / Show Status territory, but the IDE scenario specifically emphasises *suggesting corrections* the user can pick from without having to recall the right fix themselves, this is **Recognition over recall**.

X COMMON MISTAKE

"Browsing without logging in" is **Freedom** (freedom to explore); it is *not* Flexibility. Flexibility is about accommodating *different kinds of users* (novice vs. expert) or letting a user *customise* settings, not about removing a login barrier.

3.5.3 Heuristics for Feedback

Heuristic	What it means
Show status	Keep users informed about what is happening, within a reasonable time (e.g. a loading indicator, an estimate of how long an action will take, warnings provided <i>in advance</i> , clear next-step prompts after signup).
Prevent errors	Reduce the chance users make a mistake in the first place, e.g. helpful constraints (greying out an invalid option) or offering suggestions before submission.
Support error recovery	When an error does happen: make the problem clear, provide a solution, and provide an alternative path forward.
Provide help	Make help easy to search, and provide help <i>in context</i> (right where the user needs it, not buried in a separate manual).

WORKED EXAMPLE

Scenario (adapted from Jan 2025 Q57). A food-delivery app gives no real-time updates about order or delivery progress.

Answer. Show status is the missing heuristic.

WORKED EXAMPLE

Scenario (adapted from Sep 2024 Q14). A leave-application calendar picker cannot highlight holidays separately, so employees risk mistakenly applying for leave on a holiday.

Answer. Prevent errors, the interface fails to stop a likely mistake *before* it happens (this is different from *recovering* from the mistake afterward).

EXAM TIP

Prevent errors acts *before* a mistake (constraints, warnings, suggestions). **Support error recovery** acts *after* a mistake has already happened (clarify, fix, offer an alternative). **Show status** is about ongoing, real-time visibility into what the system is doing, whether or not anything has gone wrong at all.

MEMORY TRICK

Group the ten heuristics by **what question they answer**: *Understanding* → “will I get it?” (Consistency, Familiar metaphors, Clean design). *Action* → “can I move freely?” (Freedom, Flexibility, Recognition over recall). *Feedback* → “does it talk back to me?” (Show status, Prevent errors, Support recovery, Provide help).

END OF CHAPTER 3: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- **Interaction design cycle:** Identify needs → Design alternatives → Build interactive versions → Evaluate.
- **Usability goals (ISO 9241-11):** Effectiveness, Efficiency, Safety, Learnability, Memorability.
- **UX goals:** subjective, emotional (satisfying, enjoyable, motivating).
- **Prototype fidelity spectrum:** Storyboard → Paper Prototype → Digital Mock-up → Interactive Prototype.
- **Heuristics for Understanding:** Consistency, Familiar language/metaphors, Clean & functional design.
- **Heuristics for Action:** Freedom (undo/explore), Flexibility (novice+expert, personalise/-customise), Recognition over recall.
- **Heuristics for Feedback:** Show status, Prevent errors, Support error recovery, Provide help.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. “Guided tour / tutorial for first-time users” ⇒ Learnability. “Remembering after a long gap” ⇒ Memorability.
2. “Browsing/exploring without login” ⇒ Freedom, not Flexibility.
3. “Works for both experts and novices” or “personalisation/customisation” ⇒ Flexibility.
4. “Suggests options so the user doesn’t need to recall the exact command” ⇒ Recognition over recall.
5. “Stops a mistake before it happens” ⇒ Prevent errors. “Fixes/guides after a mistake” ⇒ Support error recovery.
6. Storyboard = one narrative comic. Paper prototype = separate swappable hand-drawn screens. Digital mock-up = static digital + fake data. Interactive prototype = clickable, close to real.

PRACTICE QUESTIONS

- Q1.** A ride-hailing app shows a spinning loader and “Finding your driver, usually takes 2 minutes” while matching a driver. Which heuristic for feedback is being applied?
- Q2.** A note-taking app can be fully driven by keyboard shortcuts for power users, while still offering menu buttons for everyone else. Which heuristic is this, and which usability goal does it primarily support for new users?
- Q3.** Distinguish Effectiveness from Efficiency using an example of your own.
- Q4.** A team is unsure whether users will prefer a bottom navigation bar or a side drawer menu. Which prototyping technique would let them test this cheaply, and why?
- Q5.** Explain, in one sentence each, why Storyboards “avoid commitment to a particular user interface” while Paper Prototypes do not.
- Q6.** A signup form rejects a password with no explanation of which rule was broken. Which heuristic for feedback is violated, and how would you fix it?
- Q7.** Give one example each of personalisation and customisation, and explain the difference in one sentence.

CHAPTER 4

Software Project Management

In this chapter: Project management overview · Project estimation (Empirical: Expert Judgment, Delphi; Heuristic: COCOMO) · Project scheduling (WBS, Activity Network, Critical Path, Gantt charts) · Risk management · Project management in Agile (Scrum roles, ceremonies, Points & Velocity).

4.1 Project Management Overview

► MOTIVATION

Every idea covered so far, requirements, design, development, testing, UI, still needs to be organised into an actual, executable plan: who does what, by when, at what cost, and what happens if things go wrong. That coordinating role belongs to the **software project manager**.

DEFINITION

A **Software Project Manager**'s key responsibilities are: acting as the **main point of contact** between clients and the development team, **forming and managing** the development team, **project scheduling**, **project estimation**, **risk management**, and **configuration management** (tracking and controlling versions of code, documents, and other project artefacts as they change).

QUIZ-ORIENTED NOTES

Six core responsibilities: point of contact, team formation, scheduling, estimation, risk management, configuration management. The rest of this chapter is organised around exactly these responsibilities.

4.2 Project Estimation Techniques

► MOTIVATION

Before a client will commit to a project, they usually need to know roughly what it will cost and how long it will take, especially when a company is **bidding** for the work against competitors. Estimation must happen *early*, precisely when the least is known about the project, which is what makes it hard.

DEFINITION

Project estimation establishes the expected **cost** and **schedule** of a project. Two key parameters are estimated:

- **Size:** measured in **lines of code (LOC)**, or thousands of lines of code, **KLOC**.
- **Effort:** measured in **person-months (PM)**, the amount of work one person can typically do in one month. E.g. a project needing 12 person-months could be done by 4 developers in 3 months, or 2 developers in 6 months.

Estimation techniques fall into two families:

Family	Approach
Empirical	Ask people who have completed similar projects before (Expert Judgement, Delphi Technique).
Heuristic	Model effort using a mathematical expression fitted from past project data (COCOMO).

4.2.1 Empirical Technique 1: Expert Judgement

DEFINITION

Expert Judgement is an educated guess, one or more experts estimate the cost of different components and combine the estimates.

Drawbacks: human error and individual bias, tendency toward overly optimistic estimates, and the risk of overlooking factors due to a single expert's limited knowledge. The natural fix, ask a *group* of experts, leads directly to the next technique.

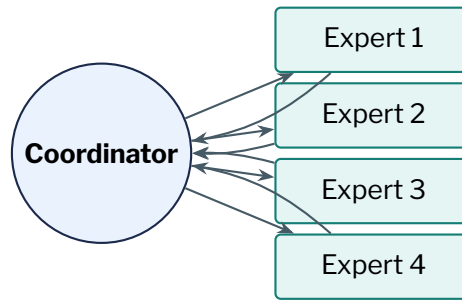
4.2.2 Empirical Technique 2: The Delphi Technique

★ FREQUENTLY ASKED IN PYQs

The Delphi Technique's purpose is directly tested (Sep 2023 Q184). ★☆☆

DEFINITION

The **Delphi Technique** combines estimates from **multiple experts** through a coordinator, over several rounds, to reach a consensus estimate, without the experts ever needing to argue face-to-face.



Four steps: (1) Coordinator sends each expert the SRS plus a form for recording a cost estimate. (2) Experts submit their individual estimates back to the coordinator. (3) The coordinator prepares an anonymised **summary** of all estimates and redistributes it to every expert (so each expert sees how their estimate compares to the group, without knowing who gave which number). (4) This process **iterates over several rounds**; the coordinator compiles the results and prepares a final consensus estimate.

4.3 The COCOMO Estimation Model

★ FREQUENTLY ASKED IN PYQs

Numeric COCOMO calculations appear in nearly every quiz paper, almost always as a comprehension-style, multi-part question. ★★★

➤ MOTIVATION

Expert-based techniques still depend heavily on subjective judgement. What if effort could instead be estimated from a formula, calibrated using data from many real, completed projects?

DEFINITION

COCOMO (COⁿstructive CO^st MO^del), proposed by **Barry Boehm in 1981**, estimates effort using:

$$\text{Effort} = a \times (\text{SIZE in KLOC})^b \quad \text{Person-Months}$$

where a and b are constants that depend on the **type of project**.

Project type	Characteristics	Formula
Organic	Well-understood application; small, experienced team	$\text{Effort} = 2.4 (\text{KLOC})^{1.05}$
Semi-detached	Mix of experienced + inexperienced people	$\text{Effort} = 3.0 (\text{KLOC})^{1.12}$
Embedded	Strongly coupled with hardware	$\text{Effort} = 3.6 (\text{KLOC})^{1.20}$

EXAM TIP

Memorise the pattern, not just the numbers: as projects move Organic → Semi-detached → Embedded, *both a and b increase*. This makes sense: harder, less-understood, hardware-coupled projects cost disproportionately more effort per line of code.

4.3.1 Worked Example: The Amazon Seller Portal

The Seller Portal is classified as a **semi-detached** project ($a = 3.0, b = 1.12$).

WORKED EXAMPLE

Step 1: Estimate size. Summing the estimated LOC for every system feature identified back in Chapter 2:

Add/Edit/Delete catalogue	300 LOC
Add/Edit/Delete inventory	300 LOC
Track orders	1000 LOC
Track payments	800 LOC
Track inventory	800 LOC
Track sales	500 LOC
Track customer feedback	300 LOC
Total	4000 LOC = 4 KLOC

Step 2: Initial estimate.

$$\text{Effort} = 3.0 \times (4)^{1.12} = 15.83 \text{ Person-Months}$$

Step 3: Convert to cost. At a rate of Rs. 75,000 per person-month:

$$\text{Cost} \approx 16 \times 75,000 = \text{Rs. 12 lakhs (approx.)}$$

4.3.2 Refining the Estimate: Cost Drivers and the Effort Adjustment Factor

The **initial estimate** above only accounts for size. Real effort also depends on factors like **people, technical attributes**, and **tools/practices used**. COCOMO captures this with **15 cost driver attributes** (e.g. required reliability, product complexity, analyst capability, use of software tools), each rated on a scale, that combine into a single **Effort Adjustment Factor (EAF)**:

$$\text{Final Estimate} = \text{Initial Estimate} \times \text{EAF}$$

WORKED EXAMPLE

Continuing the Seller Portal example: if the relevant cost-driver ratings multiply out to an EAF of $1.40 \times 1.08 \times 1.13 \times 0.95 = 1.62$, then:

$$\text{Final Estimate} = 15.83 \times 1.62 \approx 25.65 \text{ Person-Months}$$

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q11). A project has an initial COCOMO estimate of 20 person-months, and an EAF of 1.5.

Answer. Final estimate = $20 \times 1.5 = 30$ person-months.

× COMMON MISTAKE

The EAF **multiplies** the initial estimate, it does not add to it, and it is not itself a percentage subtracted or added. An EAF below 1 *reduces* the estimate (favourable cost drivers, e.g. an experienced team with good tools); an EAF above 1 *increases* it.

4.3.3 Worked Examples: Organic and Embedded Projects

WORKED EXAMPLE

Organic project (adapted from an End Term paper). A small e-commerce company, with a **small, experienced** development team and **well-understood** requirements (product listings, shopping cart, order processing, payment gateway), builds a website with **44,000 lines of code** (44 KLOC).

Step 1: classify. Small experienced team + well-understood requirements is the textbook signature of an **Organic** project ($a = 2.4, b = 1.05$).

Step 2: initial estimate.

$$\text{Effort} = 2.4 \times (44)^{1.05} \approx \mathbf{128 \text{ Person-Months}}$$

Step 3: convert to cost. At Rs. 50,000 per developer per month:

$$\text{Cost} = 128 \times 50,000 = \mathbf{\text{Rs. } 64,00,000 \text{ (approx.)}}$$

WORKED EXAMPLE

Embedded project (adapted from an End Term paper). A smart meter's software must be **strongly coupled with hardware** (recording consumption, monitoring behaviour, automatic billing), with **2,00,000 lines of code** (200 KLOC) and an EAF of 1.48.

Step 1: classify. "Strongly coupled with hardware" is the textbook signature of an **Embedded** project ($a = 3.6, b = 1.20$).

Step 2: initial estimate.

$$\text{Effort} = 3.6 \times (200)^{1.20} \approx \mathbf{2,079 \text{ Person-Months}}$$

Step 3: final estimate.

$$\text{Final Estimate} = 2,079 \times 1.48 \approx \mathbf{3,075 \text{ Person-Months}}$$

EXAM TIP

The scenario itself tells you which project type to use, look for the exact signal words: "small, experienced team, well-understood requirements" → **Organic**. "Mix of experienced and inexperienced people" → **Semi-detached**. "Strongly coupled with hardware" (embedded systems, IoT devices,

4.4. Project Scheduling

sensors/actuators) → **Embedded**. Getting the project type right matters more than the arithmetic, using the wrong (a, b) pair changes every downstream number.

4.3.4 COCOMO Model Summary

1. Determine the type of product (Organic / Semi-detached / Embedded).
2. Estimate the size (LOC/KLOC).
3. Compute the **initial estimate** using $a \times \text{SIZE}^b$.
4. Compute the **Effort Adjustment Factor** from the 15 cost drivers.
5. Compute the **overall (final) estimate** = initial $\times$ EAF.

QUIZ-ORIENTED NOTES

- Effort = $a \times (\text{KLOC})^b$; a, b increase from Organic → Semi-detached → Embedded.
- Final estimate = Initial estimate $\times$ EAF (multiplicative, not additive).
- Despite COCOMO's mathematical precision, real-world surveys cited in the course note that **92% of project managers** actually estimate primarily from **experience**, not formulas, a useful reminder that no single technique dominates in practice; choice of technique depends on the type of organisation and product.

4.4 Project Scheduling

★ FREQUENTLY ASKED IN PYQs

Work Breakdown Structure and Activity Network / Critical Path questions appear regularly (May 2024 Q12, Sep 2024 Q18, Sep 2025 Q23). ★★☆☆

► MOTIVATION

An estimate of total effort (e.g. “16 person-months”) does not by itself tell you *when* things will be done, which tasks can run in parallel, or which task, if delayed, delays the whole project. **Scheduling** turns an effort estimate into an actual, trackable timeline.

DEFINITION

Project scheduling matters because it lets a manager **monitor timely completion** of tasks and **take corrective action** early, before a small delay becomes a large one.

4.4.1 Main Activities in Scheduling

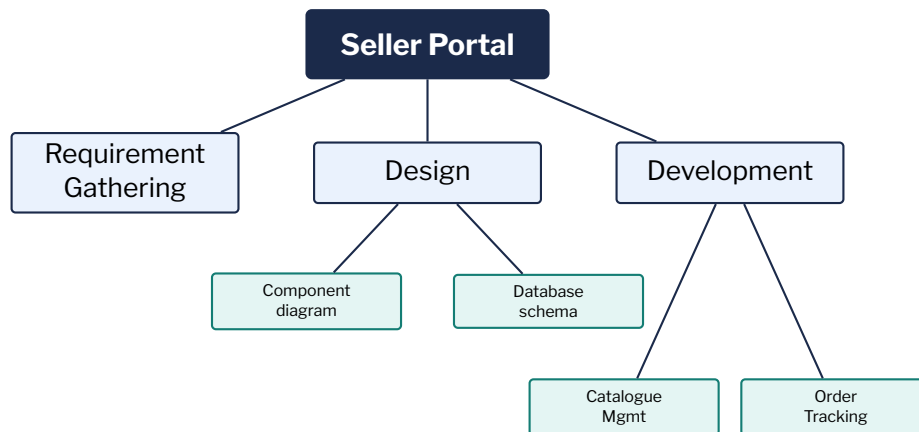
1. Identify all major activities.
2. Break down each activity into smaller tasks.
3. Determine the **dependencies** among tasks.
4. Estimate the **time duration** required for each task.

5. Represent this information visually (chart, graph, network).
6. Determine task **start and end dates** from that representation.
7. Determine the **critical path**, the chain of tasks that determines the overall project duration.
8. Allocate resources (people) to tasks.

4.4.2 Work Breakdown Structure (WBS)

DEFINITION

A **Work Breakdown Structure (WBS)** is a **tree-shaped** decomposition of a project: the **root** is the project itself, each internal node is broken down into smaller child activities, and each **leaf** represents a concrete **task** (roughly two weeks of work) that can be assigned to one developer and scheduled.



4.4.3 Activity Networks and the Critical Path

DEFINITION

An **Activity Network** takes the leaf tasks of the WBS and arranges them as nodes in a graph, showing each task's **estimated duration** and its **dependencies** (which tasks must finish before a given task can start). The **critical path** is the longest chain of dependent tasks from start to finish; it determines the *minimum possible* duration of the whole project, delaying any task on the critical path delays the entire project, while tasks *off* the critical path have **slack** (some room to slip without affecting the overall deadline).

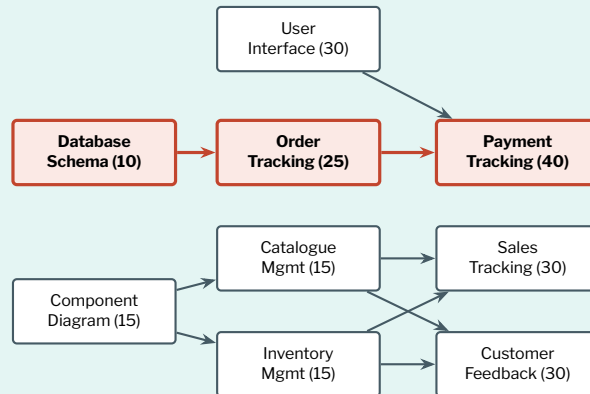
WORKED EXAMPLE

Worked example (Seller Portal, task durations in days). Suppose the design/development tasks and their dependencies are:

- **Database Schema** (10 days) and **User Interface** (30 days) and **Component Diagram** (15 days) can all start immediately.
- **Order Tracking** (25 days) needs Database Schema finished first.
- **Payment Tracking** (40 days) needs *both* Order Tracking *and* User Interface finished first.

4.4. Project Scheduling

- **Catalogue Management** (15 days) and **Inventory Management** (15 days) each need Component Diagram finished first.
- **Sales Tracking** (30 days) and **Customer Feedback** (30 days) each need Catalogue Management and Inventory Management finished first.



Forward pass (earliest finish times): Database Schema finishes at day 10; Order Tracking then finishes at $10 + 25 = 35$; Payment Tracking needs *both* Order Tracking (day 35) and User Interface (day 30), so it starts at $\max(35, 30) = 35$ and finishes at $35 + 40 = 75$. Meanwhile, Component Diagram (day 15) → Catalogue/Inventory Management (day 30) → Sales Tracking or Customer Feedback (day 60).

Critical path = Database Schema → Order Tracking → Payment Tracking = $10 + 25 + 40 = 75$ days (highlighted in red above), this is the longest chain, so the *entire project* takes at least 75 days. User Interface (30 days) has **5 days of slack**, it could take up to 35 days without delaying the project, since Payment Tracking cannot start before day 35 anyway (it is waiting on Order Tracking).

EXAM TIP

To find the critical path by hand: compute the earliest-finish time of every task by walking forward through dependencies (a task's start = the *latest* finish among all tasks it depends on). The path that produces the largest overall project finish time is the critical path. Any task *not* on that path has slack equal to the difference between its own finish time and the time its successor was actually forced to wait for.

4.4.4 Gantt Charts

DEFINITION

A **Gantt Chart** represents a schedule as a set of horizontal bars, one per task, plotted against a timeline, showing each task's start date, duration, and (via overlaps) which tasks run in parallel. It is a visual, easy-to-communicate alternative representation of the same scheduling information captured in the activity network.

X COMMON MISTAKE

A bar's **length represents the task's duration**, not an individual employee's working hours, and a Gantt chart on its own does **not** replace the **Activity Network**: the Activity Network's job is to reveal *dependencies* (which tasks must wait for which) and compute the *critical path*; the Gantt chart's job is to give an easy-to-read, at-a-glance *timeline view* once that scheduling work is done. They are complementary, not substitutes for one another, and neither replaces the **Work Break-down Structure** (Section 4.4), which is a separate, earlier step: WBS *decomposes* the project into tasks, the Activity Network then *sequences and times* those same tasks.

QUIZ-ORIENTED NOTES

- WBS = tree (root = project, leaves = ~2-week tasks). Activity Network = graph of leaf tasks with durations & dependencies. Critical path = longest dependency chain = minimum project duration.
- A task on the critical path has **zero slack**; delaying it delays the whole project.
- Gantt chart = bar-chart view of the same schedule, easier to read at a glance; bar length = duration, not per-person hours.
- WBS, Activity Network, and Gantt chart are three complementary views, none of them replaces another.

4.5 Risk Management

★ FREQUENTLY ASKED IN PYQs

Classifying a scenario as Technical/Project/Business risk, and matching it to the correct mitigation, is tested extremely frequently and consistently (Jan 2023, May 2023, Sep 2022, Jan 2024, May 2024, May 2025, Jan 2026). ★★ ★

> MOTIVATION

Software projects rarely go exactly to plan. Code is intangible (unlike a physical building, you cannot see 60% of a program “standing”), third-party components may be unreliable, and teams are made of people who can leave, disagree, or make mistakes. **Risk management** exists to anticipate these problems *before* they derail the project.

DEFINITION

A **risk** is an anticipated unfavourable event or circumstance that could occur while a project is underway. The course groups risks into three categories: **Technical**, **Project**, and **Business** risk.

Type	Cause / Example	Typical mitigation
Technical	Development team's insufficient knowledge; wrong functions/UI built; shortcomings in external/third-party components.	Communicate with clients, build prototypes for feedback; benchmark and regularly inspect external components.
Project	Problems in budget, schedule, personnel, resources, or the customer relationship, e.g. schedule slippage, insufficient domain/technical knowledge, personnel shortfall (key people leaving).	Detailed milestones, constant iteration, frequent client communication; hire experienced developers (in-house or outsourced); cross-training multiple people on required skills.
Business	Harms the business viability of the product, e.g. the product turns out not to be competitive in the market; gold plating (developers add unnecessary features nobody asked for).	Explore the market for similar/competing products; communicate with clients and perform cost-benefit analysis (for gold plating specifically).

WORKED EXAMPLE

Scenario (adapted from May 2024 Q7). “Gold plating” is described as a business risk.

Meaning. Gold plating means the development team builds **unnecessary features** that were never actually requested, wasted engineering effort that provides no corresponding business value (and can even delay the truly needed features).

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q14). Several developers with critical expertise leave the organisation partway through a project, causing delays.

Answer. This is a **Project risk** (personnel shortfall); the best mitigation listed is **cross-training** other team members on the required skills, so the project is not fatally dependent on any one person.

✗ COMMON MISTAKE

Do not confuse a project risk's *mitigation* with a technical risk's. “Benchmarking / regular inspection of external components” is the fix for *technical* risk (bad third-party component). “Cross-training / hiring experienced developers” is the fix for *project* risk (personnel problems). “Communicating with clients, building prototypes” is the fix for the *technical* risk of misunderstood functionality, while “exploring the market” or “cost-benefit analysis” fixes *business* risk. Match cause to category carefully, several options in PYQs are plausible-sounding but mismatched.

4.5.1 Risk Assessment: Building a Risk Table

DEFINITION

The project manager builds a **risk table**: for every identified risk, assign a **probability** P (chance it occurs) and an **impact** I on a 1–4 scale (negligible, marginal, critical, catastrophic). Compute

$$\text{Risk} = P \times I$$

then sort the table in **descending order** of risk value, the highest-risk items deserve the most attention.

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q246). Probability of schedule slippage is 30%, impact is rated 3.

Answer. Risk = $0.30 \times 3 = 0.9 = 0.9\%$ (note: PYQ answer options treat $P \times I$ as a direct percentage figure, so $0.30 \times 3 = 0.9$ maps to the “0.9%” option, read the answer options’ units carefully, some papers scale differently).

WORKED EXAMPLE

Scenario (adapted from May 2023 Q9). A project management team identifies a risk of personnel shortfall with probability $P = 0.7$ and impact $I = 2$.

Answer. Risk = $0.7 \times 2 = 1.4$.

Risk	P	I	$P \times I$
Schedule slippage	60%	3	1.8
Lack of e-commerce experience	60%	3	1.8
Personnel shortfall	40%	3	1.2
Database can’t scale for many users	20%	2	0.4

EXAM TIP

When asked “which risk should be prioritised first?”, simply pick the row with the **highest** $P \times I$ after sorting, no other reasoning is required, the risk table is built specifically so that prioritisation is a direct read-off.

QUIZ-ORIENTED NOTES

- Three risk categories: **Technical** (about the tech itself), **Project** (budget/schedule/people/resources/customer), **Business** (market viability, gold plating).
- Risk = $P \times I$; sort descending; highest value = highest priority.
- Gold plating = unnecessary features = business risk.

4.6 Project Management in Agile

> MOTIVATION

Everything in Sections 4.2–4.5 assumes a **Plan-and-Document** mindset: predict cost and schedule fairly precisely, up front, before much work begins. The Agile perspective, consistent with Chapter 1’s Agile Manifesto, deliberately **does not** try to predict exact cost and schedule at the very start of the project. Instead, planning happens continuously, a little at a time, one short iteration after another.

4.6.1 Iterations and Team Structure

Agile project management divides work into **iterations of 1–2 weeks**. In each iteration, a set of prioritised **user stories** (Chapter 2, Section 2.6) is implemented; stories are then re-prioritised for the *next* iteration based on what was learned. Teams are typically kept small, **4–9 people**, organised, in the Scrum framework, around fixed-length **Sprints**.

4.6.2 The Scrum Team

Role	Responsibility
Development Team	Whoever is required to complete the work committed to in a given sprint.
Product Owner	Interfaces between the client and the development team; ensures the team delivers maximum client value (recall Chapter 1).
Scrum Master	Ensures all Scrum activities/ceremonies are being carried out well; removes blockers.

4.6.3 Sprint Planning and the Product Backlog

Sprint Planning is a collaborative event involving the Product Owner, Scrum Master, and Development Team, answering two basic questions: *what* work can get done in this sprint, and *how* will the chosen work get done? A sprint planning meeting typically runs about **2 hours per week** of the iteration (e.g. a 2-week sprint gets roughly a 4-hour planning meeting).

DEFINITION

The **Product Backlog** is a prioritised list of work for the development team, derived from user stories and requirements. Items are prioritised during the sprint planning meeting.

✗ COMMON MISTAKE

The Product Backlog is **not** fixed at the start of the project and frozen thereafter, it is **continuously, collaboratively refined** across sprints as priorities shift and new stories emerge. A PYQ option claiming the backlog “is fixed at the start of the project and cannot be changed” is always false.

Example (Seller Portal product backlog): 1. Catalogue Management (add/edit/delete item); 2. Inventory Management (add/edit/delete item); 3. Order tracking; 4. Payment tracking; 5. Sales tracking; 6. Customer feedback.

4.6.4 The Recurring Ceremonies

Ceremony	Purpose
Standup / Daily Scrum	Daily meeting (dev team, Scrum Master, product manager); each member answers: what did I do yesterday, what will I do today, what's blocking me?
Sprint Review	Team demonstrates what was completed; work items move from To-Do/In-Progress to Done.
Sprint Retrospective	Evaluate the last sprint itself, team dynamics, process, tools, note what went well/didn't, and create/implement an improvement plan.

(These match the four ceremonies already introduced in Chapter 1, Section 1.7.4, this section is the project-management-focused view of the same activities.)

4.6.5 Scheduling in Agile: Points and Velocity

Rather than a Gantt chart fixed at project start, Agile progress is tracked simply by **counting user stories completed per iteration**. But a naive story *count* is misleading, not all stories require equal effort.

DEFINITION

Each user story is rated with **story points** on a relative complexity scale (e.g. 1 = straightforward, 2 = medium, 3 = very complex). **Velocity** is the number of story points a team completes per sprint, used to *predict* how much work the team can realistically take on in future sprints (as introduced already in Chapter 1, Section 1.7.4).

✗ COMMON MISTAKE

Counting raw *number of stories completed* as the progress metric is a known pitfall (this is exactly what a PYQ-style reflection question probes): two sprints that each complete “3 stories” can represent wildly different amounts of actual work if the stories differ in complexity. Points/velocity fixes this by weighting stories by their relative size.

4.6.6 Plan-and-Document vs. Agile Scheduling, Side by Side

Plan-and-Document	Agile
Scheduling happens early , up front	Scheduling is continuous , iteration by iteration
Break the whole project into tasks (WBS)	Break work into user stories
Represent the plan with Gantt charts, milestones	Track progress with Points and Velocity

◆ MEMORY TRICK

COCOMO = **CO**nstructive **CO**st **MO**del, effort = $a \cdot \text{KLOC}^b$, bigger project category, bigger a and b . **T-P-B** for risk: **T**echnical (the tech itself), **P**roject (budget/schedule/people), **B**usiness (market/competitiveness). Critical path = the *longest*, not the shortest, chain of dependent tasks.

END OF CHAPTER 4: QUICK REVISION

■ REMEMBER THIS

One-page formula sheet.

- **PM responsibilities:** point of contact, team formation, scheduling, estimation, risk management, configuration management.
- **Estimation:** Size (KLOC), Effort (person-months). **Empirical:** Expert Judgement, Delphi. **Heuristic:** COCOMO, Effort = $a \times \text{KLOC}^b$ (Organic 2.4/1.05, Semi-detached 3.0/1.12, Embedded 3.6/1.20); Final = Initial $\times$ EAF.
- **Scheduling:** WBS (tree, root=project, leaf= $\sim$ 2-week task) $\rightarrow$ Activity Network (durations + dependencies) $\rightarrow$ Critical Path (longest chain = min. project duration; off-path tasks have slack) $\rightarrow$ Gantt chart (bar-chart view).
- **Risk:** Technical / Project / Business, each with its own typical cause and mitigation. Risk = $P \times I$, sort descending.
- **Agile PM:** 1–2 week iterations; Scrum roles (Dev Team, Product Owner, Scrum Master); ceremonies (Planning, Daily Scrum, Review, Retrospective); Points & Velocity instead of Gantt charts.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. EAF **multiplies** the initial COCOMO estimate, never adds.
2. Larger/harder project category (Organic → Semi-detached → Embedded) ⇒ larger a and b .
3. Critical path = **longest** path (common trap: students instinctively look for the shortest).
4. Match risk cause to category carefully: bad external component → Technical; schedule/personnel/budget → Project; market/competitiveness/gold plating → Business.
5. Delphi Technique's core goal: reach a **consensus** across multiple experts, anonymously, over several rounds.
6. Story-count alone is a poor Agile progress metric; **velocity** (points/sprint) fixes this.

PRACTICE QUESTIONS

- Q1.** A project is Organic, 10 KLOC. Compute the initial COCOMO effort estimate.
- Q2.** If the project in Q1 has an EAF of 0.9 (favourable cost drivers), what is the final estimate? Explain in one sentence why the final estimate is lower than the initial one.
- Q3.** Draw a small WBS (root + 2 levels) for a food-delivery app of your choice.
- Q4.** Two tasks, A (12 days) and B (20 days), can start immediately; Task C (8 days) needs both A and B finished. What is the critical path length, and how much slack does A have?
- Q5.** Classify each as Technical, Project, or Business risk, with one-line justification: (a) a key back-end developer resigns; (b) a payment gateway vendor's API frequently returns wrong data; (c) a competitor launches an identical product first.
- Q6.** Build a small risk table (2 risks, with P and I) and rank them.
- Q7.** Explain why Agile project management avoids predicting exact cost/schedule at project start, connecting your answer to the Agile Manifesto from Chapter 1.
- Q8.** A team completes 4 stories worth 2,1,1,1 points respectively in Sprint 1. What is their velocity, and how would you use it to plan Sprint 2?

CHAPTER 5

Software Design

In this chapter (Week 5): Outcomes of design & characteristics of good design · Coupling and Cohesion · Object-Oriented Design basics · Class relationships · UML diagrams.

5.1 Outcomes of the Design Process

► MOTIVATION

Chapter 1 defined *design* as giving the system a big-picture structure. This chapter asks a more concrete question: when a design phase finishes, what exactly should exist on paper? And how do you tell a *good* design from a bad one?

DEFINITION

The design process should produce four concrete outcomes: the system's **components** (modules, collections of functions and data that accomplish a well-defined task), the **interfaces** among components (how they communicate), the **data structures** used inside each component, and the **algorithms** needed to implement each component.

5.1.1 Characterising a Good Software Design

Quality	Meaning
Correctness	Correctly implements all the functionalities required of the system.
Efficiency	Manages resources, time, space, cost, well.
Maintainability	Easy to change later.
Understandable	Can be understood by <i>everyone</i> on the development team, not just its original author.

QUIZ-ORIENTED NOTES

Four outcomes of design: Components, Interfaces, Data structures, Algorithms. Four qualities of good design: Correctness, Efficiency, Maintainability, Understandable.

5.2 Design Modularity: Coupling and Cohesion**★ FREQUENTLY ASKED IN PYQs**

Identifying coupling and cohesion *types* from a scenario is one of the most heavily and consistently tested topics across every quiz paper studied. ★★★

► MOTIVATION

Given a set of modules, how do you judge whether the decomposition itself is any good? Two modules could talk to each other constantly and share everything, or barely interact at all. A single module's functions could all serve one clear purpose, or be a random grab-bag. These two independent qualities, **coupling** (between modules) and **cohesion** (within a module), are the standard yardsticks.

REAL-LIFE ANALOGY

Think of a well-run restaurant kitchen. The grill station and the dessert station (two “modules”) should be able to work almost independently, low coupling, needing only a simple ticket system to coordinate. But within the grill station, every task (searing, plating, seasoning) should serve the single purpose of “cook this dish”, high cohesion. A kitchen where every station must constantly shout across the room for shared ingredients (high coupling) is chaotic; a station where one person is randomly also doing dishwashing and answering phones (low cohesion) is poorly organised.

DEFINITION

Coupling measures the degree of interaction *between* two modules. **Cohesion** measures how well the functions *within* a single module cooperate to perform a single objective. Good **modular design** means **high cohesion and low coupling**, each module is internally focused, and modules interact with each other only minimally.

✎ EXAM TIP

This is the single most repeated fact in this whole topic, memorise it as one unit: **good design = HIGH cohesion + LOW coupling**. Any PYQ option claiming “high coupling” or “low cohesion” is good design should be rejected immediately.

5.2.1 Four Types of Coupling

Type	Description
Data Coupling	Modules communicate using primitive data types (e.g. int, float) passed as parameters. <i>E.g.</i> <code>view_inventory(seller_id)</code> . This is the loosest , most desirable form.
Control Coupling	One module passes data that controls the <i>internal logic</i> of another, e.g. flags/switches. <i>E.g.</i> <code>perform_actions(OPERATION_FLAG)</code> where 0=View, 1=Add, 2=Delete.
Common Coupling	Two modules share global data , both read/write the same global variable.
Content Coupling	One module directly reaches into and modifies the internals of another module (rare in modern OO languages like Python/Java, but the tightest , worst form).

WORKED EXAMPLE

Scenario (adapted from May 2025 Q109). SellerModule calls a function in InventoryModule using a variable `operationFlag` to decide whether to view, add, or delete a product.

Answer. **Control Coupling**, a flag that changes the callee's internal behaviour is the defining signature.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 Q13). Module M1 directly modifies variables inside module M2 by referencing its internal implementation.

Answer. **Content Coupling**, reaching directly into another module's internals.

5.2.2 Five Types of Cohesion

Type	Description
Functional	All functions cooperate to complete a single task (the best/strongest form). <i>E.g.</i> an Inventory module with <code>view_inventory()</code> , <code>add_item()</code> , <code>delete_item()</code> , all serving “manage inventory.”
Sequential	Functions execute in sequence, the output of one is the input to the next . <i>E.g.</i> <code>create_order()</code> → <code>check_availability()</code> → <code>place_order()</code> .
Communicational	All functions operate on, or refer to, the same data structure , without necessarily feeding into each other sequentially. <i>E.g.</i> a single “Product” module where catalogue <i>and</i> inventory add/edit/delete all act on the same product record.
Procedural	Functions are related only by sequence of execution , but work toward <i>entirely different purposes</i> . <i>E.g.</i> <code>login()</code> → <code>add_product()</code> → <code>place_order()</code> → <code>logout()</code> , executed in order, but each doing something unrelated.
Coincidental	Functions bundled together with an essentially meaningless relationship (the worst/weakest form).

✗ COMMON MISTAKE

Sequential vs. Procedural is the single most common confusion. Both involve an ordered sequence of functions. The distinguishing test: in **Sequential** cohesion, one function’s *output feeds directly into the next as input and* they all serve one coherent purpose. In **Procedural** cohesion, functions merely happen to run one after another (sharing a sequence of *execution*, not a flow of *data*), and they serve **entirely different purposes**. If the functions’ outputs don’t feed into each other and their goals are unrelated, it’s Procedural, not Sequential.

WORKED EXAMPLE

Scenario (adapted from Jan 2024 Q33). A sound-enhancement module applies several functions in sequence, where the output of one function is used as input to the next, all working toward the single goal of improving sound quality.

Answer. Sequential Cohesion, output-to-input chaining *and* a single shared purpose.

WORKED EXAMPLE

Scenario (adapted from Sep 2024 Q9). A Deque module has `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, each a different operation, but all reading and updating the *same* underlying doubly linked list.

Answer. Communicational Cohesion: the functions don’t feed into each other in sequence, and don’t reduce to one single task (adding at the front is not part of removing from the back), what unifies them is that they all operate on the *same data structure*, exactly the pattern the course illustrates with one “Product” module handling add/edit/delete for both catalogue and inventory.

5.2.3 Connecting Cohesion/Coupling to the “Good Design” Question

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q249 / Jan 2025 Q48). Modules each focus on a single task, but all access a shared database to update common records.

Reasoning. “Single task per module” signals **high cohesion**. “Heavy shared data access across modules” signals **high coupling**. So the correct read is: **high cohesion, high coupling**, a mixed case, high cohesion is good, but the high coupling via shared data is *not* ideal. Do not assume high cohesion automatically implies low coupling, they are independent axes and must be judged separately from the scenario text.

WORKED EXAMPLE

Scenario (adapted from May 2023 Q18). An online bookshop has a Customer module (registration, profile updates, all customer-related tasks only) and a Product module (add/edit/remove products, all product-related tasks only); the two modules need **minimal interaction** and share only a very small amount of data.

Reasoning. Each module performing only its *own*, specific set of tasks signals **high cohesion**. Minimal interaction and little shared data signals **low coupling**. This is the **ideal** case: **high cohesion, low coupling**, the textbook definition of good modular design (contrast with the mixed example above).

◆ MEMORY TRICK

Coupling types from loosest (best) to tightest (worst): **Data** → **Control** → **Common** → **Content** (mnemonic: “**Don’t Call Common Code**”). Cohesion types from strongest (best) to weakest (worst): **Functional** → **Sequential** → **Communicational** → **Procedural** → **Coincidental**.

5.3 Object-Oriented Design: Basic Concepts

► MOTIVATION

Modularity says *what* a good decomposition looks like (high cohesion, low coupling). It does not say *how* to actually represent a module in code. This course represents modules using the **object-oriented** approach.

DEFINITION

An **Object** is the key building block of an object-oriented system, software is understood as a collection of interacting objects, each usually representing a tangible real-world entity (e.g. a Product, a Shopping Cart, an Order, a Payment). An object contains **data** and **methods**. **Encapsulation** bundles data + methods together, ensuring **data hiding** (also called **data abstraction**): an object’s internal data can only be accessed or changed through its own methods, not directly from outside.

WORKED EXAMPLE

A Product object might have private data members name, type, size, colour, brand, cost, etc., accessible only via methods like `add_product()`, `edit_product_details()`, `delete_product()`.

DEFINITION

A **Class** is a **template** for creating objects. All objects sharing similar attributes and methods belong to the same class (e.g. electronics, clothes, and food items can all be objects of a general Product class).

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q18). A class has private data, accessible only through defined methods.

Answer. Encapsulation, the defining concept behind data hiding.

5.4 Class Relationships

DEFINITION

Classes relate to each other in four standard ways:

Relationship	Meaning	How to identify
Association	Two classes take each other's help to perform a function (<i>Seller</i> calls <code>view_inventory()</code> on <i>Inventory</i>).	"A delegates to B" / "A needs help from B"
Composition	A whole/part relationship (<i>Catalogue</i> has a list of <i>Products</i>).	"B is a part of A" / "A contains B" / "A is a collection of Bs"
Inheritance	One class extends the features of another (<i>Product</i> → <i>Electronics</i> , <i>Books</i> , <i>Clothing</i>).	"A is a kind of B" / "A is a specialisation of B"
Dependency	Class B depends on Class A, so changes to A force changes to B (e.g. B implements interface A).	"B breaks if A changes"

EXAM TIP

The fast test for PYQ class-diagram questions: can you rephrase the relationship as "*is a*"? → Inheritance. As "*has a*" / "*is made up of*"? → Composition. As "*uses/calls*"? → Association. As "*would break if the other changed*" without a whole/part or is-a link? → Dependency.

WORKED EXAMPLE

Scenario (adapted from May 2023 Q24). “A car is a vehicle, and a car has an engine.” Model Car, Vehicle, and Engine as classes.

Answer. Car **inherits from** Vehicle (“a car *is a* vehicle”), while Engine is in a **composition** relationship with Car (“a car *has an* engine”, the engine is a part of the car). Notice both relationships involve the *same* class, Car, but in two completely different roles, one as a specialisation (inheritance, looking “upward” to Vehicle), one as a whole containing a part (composition, looking “downward” to Engine).

✗ COMMON MISTAKE

A PYQ answer option may offer “**Aggregation**” as a relationship type (e.g. “Seller *aggregates* Inventory”). This course teaches **Composition** as the whole/part relationship, treat “Aggregation” in an answer option as a plausible-sounding *distractor* for Composition, not as a fifth relationship type taught in this syllabus.

QUIZ-ORIENTED NOTES

Object → instance of a Class. Encapsulation → data+methods bundled, enabling data hiding/abstraction. Four class relationships: Association (uses), Composition (has-a/part-of), Inheritance (is-a), Dependency (breaks-if-changed).

5.5 UML Diagrams

★ FREQUENTLY ASKED IN PYQs

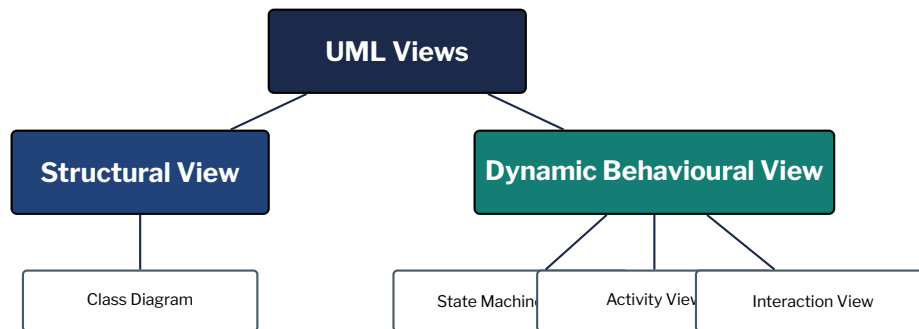
Identifying which UML view/diagram models message exchange over time (the *Interaction View* / Sequence Diagram) is extremely frequently tested, almost word-for-word identical across multiple papers. ★★ ★

➤ MOTIVATION

A design exists in an engineer’s head, or scattered across a codebase, until it is communicated. **Modelling** creates an external, explicit representation of the system so a design can be discussed, checked, and shared *before* (and during) coding.

DEFINITION

UML (Unified Modelling Language) represents a software design through multiple **views**, at a greater level of detail than plain prose. The course groups UML views into two families: the **Structural View** and the **Dynamic Behavioural View**.



View / Diagram	What it captures
Structural View → Class Diagram	The system's classes, attributes, operations, and relationships among objects, its <i>static</i> structure.
Behavioural: State Machine View	The different states an object of a class can be in, and transitions between them.
Behavioural: Activity View	The flow of control among computational activities (like a flowchart of the process).
Behavioural: Interaction View (Sequence Diagram)	The sequence of message exchanges between different parts of a system, over time.

EXAM TIP

This exact phrase, “*sequence of messages exchanged between different parts of the system*”, appears almost verbatim across many PYQ papers (Jan 2023 Q241, Jan 2024 Q32, Jan 2025 Q49, Jan 2026 Q17). The answer is always the same concept under different names: **Interaction View**, **Sequence Diagram**, or **Interaction Diagram**, all refer to the same idea. Distractor options like “Activity View” or “State Machine View” describe *different* kinds of dynamic behaviour, control flow and object state, respectively, not message sequencing.

5.5.1 Purposes of UML Modelling

- A **vehicle for communication** and idea generation among the team.
- **Guides development** of the software (a plan to code against).
- Enables **close correspondence with implementation**, in some tools, every class in a model maps onto one or more classes in the actual code, and code can even be partly **generated** from the model.

QUIZ-ORIENTED NOTES

- Structural View → Class Diagram (static structure). Behavioural View → State Machine (states), Activity (control flow), Interaction/Sequence (message order over time).
- Three purposes of modelling: communication, guiding development, code generation/correspondence.

END OF CHAPTER 5: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- **Design outcomes:** Components, Interfaces, Data structures, Algorithms. **Good design:** Correct, Efficient, Maintainable, Understandable.
- **Good modularity = High Cohesion + Low Coupling.**
- **Coupling** (loose→tight): Data → Control → Common → Content.
- **Cohesion** (strong→weak): Functional → Sequential → Communicational → Procedural → Coincidental.
- **OOD basics:** Object (instance), Class (template), Encapsulation (data+methods, hides internals).
- **Class relationships:** Association (uses), Composition (has-a), Inheritance (is-a), Dependency (breaks-if-changed).
- **UML:** Structural→Class Diagram; Behavioural→State Machine / Activity / Interaction (Sequence).

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. “Message sequence between parts of a system over time” ⇒ Interaction View / Sequence Diagram, always.
2. Sequential cohesion needs *both* output-to-input chaining *and* one shared purpose; Procedural cohesion has ordering only, with unrelated purposes.
3. High cohesion and high/low coupling are independent, judge each from the scenario separately, don’t assume one implies the other.
4. Flag/switch parameter controlling another module’s logic ⇒ Control coupling. Reaching into another module’s internals ⇒ Content coupling (tightest, worst).
5. “Is a” ⇒ Inheritance. “Has a / part of” ⇒ Composition. “Uses/calls” ⇒ Association.

PRACTICE QUESTIONS

- Q1.** A `Logger` module's functions all write to, and read from, the same shared log file, without one feeding the next. Which cohesion type is this?
- Q2.** Two modules pass only simple integers as parameters, with no shared globals. What coupling type is this, and why is it considered good design?
- Q3.** Explain, with your own example, why "high cohesion" does not automatically guarantee "low coupling."
- Q4.** A `Car` class has an `Engine` object as one of its data members, the engine cannot exist meaningfully without the car. Which class relationship is this?
- Q5.** Sketch (in words) a Sequence Diagram for a user logging into the Seller Portal: which parts of the system exchange messages, and in what order?
- Q6.** Distinguish the Structural View from the Dynamic Behavioural View in one sentence each.

CHAPTER 6

Software Development

In this chapter (Week 7): Development environments, IDEs & frameworks · Version control · Debugging (definitions, steps, techniques, strategies) · Software metrics (Cyclomatic Complexity, Raw Metrics, Halstead's) · Writing clean code, code smells, refactoring.

6.1 Development Environments, IDEs, and Frameworks

> MOTIVATION

Once a design exists, someone has to actually sit down and write, run, and share code. This section covers the everyday tools that make that possible.

DEFINITION

A **Development Environment** combines a way to **write** code (a text editor, e.g. Notepad, Vim, Emacs) with a way to **run** it (a compiler or interpreter). An **Integrated Development Environment (IDE)**, e.g. VS Code, PyCharm, Thonny, bundles *additional* features that make development more effective: syntax checking, finding every place a function is used, and safely renaming a variable/class across an entire codebase.

DEFINITION

A **Framework** bundles common features together, built and maintained by a development community, giving projects a ready-made structure so developers can build and maintain applications faster. A **Web Application Framework** (e.g. **Flask**) helps developers quickly start building a web app, including reusable page **templates**. Flask is specifically a **micro-framework**: it deliberately keeps its core small and **does not impose restrictions** on which databases or web servers a project must use, unlike some larger, more opinionated frameworks.

QUIZ-ORIENTED NOTES

Text editor + compiler/interpreter = development environment. IDE = development environment + productivity features (syntax check, find-usages, safe rename). Framework = a reusable, community-maintained structure (e.g. Flask for web apps).

6.2 Version Control

DEFINITION

A **Version Control System (VCS)**, e.g. **Git**, tracks and manages changes to code over time, allowing multiple developers to share and maintain different versions of the same codebase, see a full history of changes, work on isolated **branches**, and revert to a previous stable version if something breaks.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q19). A developer commits faulty code to the main branch, crashing the system for everyone else. The team wants to quickly revert to a stable version and identify who made the change.

Answer. **Version history tracking**, the VCS feature that lets a team see every past change (and who made it) and roll back to any earlier, known-good state.

Basic Git Workflow, In Practice

Joining an existing project and contributing a change typically follows this order: `clone` (copy the remote repository locally) → `modify` (edit files) → `add` (stage the specific files you want to commit) → `commit` (save a snapshot with a message) → `push` (upload your commits to the remote repository).

WORKED EXAMPLE

Scenario (End Term style). A developer modifies three files, `file1`, `file2`, and `file3`. Only `file1` and `file2` are ready to be committed, `file3` was just for local testing.

Answer. `git add file1 file2` followed by `git commit -m "..."`, naming the specific files to add stages *only* those files, leaving `file3` untouched. (`git add .` would stage *every* changed file, including the unwanted `file3`.)

6.3 Debugging

★ FREQUENTLY ASKED IN PYQs

Debugging *strategies* (Backwards / Forwards / Blackbox / Input manipulation) are one of the most frequently and consistently tested topics in this chapter. ★★ ★

► MOTIVATION

Code that compiles and runs is not the same as code that behaves correctly. When something goes wrong, a developer needs a disciplined process, and a vocabulary, for tracking the problem down to its root cause.

6.3.1 Error, Failure, and Fault

DEFINITION

- **Error:** a discrepancy between the actual behaviour and the intended behaviour.
- **Failure:** the *observable* symptom of an error, an incorrect output value, an exception, a crash.
- **Fault:** the actual location *in the code* (the specific line(s)) where the failure originates.
- **Debugging:** the process of determining the cause of a failure.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q20). A banking app produces the wrong interest amount because a specific line of code uses the wrong formula.

Answer. That specific incorrect line of code is the **Fault**, the wrong number the customer sees is the **Failure**, and the underlying discrepancy between intended and actual behaviour is the **Error**.

🔑 EXAM TIP

Keep the direction straight: **Fault** (in the code) *causes* an **Error** (in program state/behaviour), which *produces* a **Failure** (an observable symptom). PYQs like to test whether you can point to the right one of the three from a single scenario.

6.3.2 Steps in Debugging

1. **Reproduce** the problem.
2. **Find the cause** of the defect.
3. **Investigate** a fix.
4. **Implement** the fix.
5. **Test** the fix.

6.3.3 Debugging Techniques

Technique	Description
Logging	Insert print statements to observe values/flow at runtime.
Dump & Diff	Use a <code>diff</code> tool to compare logged data between two different executions (e.g. a working run vs. a broken run).
Stepping in a debugger	Execute code one line/statement at a time (e.g. Python's <code>pdb</code> : <code>next</code> , <code>step</code> , <code>p expression</code> , <code>break</code> , <code>continue</code>), inspecting state as you go.
Profiling tool	Measures <i>how often</i> and <i>how long</i> different parts of a program execute, used for performance problems (finding CPU/memory bottlenecks), not correctness bugs.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q21). A video app is functionally correct but slow on high-resolution video; the team wants to know which functions consume the most CPU time and where memory usage is highest.

Answer. Profiling tool, this is a performance investigation (bottlenecks), not a correctness bug, so logging or stepping through a debugger is the wrong tool for the job.

6.3.4 Debugging Strategies

Strategy	Description
Input manipulation	Edit the inputs given to the program and observe how the output changes, to isolate which input triggers the bug.
Backwards	Start from the incorrect output/statement, and trace data & control dependencies backwards to find the faulty line.
Forwards	Start from the event that triggered the incorrect behaviour, and follow control flow forward until the incorrect state is reached.
Blackbox debugging	Consult documentation and code examples to understand the <i>correct</i> use of an API you don't have (or don't want to read) the internal source code for.

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q251). A developer finds the event that triggered incorrect behaviour and follows the data/control flow *ahead* until the incorrect state is reached.

Answer. Forwards strategy, tracing *forward* from a known trigger.

WORKED EXAMPLE

Scenario (adapted from Jan 2025 Q45 / Jan 2024 Q31). An engineer fixing an issue with a third-party payment API (no access to its internal code) reviews the API's documentation and example code.

Answer. Blackbox debugging, using external documentation because the internals aren't available or aren't the point.

✗ COMMON MISTAKE

Backwards vs. Forwards is a common mix-up. Ask: does the scenario *start from the bad output* and work backward to find the cause (**Backwards**), or does it *start from a known trigger/event* and work forward to see where things go wrong (**Forwards**)? The direction of travel in the scenario is the entire test.

◆ MEMORY TRICK

I-B-F-B: **I**nterpretation (change the input, watch the output), **B**ackwards (start at the bad result, walk back), **F**orwards (start at the trigger, walk forward), **B**lackbox (don't look inside at all, read the docs instead).

6.4 Software Metrics

★ FREQUENTLY ASKED IN PYQs

Naming the metric family that uses LOC/LLOC/SLOC (**Raw Metrics**) is repeated verbatim across several papers. ★★☆☆

► MOTIVATION

"Write clean code" is good advice, but vague. Software metrics give a **quantitative**, tool-measurable way to assess code quality, so teams don't have to rely purely on subjective opinion (tools like **Radon** compute these automatically for Python code).

Metric family	What it measures
Cyclomatic Complexity	The number of independent decision points (if, while, for, etc.) a block of code contains, plus 1 . Higher values mean more branching paths through the code, and typically, harder-to-test, harder-to-understand code.
Raw Metrics	Simple counts: LOC (total lines of code), LLOC (logical lines, each containing exactly one statement), SLOC (source lines of code, not necessarily equal to LLOC), and Comments (number of comment lines).
Halstead's Metrics	Metrics derived from counting the number of distinct <i>operators</i> and <i>operands</i> in the code, used to estimate program length, vocabulary, volume, and (indirectly) complexity/effort.

WORKED EXAMPLE

A function with one if statement and one while loop has 2 decision points, so its Cyclomatic Complexity = $2 + 1 = 3$.

EXAM TIP

The phrase “*measures code quality using parameters like LOC, LLOC, SLOC*” appears almost verbatim across multiple PYQ papers (Jan 2023 Q242, Sep 2022 Q124, Sep 2023 Q192), the answer is always **Raw Metrics**, not Cyclomatic Complexity (which counts decision points, not lines) and not Halstead’s (which counts operators/operands, not lines).

6.5 Writing Clean Code

> MOTIVATION

Metrics measure some aspects of quality, but plenty of “clean vs. messy” code characteristics aren’t captured by any number, they’re about *human* readability and long-term maintainability.

Why Write Clean Code

- Easier for **teammates** to understand.
- Easier to **find and fix bugs** in.
- Easier to **maintain and extend** later.

6.5.1 Code Smells

DEFINITION

A **Code Smell** is a problematic characteristic in code, not necessarily a bug, but a warning sign of poor design that makes future work harder. (Robert Martin’s book *Clean Code* catalogues 63 of them.)

Category	Examples
Comments	Redundant comments that just restate the code (<code>i++</code> //increment i by 1); commented-out dead code left lying around.
Functions	A function doing more than one thing (should do <i>one</i> thing only); functions that are too long; functions with too many arguments; functions with flag arguments (a boolean that changes what the function does); dead (unused) functions.
General	Violating DRY (“Don’t Repeat Yourself”, identical code duplicated in many places, instead extract it into one shared method); incorrect behaviour at boundary conditions; failing to use explanatory variables (e.g. naming a list <code>thirty_day_months</code> instead of the meaningless <code>m</code>).

6.5.2 Refactoring

DEFINITION

Refactoring changes the *internal structure* of code to improve it (readability, design, elimination of smells) **without changing its external behaviour**. Common refactoring tools for Python include Pylint, flake8, and vulture.

QUIZ-ORIENTED NOTES

- Error/Failure/Fault: cause → symptom-in-code, code-you-see-that's-wrong, in that causal order: Fault (code) → Error (state) → Failure (observable symptom).
- Steps: Reproduce → Find cause → Investigate fix → Implement fix → Test fix.
- Techniques: Logging, Dump&Diff, Stepping (debugger), Profiling (performance only).
- Strategies: Input manipulation, Backwards (from bad output), Forwards (from trigger), Blackbox (docs, no source access).
- Raw Metrics = LOC/LLOC/SLOC/Comments. Cyclomatic Complexity = decisions + 1. Halstead's = operators/operands.
- Refactoring changes structure, *not* behaviour.

END OF CHAPTER 6: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- Dev environment = editor + compiler/interpreter. IDE = dev environment + productivity tools. Framework = reusable structure (e.g. Flask).
- VCS (Git): tracks history, branches, enables reverting to a stable version.
- Fault (in code) → Error (wrong internal state) → Failure (observable symptom). Debugging finds the fault that caused a failure.
- Debugging techniques: Logging, Dump&Diff, Stepping/debugger, Profiling (performance).
- Debugging strategies: Input manipulation, Backwards, Forwards, Blackbox.
- Metrics: Cyclomatic Complexity (decisions+1), Raw Metrics (LOC/LLOC/SLOC/Comments), Halstead's (operators/operands).
- Code smells: bad comments, bad functions (too long/too many args/flag args), DRY violations, missing explanatory variables. Refactoring = restructure without changing behaviour.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. “LOC/LLOC/SLOC/comments” $\Rightarrow$ always Raw Metrics.
2. “Which functions/parts are slow, CPU/memory bottlenecks” $\Rightarrow$ Profiling, not logging or stepping.
3. Scenario starts at a known trigger and moves forward $\Rightarrow$ Forwards strategy. Starts at the bad output and moves back $\Rightarrow$ Backwards strategy. No source access, reading docs $\Rightarrow$ Blackbox.
4. A specific wrong line of code = Fault. The wrong output the user sees = Failure.
5. Refactoring never changes external behaviour, only internal structure.

PRACTICE QUESTIONS

- Q1.** A function has 3 if statements and 1 for loop, and no other branching. What is its Cyclomatic Complexity?
- Q2.** Give an example each of Backwards and Forwards debugging using a bug in a shopping-cart total calculation.
- Q3.** Identify the code smell: a 200-line function that validates input, saves to a database, and sends a confirmation email, all in one place. What would you do to fix it?
- Q4.** Explain the difference between a Fault, an Error, and a Failure using a real or invented example.
- Q5.** Rewrite `if x: do_a() else: do_b()` as a flag-argument code smell, then show a cleaner alternative.
- Q6.** Why is DRY considered good practice? What risk does violating it introduce?

CHAPTER 7

Software Architecture

In this chapter (Week 8): Software architecture & architecture styles · Components & connectors · SOLID principles · Design patterns (Creational, Structural, Behavioural).

7.1 Software Architecture and Architecture Styles

★ FREQUENTLY ASKED IN PYQs

Matching a scenario to its architecture style (Client-Server / Pipe-and-Filter / Peer-to-Peer / MVC) is tested in nearly every quiz paper. ★★★

► MOTIVATION

Chapter 5 covered how individual modules should relate to each other (coupling/cohesion). **Architecture** zooms out further: it is the overall way a system's major elements are organised and how they're allowed to talk to each other, decided very early, and expensive to change later.

DEFINITION

Software Architecture is the way a system's software elements (modules), the relations among them, and the properties of both, are organised. An **Architecture Style** is a recurring, named *pattern* of information flow between components.

Style	Description
Client-Server	Data is transacted in response to requests ; a central server responds to many clients.
Pipe and Filter	Data passes from component to component in a chain, transformed/filtered at each stage (like a Unix pipeline).
Peer-to-Peer (P2P)	A distributed system where nodes share resources (processing power, storage) directly with each other; no centralised system monitors all transactions.
Model-View-Controller (MVC)	Separates data (Model) from its presentation (View) and from the coordination logic (Controller) that lets users manipulate the model.

Model-View-Controller, In Detail

- **Model:** models the data required for the service (e.g. classes/objects mirroring the Seller Portal's database tables).
- **View:** the GUI objects, the presentation layer, a visual representation of the Model.
- **Controller:** coordinates multiple Views on screen and lets users manipulate the Model.

WORKED EXAMPLE

Scenario (adapted from Jan 2023 Q252 / Jan 2026 Q22). A data-processing system is made of several filters connected one after another; each filter processes data and passes its output to the next.
Answer. **Pipe and Filter**, sequential transformation stages is the defining trait.

WORKED EXAMPLE

Scenario (adapted from Jan 2024 Q30 / Jan 2025 Q52 / May 2025 Q105). An instant-messaging or file-sharing app exchanges messages/files directly between users' devices, without relying on a central server.
Answer. **Peer-to-Peer**, no centralised coordinator, direct node-to-node exchange.

EXAM TIP

Look for the **coordinator**: a single always-present server responding to requests → Client-Server. A linear chain of transformations → Pipe-and-Filter. No central authority at all, nodes talk directly → Peer-to-Peer. Data/presentation/control explicitly kept separate → MVC.

7.1.1 Components, Connectors, and Design Patterns (Overview)

DEFINITION

A **Component** is a well-defined piece of functionality/behaviour, separate from others (e.g. in Client-Server: the *Client software* and the *Server* are the two components). A **Connector** is the code that transmits information *between* components and regulates their interaction, governed by a **protocol**, a set of predefined rules for how components should interact (e.g. function calls, REST APIs).

DEFINITION

A **Design Pattern** is a description of communicating objects and classes, customised to solve a *general* design problem in a particular context. A **Design Smell** is a warning sign that code is heading toward an **anti-pattern** (a bad, recurring, avoidable solution). **SOLID guidelines** (next section) help avoid design smells; **refactoring** (moving code between classes, creating/removing classes) is the standard remedy once a smell is found.

QUIZ-ORIENTED NOTES

- Four architecture styles: Client-Server (requests), Pipe-and-Filter (chained transforms), Peer-to-Peer (no central node), MVC (Model/View/Controller).
- Component = a unit of functionality; Connector = the communication code between components, following a protocol.
- Design pattern = reusable solution template; Design smell = a warning sign of an anti-pattern.

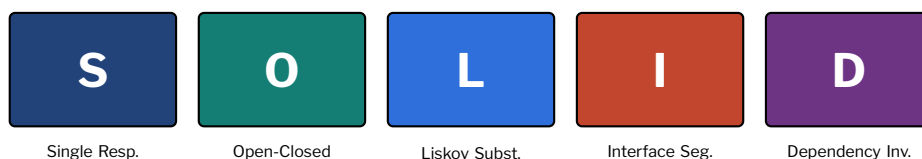
7.2 SOLID Principles

★ FREQUENTLY ASKED IN PYQs

SOLID-principle identification from a scenario is tested very heavily and consistently across almost every paper. ★★ ★

► MOTIVATION

As an object-oriented codebase grows, certain recurring problems show up again and again: a class doing too much, changing one class breaking five others, code that has to be rewritten every time a new case is added. **SOLID** is a set of five object-oriented design principles, conceptualised by **Robert Martin** (“Uncle Bob”), that make code easier to understand and to extend safely.



7.2.1 S: Single Responsibility Principle (SRP)

DEFINITION

Every class should have a **single responsibility** (one reason to change).

E.g. a Product class that also handles payment processing *and* sends emails violates SRP, split it into Product, PaymentProcessor, and NotificationService.

7.2.2 O: Open-Closed Principle (OCP)

DEFINITION

Software entities should be **open for extension, but closed for modification**, add new behaviour by *adding new code* (e.g. a subclass), not by editing existing, already-working code.

E.g. adding a new product type “Clothing” should be done by creating a new subclass, not by editing the existing Product class’s logic.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q8). A DiscountCalculator class applies discount rules for Regular and Premium customers using a single method with a switch/if-elif statement, one case per customer type. To support a new VIP customer type, the developer **edits that same method**, adding another case block.

Answer. Open-Closed Principle violated. Every time a new customer type appears, the already-working DiscountCalculator method must be reopened and modified, exactly the pattern OCP forbids. A fix would replace the switch statement with a small class hierarchy (e.g. a DiscountStrategy base class with one subclass per customer type, see the Strategy pattern, Section 7.3), so a new customer type is added purely by *adding* a new subclass.

7.2.3 L: Liskov Substitution Principle (LSP)

DEFINITION

Applies to inheritance: derived classes should be **substitutable** for their base class, if D is a subtype of B, replacing B with D anywhere in the program should **not break** its behaviour.

E.g. if a NotifySellerByEmail feature is extended to NotifySellerByEmailAndSMS, the new subclass must still honour every guarantee the original notification interface made, callers shouldn’t need to know or care which concrete class they’re using.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q6). A base class Document has a method print(). A derived class EncryptedDocument **throws an exception** whenever print() is called, because the document must be decrypted first.

Answer. Liskov Substitution Principle violated. Code written against the base class Document reasonably assumes that calling print() on *any* Document works; silently substituting an EncryptedDocument instead breaks that assumption and crashes the program. This is the textbook LSP violation pattern: **a subclass that throws where the base class guarantees success.**

7.2.4 I: Interface Segregation Principle (ISP)

DEFINITION

Do **not force** a client to implement/depend on an interface (or its methods) that is **irrelevant** to it, prefer several small, specific interfaces over one large, general one.

E.g. an order-delivery interface with methods `process()`, `notify()`, `package()`, and `deliver()` shouldn't be forced on a purely digital (download-only) order type that has no physical packaging or delivery step.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q24). A system processes both physical books and e-books through one shared interface; the e-book class is forced to implement `package()` and `ship()` methods that do nothing.

Answer. **Interface Segregation Principle** is violated, the e-book class is forced to depend on methods irrelevant to it.

7.2.5 D: Dependency Inversion Principle (DIP)

DEFINITION

Prefer depending on **abstractions/interfaces** rather than on concrete **implementations**, high-level modules shouldn't depend directly on low-level modules' details.

E.g. a "receive payment" feature should depend on a general `PaymentGateway` interface, not directly on concrete classes like `PayTM` or `GooglePay`, so new payment providers (UPI, etc.) can be added without changing the payment-receiving code.

EXAM TIP

Quick discriminators: **SRP** → "this class does too many unrelated things." **OCP** → "I had to edit old, working code to add a new case." **LSP** → "substituting a subclass broke something." **ISP** → "I'm forced to implement a method I don't need." **DIP** → "my code is hard-wired to one specific concrete class instead of an abstraction."

◆ MEMORY TRICK

SOLID: Split responsibilities, stay **O**pen to extension (closed to edits), keep substitutions **L**egit, keep **I**nterfaces lean, **D**epend on abstractions.

7.3 Design Patterns

★ FREQUENTLY ASKED IN PYQs

Matching a scenario to the correct design pattern (especially `Factory` / `Builder` / `Facade` / `Adapter`) is tested very frequently. ★★★

> MOTIVATION

SOLID gives principles; **Design Patterns** give ready-made, battle-tested *templates* that satisfy those principles for common, recurring problems. Each pattern has a **Problem** it solves, a **Solution** shape, and **Consequences** (trade-offs), it is a reusable template, not a finished piece of code, and should never be applied blindly, every pattern has a cost.

Design patterns fall into three families:

Family	Concerned with
Creational	The process of object creation (Factory, Builder).
Structural	The composition of classes/objects into larger structures (Facade, Adapter).
Behavioural	How classes/objects interact and distribute responsibility (Iterator, Observer, Strategy).

7.3.1 Creational: Factory Design Pattern

Problem	Code that directly calls <code>new SomeClass()</code> scattered everywhere becomes hard to change when the exact class to instantiate should vary (e.g. picking a courier service).
Solution	Replace direct construction calls with calls to a special factory method , which decides which concrete class to instantiate.
Example	“I need to deliver a package using a courier service”, a <code>CourierFactory</code> decides at runtime whether to create a <code>FedExCourier</code> , <code>BlueDartCourier</code> , etc.

Pros: supports Single Responsibility & Open-Closed principles (new types added without touching client code). **Con:** can add a lot of new subclasses, increasing complexity.

7.3.2 Creational: Builder Design Pattern

Problem	Creating an object (e.g. a <code>Book</code>) with many mandatory <i>and</i> optional parameters leads to “telescoping constructors”, many overloaded constructors with long, error-prone parameter lists.
Solution	Extract the construction logic out of the class into a separate Builder object, which constructs the object step by step .

Pros: step-by-step construction; supports Single Responsibility. **Con:** more classes/methods to maintain.

7.3.3 Structural: Facade Design Pattern

Problem	A task needs several objects created and several steps performed in the right order (initialise objects, call methods correctly), forcing every caller to know all these details.
Solution	Provide a simple interface in front of a complex set of classes/steps.
Example	After a buyer places an order: create order → notify seller → prepare packaging → send for delivery. A single <code>processOrder()</code> facade method internally does all four steps in order.

Pros: isolates callers from complexity. **Con:** the facade itself can become tightly coupled to many classes, harder to maintain.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q23). An e-commerce system used to require every client to call “Create Order”, “Notify Seller”, “Call Packaging”, “Call Delivery” in the correct sequence. A new class was created with a single method that internally performs all four steps.

Answer. Facade, one simple entry point hiding a multi-step, multi-class process.

7.3.4 Structural: Adapter Design Pattern

Problem	Two classes have incompatible interfaces , e.g. a <code>Product</code> class expects cost in Rupees, but a <code>CurrencyConverter</code> class returns Dollars/Euros in an incompatible format.
Solution	Introduce an Adapter class that translates one interface into the other, without modifying either original class.

Pros: keeps conversion logic separate (SRP); new adapter types can be added without touching existing code (OCP). **Con:** adds overall code complexity.

WORKED EXAMPLE

Scenario (adapted from Jan 2025 Q58). A `PaymentProcessor` only accepts USD, but payment gateways provide amounts in EUR/INR, incompatible formats.

Answer. Adapter, translating between two incompatible interfaces without changing either.

X COMMON MISTAKE

Facade vs. Adapter: both “simplify” something, but for different reasons. **Facade** hides the complexity of *coordinating many steps/classes* behind one simple call. **Adapter** bridges *two otherwise-incompatible interfaces* so they can work together, it’s not about hiding multiple steps, it’s about translating a mismatch.

7.3.5 Behavioural: Iterator Design Pattern

Problem	Different collections (Lists, Queues, Stacks, Trees) store elements differently; a client just wants to <i>access</i> elements without caring how they're stored.
Solution	Separate the “how to access elements” behaviour into its own Iterator object (e.g. Java's Iterator interface: <code>hasNext()</code> , <code>next()</code>).

Pros: SRP (access logic separated); OCP (new collection/iterator types can be added). **Con:** overkill for very simple collections.

7.3.6 Behavioural: Observer Design Pattern

Problem	A set of interested parties (e.g. subscribed buyers) need to be notified whenever something changes (e.g. a new product launches).
Solution	A Subject object maintains a list of Observers and automatically notifies them of any changes.

7.3.7 Behavioural: Strategy Design Pattern

Problem	The same feature (e.g. processing orders) can be done using different algorithms (FIFO, priority-based, etc.), and the choice may need to change at runtime.
Solution	Extract each algorithm into its own Strategy class; the original (context) class delegates the work to whichever strategy object is currently plugged in.

Pros (Iterator/Observer/Strategy, in general): isolate a specific varying behaviour, support Open-Closed (new strategies/iterators/observers added without editing existing classes). **Con:** unnecessary overhead if there is only ever going to be one simple case.

QUIZ-ORIENTED NOTES

- Creational: **Factory** (which class to instantiate), **Builder** (step-by-step construction, avoids telescoping constructors).
- Structural: **Facade** (one simple entry point over many steps/classes), **Adapter** (bridges two incompatible interfaces).
- Behavioural: **Iterator** (uniform access regardless of storage), **Observer** (auto-notify subscribers on change), **Strategy** (swap interchangeable algorithms at runtime).
- Every pattern trades some complexity for flexibility, *never* apply one blindly.

END OF CHAPTER 7: QUICK REVISION

■ REMEMBER THIS

One-page formula sheet.

- **Architecture styles:** Client-Server (requests), Pipe-and-Filter (chained transforms), Peer-to-Peer (no centre), MVC (Model/View/Controller).
- **Component** = unit of functionality; **Connector** = communication code + protocol between components.
- **SOLID:** **S**ingle Responsibility (one job per class), **O**pen-Closed (extend, don't edit), **L**iskov Substitution (subclasses must be drop-in replacements), **I**nterface Segregation (small, specific interfaces), **D**ependency Inversion (depend on abstractions, not concretions).
- **Creational:** Factory (which class?), Builder (step-by-step build).
- **Structural:** Facade (simplify many steps into one), Adapter (bridge incompatible interfaces).
- **Behavioural:** Iterator (uniform traversal), Observer (auto-notify), Strategy (swap algorithms).

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. No central coordinator, direct node-to-node $\Rightarrow$ Peer-to-Peer. Chain of transformations $\Rightarrow$ Pipe-and-Filter.
2. "Forced to implement a method that does nothing for this case" $\Rightarrow$ Interface Segregation Principle violation.
3. "Had to edit existing, working code to support a new case" $\Rightarrow$ Open-Closed Principle violation.
4. "Two interfaces don't match, need a translator" $\Rightarrow$ Adapter. "Too many steps/classes, want one simple call" $\Rightarrow$ Facade.
5. "Which concrete class to build" varies $\Rightarrow$ Factory. "Complex object with many optional parts" $\Rightarrow$ Builder.
6. "Swap the algorithm at runtime" $\Rightarrow$ Strategy. "Auto-notify a list of subscribers on change" $\Rightarrow$ Observer.

PRACTICE QUESTIONS

- Q1.** A logging class also formats reports and sends emails. Which SOLID principle does this violate, and how would you fix it?
- Q2.** Give an example (not from this book) of Liskov Substitution being violated by a subclass that throws an exception for a method the base class guarantees will work.
- Q3.** You need to support both credit-card and UPI payments, added over time, without editing existing payment code each time. Which SOLID principle guides you, and which design pattern would you likely use?
- Q4.** Explain, in one sentence each, the difference between Facade and Adapter.
- Q5.** Design a Strategy-pattern solution (in words) for sorting a list using either quicksort or mergesort, chosen at runtime.
- Q6.** Classify each as Creational, Structural, or Behavioural: (a) Builder, (b) Observer, (c) Adapter, (d) Factory, (e) Iterator, (f) Facade.

Software Testing

In this chapter (Week 9): Testing motivation & terminology · Designing test cases · Blackbox testing (equivalence classes, boundary values) · Whitebox testing (branch, condition, path coverage) · Unit, Integration, and System testing · Smoke and Performance testing · Test-Driven Development.

8.1 Testing: Motivation and Terminology

► MOTIVATION

Chapter 1 already established *that* testing exists and briefly introduced unit/integration/acceptance/alpha/beta testing (Section 1.4). What it did not answer is the much harder practical question: *how do you actually choose which inputs to test with?* Even simple software errors can cause serious real-world consequences, airport systems and spacecraft have failed because of software bugs that a handful of well-chosen test inputs would have caught. This chapter goes deep into *how* good test cases are designed and organised.

DEFINITION

A **test case** is formally a triplet $[I, S, R]$:

- I – the data **input** to the program.
- S – the **state** of the program at which the data is input.
- R – the **result** expected to be produced.

A **test suite** is the set of all test cases designed to test a given program.

The Basic Testing Loop

Provide inputs → check output → see if the output matches expectations → locate where the error is → identify the cause → fix the error → test again. Summarised, the core testing activities are: **design**

the test suite, **run** the test cases and check results, **locate** the error, **correct** the error.

★ INTUITION

Why not just throw a huge pile of *random* inputs at a program? Testing with a large collection of *randomly selected* test cases does **not** guarantee that all (or even most) errors will be uncovered. Consider a buggy $\min(x, y)$ function where the `else` branch was mistakenly written as `min = x` instead of `min = y`. Random testing might, by sheer luck, repeatedly pick cases where x happens to be the correct answer anyway, and never expose the bug. This is exactly why test cases must be *designed*, not sampled blindly.

QUIZ-ORIENTED NOTES

Test case = $[I, S, R]$ triplet. Test suite = collection of test cases for one program. Random test case selection does not guarantee error coverage, this is the core motivation for *designing* test suites deliberately.

8.2 Designing Test Cases: Blackbox and Whitebox Testing

► MOTIVATION

The domain of possible inputs to any real program is far too large to test exhaustively. The goal of test design is to pick a **minimal** set of test cases where each case is likely to expose a genuinely *different* type of error. There are two broad philosophies for doing this.

DEFINITION

Blackbox testing treats the program as a sealed box: test cases are designed purely by examining input/output values, with **no knowledge of the design or code required**. **Whitebox testing** does the opposite: it **analyses the internal structure** of the program (its code, its paths) to design test cases.

✗ COMMON MISTAKE

A frequently tested distractor swaps the two definitions, claiming blackbox testing “necessitates access to the source code” or that whitebox testing requires “no knowledge of the code.” Remember: **Blackbox** = examine *only* inputs/outputs, no code needed. **Whitebox** = analyse the code’s internal structure using heuristics like coverage.

8.2.1 Blackbox Testing: Equivalence Classes

★ FREQUENTLY ASKED IN PYQs

Identifying equivalence classes and boundary values from a short Python function is one of the most consistently repeated question types across End Term papers (Jan2023, Jan2024, May2023, Sep2024, Sep2025). ★★

DEFINITION

An **equivalence class** is a partition of the domain of input values such that the program is expected to **behave similarly** for every input within that class. Blackbox test design picks **one representative test case from each equivalence class**.

WORKED EXAMPLE

Classic example: `min(x, y)`. The equivalence classes are: $(x > y)$, $(x < y)$, $(x = y)$, three distinct classes, because the program's *behaviour* (which branch executes, or whether a special equal-case is handled) genuinely differs across them.

Classic example: `isPrime(num)`. Two equivalence classes: *prime numbers* and *non-prime numbers*, since the function is expected to return `True` for one class and `False` for the other.

WORKED EXAMPLE

Scenario (adapted from Sep 2024 End Term Q3). A function compares the lengths of two strings: returns 0 if equal, a positive integer if the first is longer, a negative integer if the first is shorter.

Reasoning. There are exactly **three** distinguishable behaviours (zero / positive / negative), so there are **3 equivalence classes**: lengths equal, first string longer, first string shorter.

EXAM TIP

The number of equivalence classes always equals the number of *genuinely different behaviours* the specification describes, count the distinct branches/outcomes in the written specification (not the code) to get this number quickly, even without seeing the exact code.

8.2.2 Blackbox Testing: Boundary Value Analysis

DEFINITION

Boundary Value Analysis examines values *at the boundaries* of equivalence classes, since off-by-one style errors (e.g. `range(n)` vs. `range(n-1)`) are especially likely to hide exactly at these edges.

WORKED EXAMPLE

For `isPrime(num)`, the boundary values to specifically check are **0 and 1** (the smallest edge cases right where “primeness” first becomes possible/impossible), not just any prime or non-prime number picked at random.

REMEMBER THIS

Blackbox testing recipe: (1) From the specification, identify input/output structures. (2) Identify equivalence classes. (3) Pick one representative test case per class. (4) Add boundary value test cases at the edges of each class.

8.2.3 Whitebox Testing: Coverage-Based Testing

★ FREQUENTLY ASKED IN PYQs

Selecting the correct set of test cases for branch coverage or multiple condition coverage, given a short snippet of code, is the single most repeated numeric/applied question type in the End Term papers, appearing in nearly every paper studied. ★★★

DEFINITION

Whitebox testing analyses a program's structure using **coverage** heuristics:

- **Branch coverage:** every branch in the program must be taken **at least once**.
- **Multiple condition coverage:** for a **composite** conditional expression (one built from several component conditions joined by and/or), *each individual component condition* must take **both** true and false values across the test suite.
- **Path coverage:** the test suite must exercise **every linearly independent path** through the program at least once, found by tracing the program's **control flow graph**.

WORKED EXAMPLE

Branch coverage. For `if (x < y): min = x else: min = y`, two test cases suffice: one with $x < y$, one with $x > y$, this exercises both branches.

WORKED EXAMPLE

Multiple condition coverage. For `if (num < 50 or num > 150): doSomething()` `else: doSomethingElse()`: the test set `{49, 100}` only achieves **branch** coverage (one case makes the whole condition true, one makes it false). To achieve **multiple condition** coverage, each of the two component conditions (`num < 50` and `num > 150`) must independently see both true and false, requiring a set like `{49, 100, 200}`.

✗ COMMON MISTAKE

Multiple condition coverage is **stricter** than branch coverage, satisfying multiple condition coverage always satisfies branch coverage, but not vice versa. When a PYQ gives four candidate test-case sets and asks which achieve multiple condition coverage, check that *every individual component condition* (not just the overall composite expression) takes both true and false values somewhere in the set; a set that only flips the overall outcome is a branch-coverage set, not a multiple-condition-coverage set.

WORKED EXAMPLE

Path coverage. Given a control flow graph with nodes 1 through 7 and linearly independent paths `{1, 2, 3}`, `{1, 2, 4, 7}`, `{1, 2, 4, 5, 6}`, `{1, 2, 4, 5, 4, 7}`, a path-coverage test suite needs at least one test case exercising each of these four paths.

DEFINITION

Cyclomatic Complexity (already introduced as a software metric in Chapter 6, Section 6.4) reappears here with a precise operational meaning: it equals the **number of decision and loop statements, plus 1**, and it tells you exactly how many linearly independent paths (and therefore how many test cases) are needed for full path coverage.

WORKED EXAMPLE

Scenario (adapted from several End Term papers, e.g. Jan2026, May2024, Sep2025). A Python function contains 2 if statements and 1 while loop, with no other branching.

Answer. Cyclomatic Complexity = 3 decisions + 1 = 4, meaning 4 linearly independent paths must be tested for full path coverage.

EXAM TIP

When a PYQ gives a Python snippet and four candidate sets of numeric test inputs, work out branch coverage first (does every if/elif/else get a True *and* a False?), then check whether composite conditions (joined by and/or) have each sub-condition independently flip for multiple condition coverage. Trace through the code by hand with each candidate input to see which line/branch it hits, this is faster and more reliable than trying to memorise a rule.

QUIZ-ORIENTED NOTES

- Blackbox: examine input/output only, no code needed → Equivalence classes, Boundary values.
- Whitebox: analyse code structure → Branch coverage, Multiple condition coverage, Path coverage.
- Multiple condition coverage $\Rightarrow$ branch coverage (implication), but not the reverse.
- Cyclomatic Complexity = (decisions + loops) + 1 = number of linearly independent paths.

MEMORY TRICK

E-B for blackbox: **E**quivalence classes first (representative cases), then **B**oundary values (the edges).
B-C-P for whitebox, in increasing strictness: **B**ranch → multiple **C**ondition → **P**ath.

8.3 Levels of Testing, Revisited

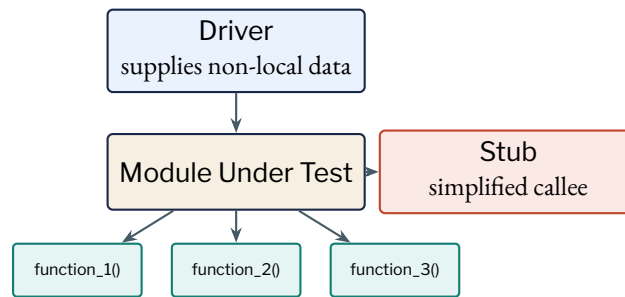
> MOTIVATION

Chapter 1 introduced unit, integration, acceptance, alpha, and beta testing at a summary level. Now that test *design* techniques (Sections 8.2–8.2.3) are covered, this section goes deeper into *when*, *who*, and *how* for each level, exactly the detail most heavily tested in the End Term papers.

8.3.1 Unit Testing, In Detail

DEFINITION

When: during the **coding** of the module, *not* during a separate later testing phase. **Who:** the **person writing the code** for that module. **Why:** (a) while a module is being written, the *other* modules it needs to interface with may not be ready yet, so it must be tested in isolation; (b) it makes debugging far easier, if a failure is only detected once many modules are integrated together, it is much harder to tell *which* module caused it.

**DEFINITION**

Unit testing needs a controlled **testing environment** built from two kinds of scaffolding:

- **Driver module:** contains/supplies the non-local data structures accessed by the module under test (plays the role of “whoever would normally call this module”).
- **Stub:** a dummy procedure that has the **same I/O parameters** as a function called *by* the unit under test, but with highly simplified (fake) behaviour, standing in for a dependency that either doesn’t exist yet or isn’t the focus of this test.

Common Python unit-testing tools: `unittest`, `pytest`.

EXAM TIP

Remember the direction: a **driver** sits *above* the module under test (it drives/calls it, supplying inputs). A **stub** sits *below* it (the module under test calls the stub, which fakes a dependency’s behaviour). This directional distinction is exactly what determines which integration-testing approaches need drivers vs. stubs, covered next.

8.3.2 Integration Testing: Four Approaches

★ FREQUENTLY ASKED IN PYQs

Identifying which integration approach requires drivers, which requires stubs, and which requires neither, is one of the most frequently and precisely tested facts in the whole course. ★★★

DEFINITION

Integration testing happens once at least a few (or all) modules have completed unit testing; its objective is to **detect errors at the module interfaces**, exactly the errors unit testing (which tests modules in isolation) cannot catch.

Consider a small system where module M1 (the root) calls M2, M3, M4, and M4 in turn calls M5.

Approach	Description	Needs Drivers?	Needs Stubs?
Big Bang	All modules integrated in a single step. Meaningful only for small systems; errors are difficult to localise .	No	No
Bottom-Up	Modules of each sub-system are integrated from the lowest level upward (e.g. M5 first, then M2/M3/M4, then M1).	Yes	No
Top-Down	Starts with the root module plus 1–2 subordinate modules; lower levels are tested progressively after the top levels.	No	Yes
Mixed	Combines top-down and bottom-up; integration happens as and when modules become available after unit testing.	Yes	Yes

★ INTUITION

Think about *what's missing* in each approach and what has to fake it. **Bottom-Up** starts with the lowest-level modules, which have no “caller” above them yet, so a **driver** must fake that caller and supply inputs. **Top-Down** starts with the root module, which calls lower-level modules that don't exist yet, so **stubs** fake those not-yet-built callees. **Big Bang** has everything at once, so nothing needs faking, but that's exactly why localising an error afterwards is so hard.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 End Term Q9). A banking application has five modules, each already unit-tested. The team integrates *all five at once* and then tests the full system; when a failure occurs, it is very difficult to find which module caused it.

Answer. Big Bang approach, “all at once” plus “difficult to localise errors” are its two textbook signatures.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q4). An approach allows early testing of the main (root) logic, even while low-level modules are not yet implemented.

Answer. Top-Down, testing the root/main logic first, before lower levels exist (using stubs to fake them), is exactly Top-Down integration.

✗ COMMON MISTAKE

A very reliable PYQ trap: “Which approach(es) do *not* require the use of stubs and drivers?” The only correct answer is **Big Bang** (everything is integrated at once, so nothing needs to be faked). Do not be tempted by “Mixed”, Mixed *does* require both, since it combines Top-Down and Bottom-Up.

8.3.3 System, Acceptance, and Other Testing Types

DEFINITION

System testing tests the **fully integrated system** as a whole. It includes **Alpha testing** (by a test team *within* the organisation) and **Beta testing** (by a *select group of customers*), both already introduced in Chapter 1, Section 1.4.3. **Acceptance testing** is performed by the **customer**, to determine whether to formally accept delivery of the software.

Testing type	Purpose
Smoke Testing	Carried out <i>before</i> initial system testing; a small number of basic test cases check whether basic functionalities work at all (e.g. for the Seller Portal: can you add a product, add it to the catalogue, add inventory?). If smoke testing fails, there is no point running the full system test suite yet.
Performance Testing	Checks whether the system meets its non-functional requirements (recall Chapter 2, Section 2.4), performance, reliability, robustness. Stress testing is a specific kind of performance testing: input volume, input rate, processing time, or memory utilisation are pushed <i>beyond</i> the system's designed capacity (e.g. simulating far more simultaneous database transactions than expected).

WORKED EXAMPLE

Scenario (adapted from May 2025 End Term Q45). A test engineer runs a small set of basic test cases to verify core functionalities work *before* conducting further testing on a newly deployed application.

Answer. Smoke Testing, the “small set of basic checks, done first, before deeper testing” phrasing is its signature.

EXAM TIP

“Determines whether the software meets *non-functional* requirements” always points to **Performance Testing** (a very frequently repeated exact phrase across multiple End Term papers), not Unit, Alpha, Beta, or Smoke testing, those check *functional* correctness or basic readiness, not performance/reliability/robustness.

8.4 Test-Driven Development (TDD)

★ FREQUENTLY ASKED IN PYQs

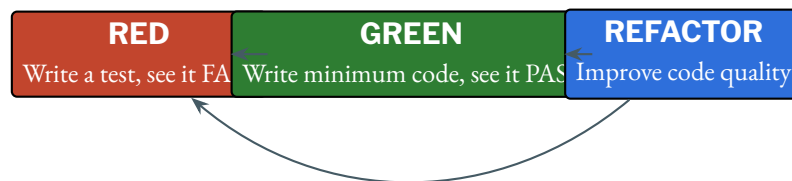
The Red-Green-Refactor cycle and its correct/incorrect characterisations are tested very frequently, usually as a “choose all correct statements” question. ★★★

► MOTIVATION

Every technique so far designs tests *for code that already exists*. Agile processes (Chapter 1, Section 1.7) suggested an even tighter loop: what if the test is written *first*, and the code is written specifically, and minimally, to satisfy it?

DEFINITION

Test-Driven Development (TDD) is used especially in Agile processes. The desired functionality is first expressed **in the form of a test**; that test then **drives** the design and development of the functionality.



- **RED**: write a test for functionality that doesn't exist yet, and watch it fail.
- **GREEN**: write the **minimum** code necessary to make that test pass.
- **REFACTOR**: improve the code's quality, modularity, elegance (recall Chapter 6, Section 6.5), *without* changing its behaviour, now that a passing test protects against regressions.

✗ COMMON MISTAKE

Frequently tested false statements about TDD: “tests are written *after* the code” (false, tests come *first*), “TDD encourages writing production code without any testing” (false, the opposite), and “TDD is not suitable for Agile” (false, it is *especially* used in Agile processes). The true statements are always: tests are written before code, TDD follows Red-Green-Refactor, and TDD is well suited to Agile.

QUIZ-ORIENTED NOTES

- Unit testing: during coding, by the code's own author; needs drivers (fake caller) and/or stubs (fake callee).
- Big Bang: no drivers/stubs, hard to localise errors. Bottom-Up: needs drivers. Top-Down: needs stubs. Mixed: needs both.
- Smoke testing: quick basic checks *before* full system testing. Performance/Stress testing: checks non-functional requirements, beyond designed capacity.
- TDD: Red (failing test) → Green (minimum passing code) → Refactor (clean up).

END OF CHAPTER 8: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- **Test case** = $[I, S, R]$. **Test suite** = set of test cases. Random testing does not guarantee coverage, hence designed test suites.
- **Blackbox** (no code needed): Equivalence classes (representative case per behaviour), Boundary values (edges).
- **Whitebox** (analyses code): Branch coverage $\subset$ Multiple condition coverage; Path coverage via control flow graph; Cyclomatic Complexity = decisions/loops + 1 = # independent paths.
- **Unit testing**: during coding, by the author; Driver = fake caller (needed going Bottom-Up); Stub = fake callee (needed going Top-Down).
- **Integration approaches**: Big Bang (neither, hard to localise) / Bottom-Up (drivers) / Top-Down (stubs) / Mixed (both).
- **System testing**: Alpha (internal team) / Beta (select customers). **Acceptance**: by the customer. **Smoke**: quick basic check first. **Performance/Stress**: non-functional requirements, beyond capacity.
- **TDD**: Red $\rightarrow$ Green $\rightarrow$ Refactor; tests before code; core to Agile.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. Count distinct *behaviours* in the spec to get the number of equivalence classes quickly.
2. A test set that only flips the overall composite condition = branch coverage; a set where *each* sub-condition independently flips = multiple condition coverage (stricter).
3. Cyclomatic Complexity = decisions/loops + 1; equals the number of test cases needed for full path coverage.
4. “Difficult to localise errors, all at once” $\Rightarrow$ Big Bang. “Root/main tested early, stubs used” $\Rightarrow$ Top-Down. “Lowest modules first, drivers used” $\Rightarrow$ Bottom-Up.
5. “Small basic checks before full testing” $\Rightarrow$ Smoke testing. “Meets non-functional requirements / beyond capacity” $\Rightarrow$ Performance/Stress testing.
6. TDD: tests *before* code, Red-Green-Refactor, especially used in Agile, never “no testing.”

PRACTICE QUESTIONS

8.4. Test-Driven Development (TDD)

- Q1.** A function `classifyAge(age)` returns “minor” for ages below 18 and “adult” otherwise. List its equivalence classes and the boundary value(s) you would test.
- Q2.** Write two test cases that achieve branch coverage for `if (a and b): f() else: g()`, and explain why they do *not* achieve multiple condition coverage.
- Q3.** A system has modules A (root) calling B and C, where C calls D. Describe, step by step, how Top-Down integration would proceed, and which scaffolding (driver/stub) is needed at each step.
- Q4.** Why is it meaningful to do Big Bang integration only for small systems?
- Q5.** A function has 2 `if` statements, 1 `elif`, and 1 `for` loop. What is its cyclomatic complexity, and how many test cases are needed for full path coverage?
- Q6.** Explain, in your own words, why smoke testing is run *before* full system testing rather than after.
- Q7.** Rewrite the following (broken) TDD process into the correct order: “write code, refactor, write test, run test.”

CHAPTER 9

Software Deployment and Monitoring

In this chapter (Week 10): Deployment environments & strategies · Deployment hosting options · Continuous Integration · Performance optimisation · Monitoring.

9.1 Deployment Environments

► MOTIVATION

The classic SDLC recap is Requirements → Design → Development → Testing (Chapter 1). This week's focus is what happens *next*: getting a tested system in front of real users (**deployment**), and then watching how it behaves once it's out there (**monitoring**).

DEFINITION

Deployment means “**placing**” the software system so users can access it. **Monitoring** means capturing and analysing information about an application's execution once it is live. An **environment** is simply the system(s) that execute a software application; there are four standard types.



Environment	Description
Development	A developer's local environment; contains an IDE and other tools; a developer runs, tests, and builds code here against their own <i>working copy</i> , before committing to the shared codebase (mainline/master).
Testing	Focused on running integration testing with dedicated testing tools, once code has left the developer's own machine.
Staging	Exactly resembles the production environment , and runs on a remote machine (also called the pre-production environment).
Production	The actual live environment; real users interact with the deployed system here.

★ INTUITION

Why bother with a whole separate Staging environment when Production already exists? Deploying involves many activities and configuration changes; if something goes wrong directly on the live system, it can **crash for real users**. Staging exists to (a) preview new features safely, and (b) run performance testing, in an environment that behaves just like production, but where mistakes are free.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q20 / Jan 2026 Q4). A developer tests a new feature on their own machine with a locally connected database, and has not yet pushed the code to the central repository.

Answer. **Development environment**, local machine, local DB, not yet shared, is the signature.

✗ COMMON MISTAKE

A common PYQ distractor claims Staging “is also known as the live environment where all users can interact with the deployed system”, that description is **Production**, not Staging. Staging *resembles* production but is not the live, user-facing system.

9.2 Deployment Strategies

★ FREQUENTLY ASKED IN PYQs

Identifying the correct deployment strategy from a scenario is, by a wide margin, the single most repeated question type in the End Term papers, appearing in almost every single paper studied. ★★★

► MOTIVATION

Once staging has validated a new version, how should it actually be rolled out to production, all at once, gradually, or with multiple versions running side by side? Each answer is a different **deployment strategy**, with different trade-offs for safety and rollback speed.

Strategy	Description
Blue/Green	A staged deployment : create a <i>new, separate</i> production environment (“Green”) for the new version, without touching the current one (“Blue”). A router switches traffic from Blue to Green once the new version is validated. Advantage : roll-back is easy, just switch the router back to Blue instantly.
Canary	A phased/incremental rollout : slowly roll out the change to a small subset of users first (randomly, or based on profile/demographics/region), before releasing to everyone. Drawback : the team must manage multiple live instances at once.
Versioned	Keep all versions alive simultaneously and let users choose which version to use; if a user updates, they are routed to the new version. Drawback : the team must maintain multiple versions indefinitely.

EXAM TIP

The fastest way to tell these apart: **Blue/Green** → keyword “instant rollback” or “switch everyone back at once.” **Canary** → keyword “small subset of users first, then everyone” (a *single* rollout that completes over time). **Versioned** → keyword “users *choose*” or “multiple versions kept alive *indefinitely*” (not a temporary rollout, a permanent standing choice).

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q19/Q21). A team deploys a major update to a new server setup; if anything breaks, they want to instantly switch all users back to the older version. **Answer. Blue/Green Deployment**, “instantly switch back” is precisely its headline advantage (easy rollback).

WORKED EXAMPLE

Scenario (adapted from Jan 2026 Q5). A company releases a new website version to only 5% of users first, monitors its behaviour, then rolls it out to everyone. **Answer. Canary Deployment**, a small initial subset, monitored, before a full rollout.

X COMMON MISTAKE

Watch for close-sounding wrong options like “Red/Black deployment”, “Shadow deployment”, or “Incremental deployment” appearing as tempting-but-wrong distractors in place of Canary or Blue/Green, these are *not* the three named strategies taught in this course; if a scenario clearly matches the *description* of Canary or Blue/Green, pick the correctly-named option even if a plausible-sounding alternative name is offered.

9.3 Deployment Hosting

★ FREQUENTLY ASKED IN PYQs

Matching a scenario to Bare Metal / IaaS / PaaS based on which layers the provider controls is tested very frequently and precisely. ★★★

► MOTIVATION

Deployment strategies describe *how* a new version reaches users. Hosting is a separate question: *where*, physically and administratively, does the running application actually live?

DEFINITION

Hosting is the infrastructure required to host applications, usually on server systems; the most important characteristic is **no unplanned downtime**. There are three hosting options, distinguished by how much of the stack the *provider* supplies versus how much the *client* must manage.

	Bare Metal	IaaS	PaaS
	Web Application	Web Application	Web Application
	Web Server	Web Server	Web Server (provider)
	Operating System	Operating System	Operating System (provider)
	Server Hardware	Server Hardware (provider)	Server Hardware (provider)

Option	Description
Bare Metal Servers	The client purchases the actual server hardware outright. Advantage: highest performance. Disadvantages: most expensive upfront cost; significant time/effort to set up and maintain.
Infrastructure-as-a-Service (IaaS)	The provider supplies only the hardware layer (e.g. an 8-core/32 GB server sliced into 8 virtual machines of 1-core/4 GB each). Examples: DigitalOcean, AWS, Linode. Advantages: cheaper, no hardware maintenance overhead. Disadvantages: each provider has its own configuration quirks; shared infrastructure can suffer performance issues.
Platform-as-a-Service (PaaS)	The provider supplies a full software layer as well, ready to deploy a web app onto directly. Examples: Heroku, Google App Engine. Advantage: very easy to deploy. Disadvantage: lack of control over the underlying stack.

📌 EXAM TIP

Ask exactly *one* question: **how much do I have to set up myself?** Everything, including buying physical hardware → **Bare Metal**. Just the software layer and app (hardware/VM handled for you) → **IaaS**. Nothing but the app itself (even the web server/OS are handled) → **PaaS**. “Software-as-

a-Service (SaaS)” is a commonly offered wrong-answer option in PYQs, it is *not* one of the three hosting options taught in this course (SaaS refers to using someone else’s finished *application*, e.g. Gmail, not hosting your own).

WORKED EXAMPLE

Scenario (adapted from Sep 2024 End Term Q5 / May 2025 Q37). A team wants the **highest possible performance** and **full control** of hardware and software, and does not mind a high upfront cost.

Answer. Bare Metal Servers, “full control” plus “highest performance, cost no object” rules out IaaS and PaaS (both hand over some control in exchange for convenience).

WORKED EXAMPLE

Scenario (adapted from Sep 2024 End Term Q5, second variant). A team wants scalability for fluctuating workloads and a pay-as-you-go model without managing hardware, but *does* want control of the software layer.

Answer. Infrastructure-as-a-Service (IaaS), hardware is handled by the provider, but the client still controls the software layer, ruling out PaaS (which also hands over the software layer).

9.4 Continuous Integration

★ FREQUENTLY ASKED IN PYQs

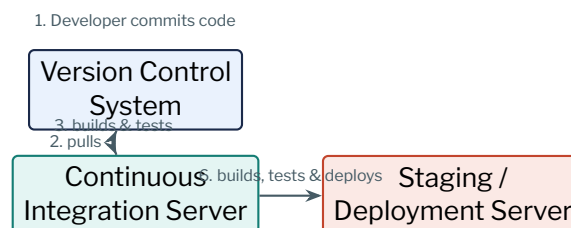
The seven best practices of Continuous Integration, and the true/false statements built around them, are tested frequently, usually as multi-select questions. ★★☆☆

► MOTIVATION

As more developers commit code more often, integrating everyone’s changes by hand becomes slow and error-prone, exactly the kind of problem automation exists to solve.

DEFINITION

Continuous Integration (CI) automates the integration and deployment of software: developers commit code to a **version control system**; a **CI server** (e.g. Jenkins, CruiseControl, Travis CI) pulls the new code, **builds** and **tests** it, and, if it passes, deploys it to a staging/deployment server.



DEFINITION

DevOps is the broader **cultural and professional movement** that CI is one concrete practice of: it aims to improve collaboration between software **development** and IT **operations** teams, breaking down the traditional wall between “writing the code” and “running it in production.”

Seven Best Practices for CI

1. Maintain a **single source repository** (not multiple repositories per version).
2. **Automate the build** (tools: Ant, Gradle, Builder).
3. **Make the build self-testing**, create automated tests; notify the developer when a test fails.
4. **Commit to the mainline every day**, infrequent commits increase the chance of conflicts; frequent commits encourage developers to break work into small chunks.
5. **Every commit builds the mainline** on an integration server that continuously monitors the repository.
6. **Fix broken builds immediately**, if needed, revert to the last known-good build.
7. **Automate deployment** via scripts.

X COMMON MISTAKE

Frequently tested false statements: “maintain *multiple* source repositories for each version” (false, should be a *single* repository), “avoid frequent commits, commit only for major updates” (false, commit to mainline *every day*), and “Continuous Deployment (CD) is synonymous with Continuous Integration” (false, CI is specifically about integrating/testing code frequently; CD, automatically deploying it, is a related but distinct practice).

Benefits and Drawback

- Reduces deployment time; avoids last-minute confusion/rush at release dates.
- Beneficial to users (can see an initial prototype early; sales teams can demo it).
- Beneficial to developers (pushes them toward writing modular code).
- **Drawback**: significant **initial effort** required to set up the process.

EXAM TIP

“No initial effort required to set up the process” and “discourages modular coding” are both **false** statements about CI’s benefits, frequently offered as wrong-answer distractors. The true drawback is the *opposite*: CI needs real upfront investment to set up, but then it *encourages* modular code.

9.5 Performance Optimisation and Monitoring

★ FREQUENTLY ASKED IN PYQs

Matching a scenario to Caching / Task Queues / Page-loading optimisation / Monitoring is tested consistently across many papers. ★★☆☆

> MOTIVATION

Once deployed, a system's job isn't done, it must stay fast and someone must watch what it's actually doing.

9.5.1 Three Ways to Improve Performance

Technique	Description
Caching	Store the results of common operations in memory , so repeat requests are served from memory instead of hitting the database every time. Tools: Memcached , Redis . Levels of caching: web browser, web server (page cache), database query cache.
Task Queues	For resource/time-consuming tasks (bulk emails, bulk DB updates): an asynchronous work queue executes them outside the normal HTTP request-response cycle, using a queue of jobs plus a pool of workers that pull jobs from it.
Loading & Rendering	Quick page loads matter directly for business: <i>e.g.</i> AliExpress increased site speed by 30% in 2016 and saw a 10.5% jump in orders; Google slowed search results by half a second in 2006 and lost 25% of user requests. Techniques: minify code (strip unnecessary characters) and compress static pages (<i>e.g.</i> with Brotli, Gzip).

WORKED EXAMPLE

Scenario (adapted from Jan 2026 End Term Q7 / Sep 2025 Q18). A product page is requested by thousands of users; the team stores the result of this database query in memory so future requests are served faster.

Answer. Caching, “store results in memory so future requests are faster” is the textbook definition.

WORKED EXAMPLE

Scenario (adapted from Jan 2024 End Term Q51 / May 2025 Q58). A web app must send notification emails to thousands of customers daily; this is coded to run *outside* the normal HTTP request-response cycle so the app's responsiveness doesn't degrade.

Answer. Task Queue, “outside the HTTP request-response cycle” plus a bulk, time-consuming job is the signature.

✗ COMMON MISTAKE

Do not confuse Caching (serving *already-computed* results faster) with Task Queues (deferring *slow, bulk* work to run *asynchronously*). A PYQ trap describes minification/compression as increasing efficiency by “fetching pages from memory instead of the database”, that description is actually **caching**, not what minifying/compressing code does (which is about page *size*, not memory look-ups).

9.5.2 Monitoring

DEFINITION

Even a well-deployed system can develop issues *live*: CPU utilisation, memory utilisation, storage (consumed vs. free), network bandwidth/latency. **Monitoring tools** (e.g. **Lighthouse**) generate reports analysing page resources, suggesting optimisations, and checking SEO/accessibility metrics.

Monitoring Customer Behaviour

- **Clickstreams**: which sequences of pages users visit most.
- **Think time** / **dwell time**: how long a typical user stays on a given page.
- **Abandonment**: within a multi-step flow (e.g. Product page → Buy Now → Add address → Add payment → Pay), what percentage of users **abandon** rather than complete it.
- **Analytics tools**: typically implemented by embedding a small piece of JavaScript on every page of the site.

DEFINITION

A/B Testing, in a monitoring context, pairs naturally with **Canary deployment**: Version A (the current version, perhaps with known issues) is shown to some users, Version B (a new workflow) to others; the team then compares metrics, traffic, dwell time, sales, abandonment, to decide which version is better before a full rollout.

QUIZ-ORIENTED NOTES

- Four environments: Development (local) → Testing (integration) → Staging (pre-production, resembles prod) → Production (live).
- Three deployment strategies: Blue/Green (instant rollback), Canary (gradual subset rollout), Versioned (permanent user choice).
- Three hosting options: Bare Metal (everything yourself) → IaaS (hardware handled) → PaaS (hardware + software layer handled). “SaaS” is a distractor, not a taught hosting option here.
- CI: single repo, automate build, self-testing build, commit daily, every commit builds on integration server, fix broken builds immediately, automate deployment.
- Caching = pre-computed results from memory. Task Queues = deferred async bulk work. Minify/Compress = smaller page payloads.
- Monitoring: clickstreams, dwell time, abandonment; A/B testing pairs with Canary deployment.

◆ MEMORY TRICK

D-T-S-P for environments: **D**evelopment is on my laptop, **T**esting checks integration, **S**taging is a dress rehearsal, **P**roduction is opening night. **B-C-V** for deployment: **B**lue/Green flips a switch, **C**anary tests the waters with a few, **V**ersioned lets everyone pick their own.

END OF CHAPTER 9: QUICK REVISION

REMEMBER THIS

One-page formula sheet.

- **Environments:** Development → Testing → Staging → Production.
- **Deployment strategies:** Blue/Green (new full env, easy rollback), Canary (gradual subset), Versioned (permanent multi-version, user chooses).
- **Hosting:** Bare Metal (buy hardware) < IaaS (hardware provided) < PaaS (hardware + software layer provided).
- **CI:** automates build/test/deploy from commits; 7 best practices; drawback = upfront setup effort.
- **Performance:** Caching (memory, repeat reads), Task Queues (async bulk work), Minify/-Compress (smaller payloads).
- **Monitoring:** resource metrics, clickstreams/dwell time/abandonment, A/B testing (pairs with Canary).

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. “Instant rollback to older version” ⇒ Blue/Green. “Small subset first, then everyone” ⇒ Canary. “Users choose, multiple versions kept alive” ⇒ Versioned.
2. “Full control + highest performance + cost no object” ⇒ Bare Metal. “Hardware handled, software layer still yours, pay-as-you-go” ⇒ IaaS. “Just deploy the app, everything else handled” ⇒ PaaS.
3. CI: single repo (not multiple), commit *daily* (not rarely), CD ≠ CI.
4. “Serve repeat requests from memory” ⇒ Caching. “Run slow bulk jobs outside the request cycle” ⇒ Task Queue.
5. Staging *resembles* production but is not live; don’t confuse the two.

PRACTICE QUESTIONS

- Q1.** A company wants to test a risky new checkout flow on 2% of users before a full launch. Which deployment strategy fits, and what should they monitor to decide whether to proceed?
- Q2.** Compare Blue/Green and Canary deployment along one dimension: how quickly can you fully undo a bad release?
- Q3.** A startup wants to deploy quickly with minimal DevOps effort and doesn't care about fine-grained server control. Which hosting option fits, and why?
- Q4.** List the seven CI best practices from memory, then check your list against Section 9.4.
- Q5.** A news website wants to reduce load time for its homepage. Suggest two concrete techniques from this chapter and explain how each helps.
- Q6.** Explain the difference between Caching and a Task Queue using an example that is *not* from this book.
- Q7.** Why does A/B testing naturally pair with Canary deployment rather than Blue/Green?

Other Aspects in Software Engineering

In this chapter (Week 11): Software organisations and roles · Communication, collaboration, and conceptual integrity · Knowledge-sharing tools · Productivity · What makes a great software engineer · Decision making.

10.1 Software Organisations

★ FREQUENTLY ASKED IN PYQs

Matching a described responsibility to the correct organisational role is tested very frequently and consistently across the End Term papers. ★★★

► MOTIVATION

Every chapter so far has focused on activities: gathering requirements, designing, coding, testing, deploying. But none of this happens in a vacuum, real software is built inside a **software company**, staffed by many roles beyond “developer.” Understanding who does what is essential both for working effectively in a team and for correctly answering scenario-based exam questions that describe a role without naming it.

DEFINITION

The **purpose of a software company** is to provide **value** to potential users of the software system. Achieving this requires far more than engineers writing code.

Role	Responsibilities
Marketing Team	Look for opportunities to provide value in the market; conduct market research; identify target audiences.
Product Managers	Work closely with marketing and sales; understand business goals (how the product maximises ROI); know the technology stack well enough to understand effort involved and take technical decisions; passionate about the user and what the user wants.
Designers	User Experience (UX) roles (recall Chapter 3); transform requirements into solutions; talk to users and create prototypes.
Software Engineers	Write code, together with others on the team, to implement requirements. Notably, engineers don't usually decide what product is built or what problem it solves.
Engineering Managers	In large organisations, transmit information between higher and lower levels; also called project managers : organise/prioritise work, coordinate between teams, resolve interpersonal conflict between engineers.
Sales Team	Sell the product to the users marketing identified; feed back user feedback to marketing, product, and design teams, which engineers then address.
Support Team	Resolve problems that clients have; feed back defects/shortcomings to product, design, and engineering.
Data Scientists	Analyse data generated by different teams/users to help the organisation make better decisions, e.g. helping marketing analyse business data, tracking sales targets, helping engineers understand app-usage patterns.
Ethics and Policy Specialists	Background in law, social science, or policy; shape terms of service, software licences, privacy policy; especially important for any company that works with data.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q22). A customer reports being unable to track orders in an app; the query is first handled with basic FAQs and scripted responses, before being escalated further.

Answer. Support Team, the first line of response to client-reported problems, before escalation, is exactly the support team's role.

WORKED EXAMPLE

Scenario (adapted from Jan 2026 End Term Q3). Before writing any code for a new Seller Portal, a team studies competitors, identifies what sellers need, and checks market demand.

Answer. Marketing Team, "studying competitors and checking market demand, before any code is written" is squarely market research.

X COMMON MISTAKE

Product Managers and Engineering Managers are the pair most often confused. **Product Managers** decide *what* gets built and why (business goals, ROI, user needs), working closely with marketing/sales. **Engineering Managers** decide *how* the team's work gets organised and coordinated (scheduling, conflict resolution), much closer to the “project manager” role from Chapter 4. If a scenario is about *prioritising features for business value*, it's Product Manager; if it's about *coordinating people and resolving team conflict*, it's Engineering Manager.

EXAM TIP

Fast keyword matching: “market research / target audience” → Marketing. “ROI / business goals / technical decisions” → Product Manager. “Prototypes / talk to users / UX” → Designers. “Write code” → Software Engineers. “Coordinate teams / resolve conflict” → Engineering Manager. “Sell to identified users” → Sales. “Resolve client problems” → Support. “Analyse data for decisions” → Data Scientists. “Terms of service / privacy policy / licenses” → Ethics and Policy Specialists.

10.2 Communication, Collaboration, and Productivity

> MOTIVATION

A software company might have every role in Section 10.1 filled with talented people, and still fail, if those people cannot communicate and stay aligned on what is actually being built.

DEFINITION

Conceptual integrity means everyone on a team has the **same understanding** of what is being built and why. **Effective communication** is what ensures conceptual integrity is maintained as a team and product grow.

10.2.1 Knowledge-Sharing Tools

★ FREQUENTLY ASKED IN PYQs

The purpose and benefits of knowledge-sharing tools, and identifying which tool serves which purpose, are tested repeatedly. ★★☆☆

Tool type	Purpose / Examples
Issue Trackers	Track different issues and maintain a history of who worked on them . Examples: JIRA, Pivotal Tracker .
Code Hosting	Many libraries/frameworks are hosted (and documented) on GitHub , e.g. Vue.js.
Chat / Messaging	E.g. Slack , for day-to-day team communication.
Q&A Communities	E.g. Stack Overflow , helps resolve issues and links to further learning resources.

DEFINITION

The main purpose of knowledge-sharing tools is **sharing documents and archiving decisions**. Their importance: knowledge from one project may be needed in another, and is only useful later if it was **documented and archived properly**. When people leave an organisation, their specialised knowledge leaves with them, unless it was captured; a further mitigation for this risk is **cross-training**, deliberately rotating developers between projects so knowledge isn't siloed in one person.

WORKED EXAMPLE

Scenario (adapted from Sep 2025 End Term Q21). A team uses GitHub, JIRA, and Slack; a new intern wants to know why a certain function exists, but no documentation is available.

Answer. **Archiving code decisions in issue trackers** is the practice that would have prevented this gap, the whole point of a knowledge-sharing tool is that decisions, once made, are written down somewhere a future teammate can find them.

EXAM TIP

Do not confuse a **Knowledge Sharing Tool** (JIRA, Slack, GitHub, Stack Overflow, documents decisions and history) with a plain **Development Tool** (an IDE like VS Code or IntelliJ IDEA, helps you write code, but doesn't archive team decisions). A PYQ may ask "what type of tool is an IDE?", the correct answer is **Development Tool**, not Knowledge Sharing Tool.

10.2.2 Productivity

DEFINITION

Productivity is traditionally defined as *work done per unit time*. This is **difficult to define** cleanly in software engineering, it is **not** simply "number of lines of code" (recall from Chapter 6, Section 6.4, that raw LOC is a weak proxy for quality, and the same is true for productivity). For example, a developer who delivers only their own assigned features while ignoring teammates' needs can actually **harm the team's overall productivity**, even while looking individually "productive."

Right tools improve productivity: **project management tools** (e.g. JIRA, Pivotal Tracker) give the whole team both a big-picture and detailed view of progress; **development tools** (IDE features) help individual developers work faster.

QUIZ-ORIENTED NOTES

- Conceptual integrity = shared understanding across the team, maintained via effective communication.
- Knowledge-sharing tools: Issue trackers (JIRA, Pivotal Tracker), GitHub, Slack, Stack Overflow, purpose = document/archive decisions.
- Cross-training mitigates knowledge loss when people leave.
- Productivity $\neq$ lines of code; individually "productive" work can still harm team productivity.

10.3 What Makes a Great Software Engineer? Decision Making

★ FREQUENTLY ASKED IN PYQs

Distinguishing macro decisions from micro decisions is tested precisely and repeatedly. ★★☆☆

► MOTIVATION

Beyond writing correct code, engineers constantly make judgement calls, some small, some with company-wide consequences. Recognising the *scale* of a decision, and following a disciplined process for making it, is itself a skill.

DEFINITION

Macro decisions are made at the **design and architecture** level, e.g. which libraries or frameworks to use for a whole system (recall Chapter 7's architecture styles and design patterns). **Micro decisions** are made at a much smaller scale, e.g. which algorithm or data structure to use for one particular module.

WORKED EXAMPLE

Scenario (adapted from Sep 2023 End Term Q27). Situation 1: deciding which **software architecture** to use for a system. Situation 2: deciding which **data structure** to use for a specific module.

Answer. Situation 1 is a **macro decision** (architecture-level, affects the whole system); Situation 2 is a **micro decision** (affects one module).

🔑 EXAM TIP

Quick test: does this decision affect the *whole system's structure* or *how multiple modules interact*? → Macro. Does it only affect the internal implementation of *one* module or function? → Micro. This maps directly onto the Design vs. Development distinction from Chapter 5: architecture/design-level choices are macro, implementation-level choices are micro.

DEFINITION

The **Rational Decision-Making Process** has five steps: (1) **Identify** the decision to be made. (2) **Systematically identify alternatives**. (3) **Think through** potential outcomes for each alternative. (4) **Evaluate** which outcome is best for the given context. (5) **Make** the decision.

■ REMEMBER THIS

Both macro and micro decisions benefit from the same five-step rational process, what changes with scale is not *how* you decide, but how much is riding on getting it right, and how many people the decision affects.

QUIZ-ORIENTED NOTES

- Macro decision = architecture/design-level (affects whole system). Micro decision = implementation-level (affects one module/function).

- Rational decision-making: Identify → Alternatives → Outcomes → Evaluate → Decide.

◆ MEMORY TRICK

Roles, A to Z of a launch: Marketing finds the opportunity → Product Managers decide what to build → Designers shape how it feels → Software Engineers build it → Engineering Managers keep the team on track → Sales sells it → Support keeps users happy → Data Scientists tell everyone what's really happening → Ethics & Policy keeps it all lawful.

END OF CHAPTER 10: QUICK REVISION

■ REMEMBER THIS

One-page formula sheet.

- **Purpose of a software company:** provide value to users.
- **Nine roles:** Marketing (research), Product Managers (business goals/ROI), Designers (UX/prototypes), Software Engineers (build), Engineering Managers (coordinate/resolve conflict), Sales (sell + feedback), Support (resolve client issues + feedback), Data Scientists (analyse data), Ethics & Policy (terms of service/privacy/licenses).
- **Conceptual integrity** = shared team understanding, maintained through communication.
- **Knowledge-sharing tools:** Issue trackers (JIRA, Pivotal Tracker), GitHub, Slack, Stack Overflow; purpose = document/archive; cross-training mitigates knowledge loss.
- **Productivity** ≠ LOC; right tools (project management + dev tools) help.
- **Macro decision** = architecture-level. **Micro decision** = module-level. **Rational process:** Identify → Alternatives → Outcomes → Evaluate → Decide.

QUIZ PREPARATION

QUIZ-ORIENTED NOTES

1. Product Manager (business goals/what to build) vs. Engineering Manager (team coordination/how work gets organised), do not swap them.
2. “First point of contact for a client problem, before escalation” ⇒ Support Team, not Product Management.
3. An IDE is a Development Tool, not a Knowledge Sharing Tool.

4. Architecture/framework choice $\Rightarrow$ macro decision. Data structure/algorithm choice for one module $\Rightarrow$ micro decision.
5. Productivity is not lines of code; ignoring teammates while hitting your own targets can hurt team productivity.

PRACTICE QUESTIONS

- Q1.** A company is deciding whether to build a mobile app or a web app for a new product, and separately, which sorting algorithm to use inside one reporting module. Classify each decision as macro or micro.
- Q2.** Which role would you approach to understand why a feature was prioritised over another, and which role would you approach to understand how a particular class was implemented?
- Q3.** Explain how cross-training reduces the risk described under “knowledge-sharing tools.”
- Q4.** Apply the five-step rational decision-making process to choosing between two possible databases for a new project (in words, no need to actually decide).
- Q5.** A support ticket reveals a recurring product complaint. Trace, using the roles in Section 10.1, how this feedback should flow through the organisation.